

Trustfile: A File security platform that scans and protects files before access

In partial fulfillment for the requirements in Capstone 1 and Research Subject

Taño, Joshua- BSIT 4
Aportadera, Reymart- BSIT 4
Corcino, Eduardo- BSIT 4
Dejaro, Rencel- BSIT 4
Lemeric, Ma. Robella Beatrice- BSIT 4

Cyber Security Track

Second Semester
2025-26

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION.....	3
Background of the Study.....	3
Statement of the Problem.....	10
Objectives of the Study.....	16
Scope and Delimitation.....	23
Significance of the Study.....	29
Definition of Terms.....	34

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES.....	54
Foreign Studies.....	54
Local Studies.....	64
Related Literature.....	67
Synthesis of the Reviewed Literature.....	75

CHAPTER 3

METHODOLOGY.....	85
Research Design.....	85
System Development Methodology.....	91
System Architecture.....	97
System Preview and Manual.....	102
Tools and Technologies Used.....	115
Data Gathering Techniques.....	120
Testing Procedures.....	124
Evaluation Criteria.....	130

SUPPORTING DIAGRAMS.....	138
Data Flow Diagram (DFD – Level 0).....	138
Entity Relationship Diagram (ERD).....	
Use Case Diagram.....	
References.....	
Appendices.....	

CHAPTER 1

INTRODUCTION

Background of the Study

The rapid growth of information technology has changed the way people, businesses, schools, and government offices create, store, and share information. Digital files are now part of almost every daily task — used for communication, record-keeping, collaboration, and storage. Common formats such as PDF, DOCX, XLSX, PPTX, ZIP/RAR archives, images, audio, video, and executable files move constantly through email, cloud storage, USB drives, websites, and messaging platforms. This convenience has boosted productivity across sectors, but it has also opened new doors for cybercriminals to distribute malicious files disguised as ordinary documents.

Organizations receive large volumes of files every day from clients, employees, suppliers, and partners — many containing confidential records, financial data, contracts, and other sensitive information. This constant exchange makes offices, agencies, and businesses attractive targets, since attackers can easily hide malicious code inside files that look completely legitimate, leaving users with little way to tell a safe file from a dangerous one just by looking at it.

The growing dependence on digital files has also changed the way organizations manage their day-to-day operations. Processes that were once completed using paper documents are now almost entirely digital, allowing information to be accessed, updated, and transmitted within seconds. Educational institutions distribute learning materials through learning management systems, businesses exchange financial records through cloud-based platforms, healthcare providers maintain electronic medical records, and government agencies process applications and official documents

electronically. This shift toward digital transformation has improved efficiency, reduced operational costs, and enhanced collaboration among different departments and organizations. However, it has also increased the number of digital assets that require protection, making cybersecurity an essential component of modern information management.

As organizations continue to digitize their operations, the volume of files exchanged every day continues to increase exponentially. Employees frequently download reports, invoices, spreadsheets, multimedia presentations, compressed archives, and software installers from various internal and external sources. While many of these files originate from trusted senders, others may come from unfamiliar websites, third-party vendors, or unknown email addresses. The challenge lies in the fact that users often cannot distinguish between a legitimate file and one that has been intentionally modified to contain malicious code. Even experienced computer users may unknowingly interact with infected files because modern cyber threats are specifically designed to imitate authentic documents with remarkable accuracy.

Another important concern is that organizations increasingly rely on interconnected digital environments. Cloud computing, remote work arrangements, collaborative online platforms, and mobile devices have enabled employees to access files from virtually anywhere. Although these technologies provide significant operational advantages, they also broaden the attack surface available to cybercriminals. A single infected file uploaded to a shared cloud storage service or collaborative workspace may quickly spread to multiple users, departments, or even partner organizations if appropriate security measures are not in place. Consequently, the security of individual files has become directly connected to the overall security posture of an organization.

The increasing sophistication of cyberattacks has demonstrated that protecting digital systems requires more than simply securing networks and servers. Individual files themselves have become one of the most common vehicles through which malicious software enters organizational environments. Every downloaded attachment, shared

document, archived file, or executable application represents a potential cybersecurity risk if it is not properly verified before use. For this reason, organizations are placing greater emphasis on implementing preventive measures that can identify suspicious files before they are opened, thereby reducing the likelihood of malware infections, unauthorized access, and costly security incidents.

The malware threat itself has grown far more advanced over time. What began as simple viruses has evolved into a wide range of threats — worms, Trojan horses, ransomware, spyware, adware, rootkits, and keyloggers — each with its own way of infecting a system. Viruses spread when infected programs are run; worms move across networks on their own; Trojans hide malicious behavior inside what looks like a normal application; ransomware locks up files until a ransom is paid; and spyware quietly harvests passwords and financial information in the background. The damage these attacks cause is no longer just technical — it's financial and reputational. According to IBM's 2024 Cost of a Data Breach Report, the global average cost of a data breach reached US\$4.88 million in 2024, a 10% jump from the year before and the largest single-year increase since the pandemic began [1]. That increase was driven largely by business disruption and the long tail of post-breach recovery costs — not just the initial intrusion itself [1].

The continuous evolution of malware has also been driven by the rapid advancement of technology and the increasing availability of cyberattack tools. In the past, developing malicious software required extensive technical expertise and programming knowledge. Today, however, cybercriminals can obtain ready-made malware kits, exploit frameworks, and attack tools through underground marketplaces and cybercrime-as-a-service platforms. These resources significantly lower the barrier to entry, allowing even individuals with limited technical skills to launch sophisticated cyberattacks. As a result, malware campaigns have become more frequent, more organized, and more difficult to defend against, posing a growing challenge to organizations of all sizes.

Another factor contributing to the increasing prevalence of malware is the expanding digital ecosystem. Modern organizations rely on numerous interconnected systems, including cloud computing services, web applications, mobile devices, Internet of Things (IoT) devices, and remote access technologies. While these innovations improve operational efficiency and business continuity, they also create additional entry points that attackers can exploit. A single compromised device or malicious file introduced into one part of the network may provide attackers with an opportunity to move laterally across connected systems, ultimately affecting multiple departments and disrupting critical organizational operations. This interconnected nature of modern information systems highlights the importance of detecting malicious files before they can interact with other digital assets.

Furthermore, cybercriminals continually adapt their techniques to evade existing security controls. As antivirus software and security technologies improve their detection capabilities, attackers develop new methods to bypass these defenses through obfuscation, encryption, code mutation, and social engineering strategies. This ongoing cycle of attack and defense has created a constantly evolving cybersecurity landscape in which organizations must continuously update their protective measures. Traditional security solutions that rely solely on previously identified malware signatures often struggle to recognize newly emerging threats, emphasizing the need for complementary detection approaches that analyze the characteristics and behavior of files rather than relying exclusively on known malware databases.

Beyond financial losses, malware attacks can have lasting consequences on an organization's reputation and public trust. Customers, clients, students, and business partners expect organizations to protect the information entrusted to them. When confidential data is compromised due to a malware infection, affected organizations may experience reduced customer confidence, negative public perception, legal liabilities, and increased regulatory scrutiny. Recovering from such incidents often requires not only technical remediation but also extensive communication efforts, policy revisions, employee retraining, and investments in stronger cybersecurity infrastructure.

These indirect consequences may continue long after the malware itself has been removed, demonstrating that cybersecurity incidents have organizational impacts extending far beyond immediate technical damage.

The growing complexity of malware has therefore shifted the focus of cybersecurity from simple threat removal toward proactive risk management. Rather than responding only after malicious software has infected a system, cybersecurity professionals increasingly emphasize preventive measures that identify potential threats before users interact with suspicious files. Preventive approaches, such as file verification, threat intelligence integration, static analysis, heuristic evaluation, and anomaly detection, allow organizations to reduce security risks during the earliest stages of a potential attack. These strategies support a more resilient cybersecurity posture by helping users recognize suspicious files before they compromise sensitive systems or valuable organizational information.

Attackers rarely need to break through technical defenses when they can simply get a person to open the wrong file. Phishing emails disguised as invoices, job offers, shipping notices, or official government correspondence remain one of the most common ways malware gets in, relying on urgency, curiosity, or authority to lower a user's guard. This problem escalated sharply during the COVID-19 pandemic. INTERPOL reported a 569% increase in malicious domain registrations between February and March 2020 alone, many of them built around COVID-19 themes to exploit public fear and confusion [2]. Separately, phishing volumes tied to remote work were reported to have risen by as much as 600% during the same period, as employees shifted to less secure home networks and personal devices almost overnight [3]. The pandemic didn't invent the file-based malware problem, but it made painfully clear how quickly attackers can scale it when the right conditions — fear, urgency, and a distracted, dispersed workforce — line up.

Commercial antivirus software remains the most common line of defense, but it comes with a real limitation: it usually functions as a "black box." A file gets flagged, a threat name appears, and that's often the extent of what the user is told — with little explanation of why the file was flagged or what specifically made it dangerous. Much of this comes down to how traditional antivirus tools work. Most rely on signature-based detection, comparing a file against a database of known malware signatures. This approach is fast and reliable for threats that have already been identified and catalogued — but it is structurally unable to catch zero-day malware, or threats that are new, modified, or intentionally disguised to avoid matching any known signature [4], [5]. Malware authors exploit this gap deliberately, using polymorphic and metamorphic techniques that alter a file's code just enough to slip past signature matching entirely [4].

Modern malware also goes out of its way to look harmless. Attackers use file extension spoofing, double extensions, packing, encryption, and steganography to bury malicious code inside what appears — visually and structurally — to be an ordinary document or media file. An executable can be renamed to resemble a PDF; a malicious script can be tucked inside an Office document or an image file without any visible sign of tampering. One of the more established techniques for catching this kind of disguise is entropy analysis — measuring how "random" the data inside a file is, since encrypted or packed content tends to have a much higher entropy than typical plain text or standard file structures [6]. Lyda and Hamrock's foundational work on this technique, published through IEEE Security & Privacy, showed that entropy analysis lets analysts flag packed or encrypted malware quickly, without needing to fully reverse-engineer the file first [6] — a principle later studies have continued to build on using statistical and machine-learning models across different file types [7].

What all of this points to is a gap that goes beyond any single piece of software: users are rarely equipped to make an informed judgment about a file's safety on their own. Cybersecurity training in schools and workplaces tends to focus on password hygiene, phishing awareness, and keeping software updated — but rarely touches on practical,

file-level analysis concepts like hashing, entropy, or file-type verification. As a result, most users are left entirely dependent on whatever their antivirus tool decides to tell them, with no way to independently reason about a file before deciding whether to open it.

This is the gap that TrustFile, the system developed in this study, was built to close. Rather than functioning purely as an automated gatekeeper, TrustFile was designed to combine detection with explanation. It analyzes uploaded files using a combination of static analysis techniques — cryptographic file hashing to catch known malicious files by their digital fingerprint, entropy analysis to flag encrypted or packed content, script detection to surface embedded executable code, file-size anomaly checks, and file-type verification to confirm that a file's extension actually matches its internal structure. These indicators are not evaluated in isolation; an AI analysis layer weighs them together to produce a single, coherent assessment, while a heuristic analysis component looks for structural and behavioral patterns associated with malware that a pure signature match would miss entirely — including previously unseen threats [4], [5]. The system also draws on external malware-intelligence APIs, giving it access to continuously updated global threat databases so that local analysis is reinforced by real-world detection data from the wider security community.

What distinguishes TrustFile most, though, is that every scan result comes with a plain-language explanation of why a file was classified the way it was — what indicators were found, what they mean technically, and what risk they carry — written to be understood by technical and non-technical users alike. The goal is for TrustFile to function not only as a detection tool but as a quiet, ongoing form of cybersecurity education: each scan is a chance for a user to understand a little more about what makes a file dangerous, rather than simply being told to trust or distrust it.

Built to handle a wide range of file types and sizes — documents, archives, executables, images, and multimedia — TrustFile is aimed squarely at the environments where this problem is most acute: offices, government agencies, schools, and

businesses that handle a constant stream of incoming files as a normal part of daily operations. By pairing multiple static analysis techniques with AI-driven reasoning, heuristic analysis, and external threat intelligence — and by explaining its findings rather than concealing them — TrustFile addresses both the technical and the educational gaps left open by conventional antivirus solutions, giving users the tools to make informed decisions before a file is ever opened.

Statement of the Problem

The growing reliance on digital technologies has changed how organizations and individuals manage, store, and exchange information. Digital files — documents, spreadsheets, presentations, compressed archives, executable programs, and multimedia — now move constantly between internal and external sources as part of ordinary business. That convenience comes at a cost: malicious files disguised as legitimate ones remain one of the most effective tools cybercriminals use to compromise systems, steal data, and spread malware across networks.

Over the course of this study, the researchers observed a problem that sits at the center of almost every file-based attack: most users simply cannot tell, on their own, whether a file is safe before they open it. Unlike a physical object, a digital file hides its actual contents. What a user sees — a name, an icon, an extension — is not necessarily what the file is. Any of those visible cues can be faked, while the code that actually matters stays buried inside the file's internal structure, completely out of view. This problem is made worse by a default setting most users don't even know exists: Windows hides known file extensions by default, meaning a file named `invoice.pdf.exe` displays to the user as simply `invoice.pdf` [1]. A user scanning their downloads folder has no visual way of knowing they're looking at an executable rather than a document — the operating system itself is quietly doing the disguising. Left without a reliable way to verify a file, most users fall back on trust, habit, or assumption — and that gap is exactly what attackers exploit.

The researchers also found that commercial antivirus software, while a necessary layer of protection, tends to prioritize automated detection over user understanding. Most tools notify users that "a threat was detected," attach a generic malware name, and stop there — offering no real explanation of what specifically made the file dangerous. This opacity has a quieter cost beyond the individual scan: it keeps users permanently dependent on the software's verdict rather than helping them build any independent judgment of their own. Over time, a user who has only ever been told "yes" or "no" learns nothing about why, and is no better equipped to evaluate the next suspicious file that slips past the software entirely.

That gap becomes especially dangerous because the underlying detection method most antivirus software relies on — signature-based detection — has a structural blind spot. It works by comparing a file against a database of previously identified malware fingerprints, which makes it fast and reliable against known threats but essentially unable to recognize anything new. Zero-day malware, newly modified variants, and polymorphic code that deliberately changes its own signature with every infection can all slip through simply because no matching signature exists yet [2]. Heuristic analysis was developed to close part of this gap by examining a file's structure and behavior for suspicious patterns instead of requiring an exact match — but it introduces a different tradeoff. Heuristic systems are inherently prone to false positives, sometimes flagging legitimate software as malicious simply because its structure resembles patterns associated with threats [3]. Too many false alarms erode user trust in the system, while rules loose enough to avoid them often let genuinely malicious files through. Attackers have also learned to work around heuristics directly: packing and encrypting a payload artificially raises the file's entropy in ways that can mimic, or hide behind, patterns heuristic engines are tuned to expect [3]. This back-and-forth between evasion and detection is precisely why relying on any single check in isolation is no longer sufficient, and why an approach that weighs multiple indicators together — file hashing, entropy, script content, size, and type — has a real advantage over any one method alone.

File size is one of those indicators that is easy to overlook but often quietly revealing. Malware is frequently appended to an otherwise legitimate file, inflating it beyond what is normal for its format; conversely, an unusually small file can indicate a stub loader — a lightweight file whose only job is to download the actual malicious payload after execution. Neither pattern is something a user would ever notice by eye, and building a system that can flag it reliably requires accounting for the fact that "normal" file size varies significantly across formats, compression levels, and embedded content.

Beyond the technical detection gap, the researchers identified a deeper problem: most existing systems don't explain themselves at all. The typical scanning process functions as a black box — a file goes in, a verdict comes out, and the reasoning in between stays hidden. Users are never shown which indicators were examined, what a given entropy value or script detection actually means, or how confident the system is in its own determination. Left in the dark, users have no way to build the kind of pattern recognition that would help them catch a suspicious file on their own next time, nor any way to sanity-check the software when a result seems questionable — say, a personal file wrongly flagged, or a borderline case where the "safe" verdict doesn't quite match their own instinct. That opacity has consequences that reach beyond any one person's scan results. Within an organization, employees who understand what actually makes a file risky become more careful with how they handle attachments and downloads generally, and are more likely to flag something suspicious to IT before it becomes a wider incident. Transparent systems also make it easier to justify exceptions when a legitimate business file gets flagged, instead of leaving staff to either blindly override a warning or blindly obey it. Producing that kind of explanation is not a trivial add-on, either — describing a hash mismatch, an entropy spike, or an embedded script in a way that is accurate without either dumbing it down into meaninglessness or burying it in jargon a non-technical employee can't use is a genuine design challenge in itself.

The increasing dependence on digital documents in everyday operations has made file security an essential concern for organizations across various sectors. Educational institutions, government agencies, healthcare providers, financial institutions, and private businesses routinely exchange hundreds or even thousands of digital files every day. These files often contain confidential information, financial records, legal documents, research data, and other sensitive information that must be protected from unauthorized access or malicious manipulation. Despite the widespread use of antivirus software and cybersecurity awareness programs, many organizations continue to experience malware-related incidents because users frequently encounter files whose safety cannot be determined through visual inspection alone. This situation highlights the need for solutions that not only detect threats but also help users understand why a file may be considered suspicious.

Although existing cybersecurity tools provide valuable protection against known threats, many of them prioritize automated detection over user understanding. In most cases, users receive only a notification indicating whether a file is safe or malicious, accompanied by a technical threat name that may be unfamiliar to non-technical individuals. Such limited feedback often prevents users from learning about the characteristics of malicious files or understanding the reasoning behind the system's assessment. Consequently, users remain heavily dependent on automated software and may struggle to make informed decisions when encountering unfamiliar files outside the protection of their security applications.

Another challenge lies in the continuously evolving nature of malware. Cybercriminals regularly develop new techniques that allow malicious files to evade traditional detection mechanisms through code obfuscation, encryption, file packing, and other methods intended to conceal harmful content. Because of these rapidly changing attack strategies, relying solely on signature-based detection may not provide sufficient protection against emerging or previously unknown threats. This creates a growing demand for intelligent file analysis systems capable of examining multiple

characteristics of a file rather than depending exclusively on known malware signatures. Such systems can provide users with a more comprehensive assessment of potential risks while supporting proactive cybersecurity practices.

Furthermore, cybersecurity is not solely a technical issue but also a human-centered challenge. Numerous studies have shown that human error remains one of the leading causes of successful cyberattacks. Even organizations equipped with advanced security technologies remain vulnerable when users unknowingly open malicious attachments or download infected files from untrusted sources. Strengthening users' ability to recognize suspicious files and understand the factors that contribute to cybersecurity risks is therefore an important aspect of organizational security. Providing clear explanations alongside automated detection results enables users to become more informed participants in protecting digital information rather than relying entirely on software-generated decisions.

Recognizing these challenges, this study proposes the development of TrustFile, an intelligent file verification system designed to combine multiple static analysis techniques, heuristic analysis, artificial intelligence-assisted assessment, and external malware intelligence into a unified file evaluation process. Rather than simply determining whether a file is malicious or safe, the system seeks to provide transparent explanations that help users understand the indicators identified during analysis and the level of risk associated with each file. Through this approach, the study aims to address both the technical limitations of traditional malware detection methods and the educational gap experienced by many computer users, thereby promoting safer decision-making before files are opened or executed.

Specifically, this study identifies the following problems:

1. Users cannot easily identify if a file contains malware before opening it.

Most users rely on visible cues — file names, icons, extensions, or a sender's reputation — none of which reliably reflect what a file actually contains. Attackers exploit this directly through extension spoofing, double extensions, and hidden executable code, techniques made easier by the fact that operating systems often hide extensions by default in the first place [1]. Actually identifying a malicious file requires understanding executable structures, embedded scripts, hashes, and metadata inconsistencies — technical knowledge well beyond what most users have, which leaves them relying on assumption rather than analysis.

2. Many systems rely on basic detection methods and may not detect new or hidden threats.

Signature-based detection cannot recognize malware it hasn't seen before, creating a window of exposure between a threat's emergence and the moment a signature is finally issued for it [2]. Heuristic analysis narrows that gap but introduces its own tradeoffs — false positives that erode trust, and evasion techniques (like packing and encryption) specifically designed to slip past it [3]. Abnormal file size, in either direction, is a further indicator that basic detection methods routinely miss entirely, despite how effective it can be as an early filter.

3. There is a lack of systems that explain how malware is detected.

Most security software returns a bare verdict — "threat detected" or "file clean" — without surfacing the reasoning behind it: which indicators were checked, what they meant, or how confident the system actually was. That opacity blocks users from developing their own judgment, makes it harder to sanity-check questionable results, and reduces trust in the system generally. Producing genuinely useful explanations also

isn't simple — it requires communicating technical findings accurately without oversimplifying them into uselessness.

4. Users have limited knowledge about identifying malicious files.

Most users know the basics — install antivirus, avoid sketchy sites — but few understand the practical file-analysis concepts security professionals rely on daily. This gap shows up as risky, habitual behavior: opening unverified files, ignoring warnings, and trusting appearances. Folding education directly into the scanning process, rather than treating it as a separate training exercise, is what allows a system to close this gap over time instead of just working around it scan by scan.

To address these problems, the researchers designed and developed TrustFile, a web-based file security platform that combines static analysis techniques, AI-driven analysis, heuristic analysis, and external malware intelligence to evaluate files before a user ever opens them — and that explains, in plain language, the reasoning behind every result it produces.

Objectives of the Study

General Objective

The general objective of this study was to design, develop, and evaluate TrustFile, a web-based file security platform that assessed the safety of digital files before user access by combining multiple static analysis techniques, rule-based heuristic analysis, and external malware intelligence services to determine each file's threat classification — paired with an AI-generated explanation layer that translated those technical findings into plain language. Beyond detection alone, the system was built to improve cybersecurity awareness through scan results that were transparent and genuinely understandable — intended for offices, government agencies, private organizations,

educational institutions, and businesses that regularly handle large volumes of incoming digital files.

This objective covered three things at once: building the system itself, making its detection genuinely intelligent, and using that detection process as an opportunity to teach. The design phase involved identifying TrustFile's functional and non-functional requirements, selecting the right technologies, mapping out the system architecture, and designing a web interface simple enough for non-technical users while still capable of secure file scanning. The implementation phase turned those specifications into a fully working web application. The evaluation phase then tested whether the finished system actually solved the problems it set out to solve — accurate detection, explanations that made sense, and a usable overall experience — rather than assuming success from the design alone.

What sets TrustFile apart from conventional antivirus software is when it acts. Most antivirus tools are reactive — they clean up an infection after it has already happened, or block a threat only once it has already matched a known signature. TrustFile, by contrast, was built around preventive security: evaluating a file before it is ever opened or executed, so that the decision to trust it is made with full information rather than after the fact. Instead of leaning on signature matching alone, the platform combines several detection techniques into one assessment, so that a file's fate isn't decided by any single check that a sufficiently clever piece of malware might slip past. The combination of AI-driven analysis, heuristic analysis, and external threat intelligence lets the system catch both known and previously unseen threats, while explaining, to the user, exactly what it found and why that mattered.

That explanatory layer is deliberate, not incidental. Rather than treating cybersecurity awareness as a separate training exercise the user has to seek out on their own, TrustFile builds it directly into the scanning experience itself. Every scan becomes a small, real teaching moment — showing the user why a file landed in a particular threat category and what specific indicators drove that decision. Over repeated use, this is

meant to leave users measurably better at recognizing suspicious files on their own, rather than simply more reliant on a tool they don't understand.

In practical terms, the organizations that stand to benefit most are the ones handling the highest daily volume of files: government offices, corporate environments, educational institutions, and private businesses that constantly move documents, spreadsheets, archives, multimedia, and executables through email, cloud storage, removable drives, and collaboration platforms. By offering an accessible, browser-based platform that evaluates files before they're ever opened, this study contributes to strengthening organizational cybersecurity practices at exactly the point where most file-based attacks succeed — the moment right before a user clicks "open."

Specific Objectives

Specifically, the study aims to:

1. To develop a web-based system that allows users to upload files for security scanning before access.

The first objective centered on building a browser-based platform that required no additional software installation — users could scan a file the same way they'd check an email, from any internet-connected device, regardless of operating system. This choice was deliberate: a web-based architecture sidesteps compatibility issues entirely, and lets updates be pushed centrally rather than requiring every user to manually download a new version each time the system improves.

The upload mechanism itself was built to transmit files securely over encrypted channels, and to accept the range of formats organizations actually deal with day to day — documents, compressed archives, executables, multimedia, and images — regardless of file size. Every file that comes through undergoes a full security

assessment before the user can access its contents, so the check always happens before exposure, not after.

Usability was treated as a first-class requirement rather than an afterthought. Features like drag-and-drop uploading, visible progress indicators, scan status notifications, and clearly laid-out result pages were built specifically so that someone with no cybersecurity background could still use the system confidently and understand what it was telling them.

2. Implement a rule-based heuristic classification system that categorizes threats into BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS levels, paired with AI-generated explanations for clear risk communication with non-technical staff.

The second objective was about how risk gets decided and then communicated — two distinct steps handled by two distinct components. A binary "safe or unsafe" result throws away information a user actually needs — it treats a file with one minor, easily explained anomaly the same as one with an outright malware hash match. TrustFile instead sorted every scanned file into one of five threat levels, giving users a much more honest picture of where a file actually fell on the risk spectrum.

The heuristic engine sat at the center of this decision-making process, weighing everything the system had found — file hashing, entropy analysis, script detection, size anomalies, file type verification, and the VirusTotal lookup result — through a defined set of rules and thresholds to arrive at a single classification. Considering how these indicators related to each other, rather than evaluating each one in isolation, is what allowed the heuristic engine to produce a more reliable classification than any single check could on its own.

Under this framework: files with no suspicious characteristics at all were marked BENIGN; files with minor anomalies that had a reasonable, legitimate explanation were LOW RISK; files showing unusual behavior that genuinely warranted a closer look were SUSPICIOUS; files where several high-confidence indicators pointed toward malicious activity were HIGH RISK; and files confirmed malicious through strong evidence — a direct hash match against known malware, or overwhelming corroborating indicators — were labeled MALICIOUS, triggering the appropriate warnings.

Once the heuristic engine had reached this classification, the finished result — along with the specific indicators that had contributed to it — was passed to the AI analysis layer (built on the NVIDIA NIM API and the Llama 3.1-70B-Instruct model), whose sole role was to translate that already-decided technical outcome into a clear, plain-language explanation. This division let users, and organizations, respond proportionally to a classification they could trust as consistent and rule-based, while still receiving an explanation written in language a non-technical reader could actually understand.

3. To apply entropy analysis to detect encrypted, compressed, or potentially malicious files.

The third objective integrated entropy analysis as one of TrustFile's core static analysis techniques. Rooted in information theory, entropy measures how random the data inside a file is — and because encryption and compression both sharply increase that randomness, unusually high entropy is often a sign that a file is hiding something: an encrypted payload, a packed executable, or code deliberately obfuscated to dodge conventional signature-based scanning.

This mattered because a large share of modern malware relies on exactly this kind of obfuscation to slip past traditional antivirus tools. But the researchers were also careful not to treat high entropy as automatic proof of malicious intent — plenty of entirely legitimate files (compressed archives, encrypted backups, standard multimedia formats)

are naturally high-entropy simply because of how those formats work. For that reason, entropy analysis was never used as a standalone verdict. It was one input among several — combined with hashing, heuristic analysis, script detection, type verification, size analysis, and the AI engine's reasoning — specifically to avoid false positives that would otherwise undermine trust in the system.

4. To connect the system with external API-based malware detection services for detection.

The fourth objective extended TrustFile's detection beyond what it could determine locally, by tapping into external threat-intelligence services with access to malware databases, behavioral profiles, and reputation data built from global sensor networks and security research — resources far beyond what any single organization could replicate on its own.

Integrating with these services meant balancing several competing demands at once. Query efficiency keeps latency and resource use low while still maximizing coverage. Privacy protection — typically achieved by sending only a file's cryptographic hash rather than its actual contents — ensures sensitive files are never unnecessarily exposed to a third party. Reliability requires fallback behavior for when an external service is temporarily unreachable, so the system degrades gracefully instead of failing outright. Cost management keeps API usage sustainable through caching and smart query batching.

Among the services considered, VirusTotal aggregates detection results from more than 70 different antivirus engines and URL/domain reputation tools into a single consensus report, so that a submitted file's fingerprint can be checked against dozens of independent detection systems at once rather than just one [1], [2]. MetaDefender offers a comparable multi-scanning approach alongside file sanitization capabilities. Each service carries its own tradeoffs in coverage, accuracy, speed, and cost, which is why

the integration was designed to draw on more than one source rather than depending on a single provider.

Beyond direct detection, this external intelligence layer also serves a broader purpose: comparing local results against outside sources highlights where TrustFile's own detection logic could be sharpened, and patterns emerging across a global user base can act as early warning of new attack campaigns well before they show up locally.

5. Display clear scan results and an explanation to help users understand the analysis

The fifth objective is where TrustFile's educational purpose becomes most concrete. A detection result that isn't understood doesn't actually build security resilience — at best it produces uncritical trust in the tool, and at worst it teaches users to ignore warnings altogether once enough of them turn out to be confusing or seemingly irrelevant. Presenting results well means applying real principles of information design: leading with what matters most (the classification and any immediate action needed) before supporting technical detail; using color, icons, and layout to convey severity at a glance; replacing jargon with plain language without sacrificing accuracy; tying findings to real-world consequences the user can actually picture; and always pairing a result with a concrete, feasible next step.

The explanation layer is what answers the "why." A hash match comes with a description of the malware family it belongs to and how it typically behaves. An entropy anomaly comes with a plain explanation of what elevated randomness usually means — and what legitimate reasons might explain it. A detected script explains what kind of script it is and why its presence is suspicious in that specific file's context. A size anomaly explains what a normal range looks like for that file type and what a deviation might suggest. A type mismatch explains exactly how the file's real format differs from what it claimed to be, and the spoofing techniques that produce that kind of mismatch.

Rather than pulling from generic, one-size-fits-all templates, TrustFile's explanation system builds each response from the actual combination of indicators found in that specific file — producing an explanation that's genuinely tailored to what was detected, not a boilerplate warning. The cumulative goal across all five objectives is the same one that runs through the whole study: a user base that, scan by scan, becomes measurably better at recognizing threats independently — reducing blind reliance on automated tools while strengthening both individual and organizational security awareness over time.

Scope and Delimitation

This study focused on the development of TrustFile, a web-based system that allowed users to scan and analyze individual files for possible malware directly within their browser, before those files were opened or executed.

Before detailing what TrustFile did, it was necessary to define clearly what this research did and did not attempt, since that boundary shaped how the system was fairly evaluated. TrustFile was built and deployed as a web-based system, meaning users accessed it through an ordinary browser rather than installing anything locally. The system was hosted on PythonAnywhere, a cloud-based Python hosting platform, so that anyone with an internet connection could reach it without configuring a server or dealing with any technical setup on their end. This was a deliberate design decision. The organizations this study targeted — offices, private companies, government agencies, and similar institutions that handled a high volume of incoming files — did not necessarily have the technical staff or infrastructure needed to maintain a locally installed security tool. A lightweight, browser-accessible platform made far more practical sense for that context.

That choice, however, came with trade-offs the researchers considered worth stating plainly. On the positive side, because the system was centrally hosted, any improvement made to the scanning logic, the AI analysis engine, or the heuristic rules could be rolled out immediately to every user, with no separate update process required on their part. On the other hand, browsers were sandboxed environments by design, meaning TrustFile could not reach into the operating system the way a native application could — it had no ability to hook into the file system at a low level or intercept file operations as they occurred elsewhere on the device. The system's availability was also inherently tied to a stable internet connection and to whatever resource limits and uptime came with the specific PythonAnywhere hosting tier in use. Given that this study set out to assess files before they were opened, rather than provide continuous, system-embedded protection, these trade-offs were judged acceptable for the scope of an academic project.

In terms of how scanning actually worked, the system's scope was strictly upload-based. A user submitted one file at a time through the web interface; that file was transmitted to the PythonAnywhere-hosted server, where the system performed the full range of static scans — hashing, entropy analysis, script detection, size verification, and file type verification — queried external threat-intelligence APIs, and then ran both AI-driven and heuristic analysis on the combined results. What this study deliberately excluded was batch scanning of multiple files at once, scanning of entire folders or directories, and any form of continuous or scheduled monitoring. The intent was to give a user a reliable way to check one specific file they were about to open, not to build a full endpoint-protection suite. Keeping the scope this narrow kept the project realistic within an academic timeframe while still addressing the core problem this study set out to solve — that people lacked an easy, explainable way to check whether a file was safe before opening it.

At the heart of the system was its AI analysis engine, which worked from the combined results of several scanning techniques rather than treating each one as an isolated pass-or-fail check. The engine considered everything together — the hash comparison, the entropy value, whether a script was detected, whether the file size fell outside the norm, and whether the file's actual type matched what it claimed to be — and weighed how these pieces related to one another before producing a classification and an accompanying confidence level. This was, in effect, an attempt to replicate how an experienced human analyst would reason through a file: not fixating on any single red flag, but weighing several pieces of evidence at once. Heuristic analysis operated alongside this, applying more straightforward, rule-based checks for known suspicious patterns and obfuscation techniques. This layer mattered specifically for catching threats new enough that neither the AI engine nor any external database had encountered them before.

The system also connected to external malware-intelligence APIs to check file hashes against databases of known threats, while everything else — script detection, structural pattern analysis, and checking whether a file's actual structure matched its stated extension — was carried out locally on the server. Based on the combined output of all these checks, every scanned file was sorted into one of five threat levels:

- **BENIGN** — the file passed every check cleanly, with no anomalies detected anywhere, and was allowed through without restriction.
- **LOW RISK** — the file showed small deviations that typically had an innocent explanation, such as slightly elevated entropy from ordinary compression; access was still granted, accompanied by a note explaining what was flagged and why it was likely nothing.

- **SUSPICIOUS** — the file showed something genuinely unusual, not enough to call outright malicious, but enough to warrant a closer look or a temporary restriction while that review took place.
- **HIGH RISK** — several indicators lined up together, or a single indicator was strong enough on its own (a hit from an external threat-intelligence API, an entropy value far outside the expected range, a script attempting to contact an external server), meaning the file was likely malicious but not yet confirmed, and was quarantined pending further review.
- **MALICIOUS** — the file was confirmed harmful, most commonly through a direct hash match to a known threat, triggering an immediate block, quarantine, and alert.

The purpose of using five levels instead of a simple safe/unsafe flag was to allow a response proportional to the severity of what was actually found. Every classification was paired with a plain-language explanation of what triggered it, along with relevant tips, so that users walked away understanding a little more about what made their file risky — or not.

Technically, this amounted to a hybrid detection setup that relied on both external intelligence and local analysis, since each covered gaps the other left open. The external side worked through file hashing — generating a unique fingerprint of a file's contents and checking it against databases of known malicious hashes maintained by established threat-intelligence services. This method was fast and effective against anything already documented, but it could not, by definition, catch something entirely new, or even a previously known file with a single byte altered to produce a different hash [1]. This is where local, server-side scanning came in. Script detection searched for executable code hidden inside files that should not normally contain any — JavaScript embedded in a PDF, a macro buried inside a Word document, PowerShell

commands hidden in an archive — any of which could indicate trouble even when the file appeared entirely ordinary on the surface. Heuristic rules looked for suspicious structural patterns and known obfuscation tricks. File type verification checked whether a file's actual internal structure matched what its extension claimed, catching cases where, for instance, an executable had been dressed up to resemble a document. Entropy analysis flagged files that were unusually random-looking, which could point to encryption or packing used to conceal malicious code — though this had to be interpreted carefully, since a number of entirely harmless files, such as compressed archives and certain media formats, were naturally high-entropy as well, and the system needed to reliably distinguish between the two cases. All of these local results fed into the AI analysis engine, which tied the entire detection process together, picking up on combinations of small anomalies that might not mean much individually but added up to something worth flagging when considered together. Heuristic analysis ran in parallel, catching suspicious characteristics that matched no known signature or pattern the AI engine had learned from — which mattered specifically for genuinely novel or deliberately obfuscated threats.

Beyond detection and classification, this study also placed real weight on helping users understand what was happening, which is why explanations were built into every scan result rather than added as an afterthought. Whenever the system flagged something, it explained precisely what triggered the flag — a hash match to a known malware family, an unusually high entropy value and what that typically indicated, a detected script and why its presence was suspicious in that particular type of file, a size deviation from what was expected, or a file type that did not match its stated extension — all explained in plain terms that assumed no prior cybersecurity background. Alongside these explanations, the system offered general guidance on safe file handling, recognizing common attack tactics, understanding different malware categories, and maintaining day-to-day system security. Folding this education directly into the scanning process, rather than treating it as a separate lesson, was intended to help users build genuine security awareness over time — with the underlying goal being that users came away understanding why a file was flagged, not simply that it was.

That said, the system operated within clear limits. It was built around static analysis only — meaning it examined a file's contents and structure without ever executing it [2]. It offered no real-time protection and did not attempt to replicate the full feature set of commercial antivirus software. It could only scan files a user actively chose to upload, and, like any security tool, it could not guarantee it would catch every possible threat. It was also, first and foremost, an academic project — tested against a curated set of sample files rather than deployed as a finished commercial product.

A few specific boundaries were worth spelling out directly. Because the system only ever examined a file's contents and structure without running it, it had no way of catching malware that revealed itself solely through execution — logic bombs that remained dormant until a specific condition was met, fileless malware that existed entirely in memory without ever writing a file to disk, or malware that only downloaded its actual payload after an initial, seemingly harmless file had already been run [2], [3]. Dynamic analysis — which involves actually executing a file in a sandboxed environment to observe its behavior directly — would have been capable of catching these cases, but building that capability was outside what this study set out to do [2]. Static analysis was safer and faster to implement, but it traded away that behavioral insight, and that trade-off was made deliberately within this study's scope.

Real-time protection was excluded for a related reason. Traditional antivirus software runs continuously in the background, scanning files the instant they are touched anywhere on a device — but doing so properly requires deep integration with the operating system, kernel-level access, and constant background resource consumption, none of which was realistic for a browser-based tool running on shared cloud hosting such as PythonAnywhere. Within this study, scanning therefore only occurred when a user actively chose to upload a file; there was no passive, always-on protection layered across everything a user did on their device.

The system was also limited to files a user could legitimately access on their own. It could not reach files locked behind operating system permissions, files inside encrypted volumes without the corresponding key, or anything on a system it had no access to in the first place — and this limitation was not purely technical. Scanning files a user did not have legitimate access to would have required elevated privileges that raised genuine privacy and ethical concerns well beyond the intent of this study.

None of this was meant to overstate what TrustFile could do. No security tool, however well designed, catches every threat, and the specific combination of techniques used here — static analysis, reliance on files the user actively chose to upload, and a defined set of detection methods — inevitably left some gaps. That limitation is shared by security research generally, not unique to this project, and it does not diminish what TrustFile was able to demonstrate within the scope it set for itself. Finally, it bears repeating that this was an academic research project, not a finished commercial security product ready for public release — a distinction that mattered for how the system was evaluated and what could reasonably be expected of it. In keeping with that, testing relied on a deliberately curated set of sample files representing both benign and malicious characteristics, rather than exposing the system to live malware in an uncontrolled, real-world setting — a choice made to validate the system's capabilities responsibly, without risking actual harm in the process.

Significance of the Study

The proposed system will deliver significant benefits to the following stakeholders:

- **End Users** - End users experienced safer file interactions through automatic pre-access scanning, which ensured that only verified, threat-free files reached their workstations without requiring technical expertise or manual intervention on their part.

This stakeholder group encompassed everyone who interacted with digital files in a personal or professional capacity — students exchanging assignments and resources, employees processing work documents, consumers downloading software and media, and general computer users engaging with digital content as part of daily life. The significance for this group centered on a fundamental shift: file security moved from a specialized technical concern that demanded real expertise into an accessible, largely automated service available to anyone, regardless of their technical background. This safety came from the systematic layering of multiple detection techniques, AI-driven analysis, and heuristic analysis before a file ever reached a user's workstation. Rather than depending on users to personally recognize threats, or relying on remediation after an infection had already occurred, TrustFile intercepted potentially dangerous files at the point of access, preventing exposure before it could happen at all. This shifted the underlying security model from reactive to proactive, which in practical terms translated into fewer successful infections and fewer of the downstream consequences — lost data, compromised credentials, disrupted work — that came with them.

The phrase "without requiring technical expertise" addressed a real accessibility gap in how file security was typically practiced. Existing tools often quietly assumed a level of user knowledge that most people simply did not have — understanding what a file extension actually signified, recognizing the subtler indicators of a phishing attempt, correctly interpreting a security warning, or even knowing how to run a manual scan in the first place. By automating these functions and presenting results in accessible language, complete with clear, AI-generated explanations, TrustFile extended meaningful protection to users who would otherwise have remained exposed simply because they had never had the technical background needed to protect themselves.

Removing the need for manual intervention also reduced both the mental burden placed on users and the room for human error. Users did not have to remember to run a scan, did not have to interpret ambiguous technical indicators on their own, and were not left making risk judgments under uncertainty with incomplete information. The system took on these responsibilities consistently and thoroughly, removing the user as a potential point of failure in the security chain, while still keeping them informed — and gradually more knowledgeable — through the explanations it provided with every scan.

- **The Organization** - The organization demonstrated a responsible and intelligent approach to file security, establishing TrustFile as a practical model for protecting digital assets in government offices, private sector organizations, and corporate environments that regularly received and downloaded files.

This stakeholder group included businesses, government agencies, educational institutions, non-profit organizations, private sector companies, and any entity that managed digital assets and faced file-based security risks as a routine part of its operations. The significance for organizations extended beyond a direct improvement in security posture, reaching into reputational, operational, and strategic benefits as well. Demonstrating a responsible approach to file security meant putting in place visible, effective controls that protected organizational data, kept operations running smoothly, and fulfilled an organization's duty of care to the people it served. In a climate of increasing regulatory scrutiny around data protection — under frameworks such as GDPR, HIPAA, and PCI-DSS, along with various sector-specific regulations — having demonstrable security measures in place offered tangible evidence of compliance effort and due diligence. TrustFile supported this by producing auditable scan records, applying its checks consistently across every file, and documenting exactly what was detected and why. Describing the system's approach as "intelligent" reflected its integration of multiple detection techniques — AI-driven analysis, heuristic analysis, and external threat intelligence — working together, rather than any

single detection method carrying the full weight on its own. This layered strategy reflected established security best practices and positioned any adopting organization as embracing a more sophisticated, contemporary approach to protection, rather than depending on a single point of failure the way older signature-only tools often did.

The intelligence built into the system also extended to how the AI analysis engine reasoned across combinations of indicators rather than evaluating them individually, allowing its assessments to remain relevant even as new patterns of malicious file characteristics continued to emerge over time. The specific focus on government offices, private sector organizations, and corporate environments reflected how relevant file security was in exactly the professional contexts where files constantly arrived from outside the organization. Government agencies handled sensitive citizen data, classified information, and critical infrastructure controls that made them attractive, high-value targets for attackers. Private sector companies managed proprietary business information, financial records, and client databases, all of which carried real consequences if compromised. Corporate environments, more broadly, dealt with a constant stream of incoming documents from partners, clients, and vendors, any one of which could carry a hidden threat. Budget limitations, aging legacy systems, and uneven technical literacy among staff often made proper security implementation harder in these settings than it appeared on paper. TrustFile's web-based, straightforward, and low-cost approach was designed with exactly these constraints in mind, offering a practical model that could reasonably be adopted and adapted across many different kinds of organizations, regardless of their existing technical infrastructure.

Positioning TrustFile as a practical model meant more than simply proving that the underlying technology worked — it meant documenting how it was implemented, honestly evaluating how effective it actually was, and sharing those findings in a way that allowed other organizations to learn from and build on the

work. In that sense, this research contributed to the broader body of knowledge around accessible file security solutions, offering something other organizations could reasonably adapt to their own particular needs and constraints.

- **Future Researchers** - Future researchers were able to use this study as a reference, build on it to advance learning-based detection systems, and draw on it to help improve security applications in research and development more broadly.

This stakeholder group covered academic researchers, graduate students, industry researchers, and developers who went on to investigate file security, malware detection, web-based security tools, AI-driven applications in cybersecurity, heuristic analysis techniques, and related areas. The significance for this group lay in the study's contribution to a growing, cumulative body of knowledge, and in its role as a starting point for further work rather than an endpoint in itself. As a reference, this study offered a documented methodology, implementation details, evaluation results, and honestly reported lessons learned — all of which future researchers could consult, critique, and build upon as they saw fit. The documented approach to the hybrid detection architecture, the AI analysis engine's design, the integration of heuristic analysis, the API integration strategy, the interface design choices made for a security tool aimed at non-technical users, and the integration of educational content into the scanning workflow all offered concrete technical guidance that went beyond abstract discussion. The evaluation framework used in this study also gave future researchers a benchmark against which their own systems could reasonably be measured.

Building on this study to advance learning systems pointed specifically to the potential for extending TrustFile's capabilities using more advanced AI techniques in future work. The AI-driven, five-tier classification approach described in the Objectives provided a foundation that later work could refine

further — through more sophisticated reasoning approaches, a broader and more diverse set of indicators, or more precise weighting of how different anomalies related to one another. The heuristic analysis component, similarly, could be expanded with a more comprehensive set of rules and behavioral indicators over time. The static analysis techniques used in this study could eventually be supplemented with dynamic analysis, behavioral modeling, or even automated reverse engineering, for a fuller picture of a file's actual risk. And the web-based architecture itself left room for future extension into mobile applications, browser extensions, or deeper enterprise integration.

Improving security applications in research and development, more broadly, reflected the real influence that academic work of this kind could have on practical security tools used in the wider world. Findings about which detection techniques actually worked, how effective AI-driven and heuristic analysis proved to be in practice, what interface choices genuinely helped non-technical users, which educational approaches actually built understanding rather than just producing another warning users learned to ignore, and how all of these pieces fit together architecturally — all of this fed into commercial product development, open-source security projects, and the security implementations organizations eventually adopted. In that sense, this research formed part of the pipeline that turned academic insight into protections people actually used.

Definition of Terms

- **Abnormal File Size**

Refers to a file whose size falls outside the range normally expected for its declared type or content, a deviation that can indicate an embedded malicious payload, appended code, or a stripped-down stub file. Every file format carries its own rough expectations for size, shaped by what the file contains, how complex that content is, and how much compression has

been applied — a plain text document typically runs from a few kilobytes to a few megabytes, while high-resolution images or video files can legitimately run much larger simply because of their resolution and length. When a file is significantly larger than expected for its type, this can indicate that additional code — potentially malicious — has been appended to an otherwise ordinary file. Conversely, an unusually small file can indicate a stub loader: a minimal file whose only function is to quietly download the real malicious payload once it is executed. Neither pattern is something a user could reasonably notice simply by glancing at file size, which is why abnormal file size verification was treated in TrustFile as one useful preliminary filter among several, rather than a standalone verdict.

- **API (Application Programming Interface)**

A defined set of rules and protocols that allows separate software applications to communicate, exchanging data and functionality without either system needing to know the internal workings of the other. An API functions essentially as a contract: it specifies what operations are available, what input each operation requires, what response to expect, and how errors should be handled. In cybersecurity specifically, APIs served as the connective layer that let TrustFile query external threat-intelligence services — submitting a file's hash and receiving back reputation data, detection results from multiple antivirus engines, or known-malware matches — without needing to host or maintain those enormous databases locally.

- **Artificial Intelligence (AI) Analysis**

Within TrustFile, AI analysis referred specifically to the decision-making layer that combined the results of every other detection technique — file hashing, entropy analysis, script detection, size verification, file type verification, heuristic analysis, and external threat-intelligence lookups — into a single, coherent threat classification. Rather than treating each of

these checks as an isolated pass/fail test, the AI engine reasoned across how they related to one another, recognizing that several individually minor anomalies could add up to a meaningfully higher risk when considered together, in much the same way an experienced human analyst would weigh several pieces of circumstantial evidence rather than fixate on any single one. This approach allowed the system to produce classifications, and confidence levels, that were more reliable than any single detection method could produce on its own.

- **Benign Classification**

The cleanest possible outcome TrustFile could assign to a scanned file, given when a file passed every check performed — hashing, entropy analysis, script detection, size verification, and file type verification — with nothing flagged as unusual, and when the file's actual structure and behavior matched exactly what would be expected for its stated type. A benign classification meant the file was treated as safe, and access proceeded without restriction. As with the low-risk classification, this label reflected a high degree of confidence rather than an absolute guarantee, since no static-analysis system — including TrustFile — could claim to rule out every conceivable threat with total certainty.

- **Black Box**

A term describing any detection system whose internal reasoning is hidden from the user, such that the system accepts an input (a file) and produces an output (a verdict) without ever revealing what specific indicators were examined, what they meant technically, or how confident the system actually was in its own conclusion. Traditional antivirus software commonly operated this way: a threat was quarantined or deleted, and the user was left with little more than a generic threat name. This opacity carried real costs — it prevented users from verifying whether a detection was accurate, gave them no way to build their own recognition

skills, and occasionally bred outright distrust of the software when a legitimate file was mistakenly flagged. TrustFile was deliberately designed to work against this pattern, generating a plain-language explanation for every classification it produced rather than leaving the reasoning hidden.

- **Data Breach**

An incident in which sensitive, protected, or confidential information is accessed, exposed, or extracted by an unauthorized party — whether through a successful malware infection, a compromised credential, or a direct network intrusion. Data breaches carry consequences well beyond the immediate loss of data: they generate direct financial costs from investigation and remediation, interrupt normal business operations, expose organizations to regulatory penalties, and cause lasting reputational damage. As referenced earlier in this chapter, IBM's 2024 Cost of a Data Breach Report placed the global average cost of a single breach at roughly US\$4.88 million, underscoring why preventing a malicious file from executing in the first place — TrustFile's core purpose — carries financial significance well beyond the individual file being scanned.

- **Defense-in-Depth**

A cybersecurity strategy built on layering multiple, overlapping security controls throughout a system, so that no single point of failure can compromise the whole — if one layer is bypassed, another remains in place to catch the threat. The concept borrows from military strategy, where defensive lines are staggered to slow and weaken an advancing threat rather than relying on one line to hold entirely. TrustFile applied this principle directly by combining several complementary static-analysis techniques with AI-driven analysis, heuristic analysis, and external threat intelligence, so that each technique compensated for another's blind spot: hashing caught known threats, entropy analysis caught packed or

encrypted payloads, script detection caught embedded malicious code, size analysis caught structurally abnormal files, and type verification caught disguised executables. This layered approach meaningfully increased the likelihood that at least one detection technique would catch a threat, even where another had been successfully evaded.

- **Entropy**

A concept borrowed from information theory that measures how random or unpredictable the data within a file is. Encryption and compression both sharply increase a file's entropy, which is why entropy analysis became a useful indicator for identifying files that might be hiding malicious content — an unusually high entropy value can point to an encrypted payload, a packed executable, or otherwise obfuscated code, while legitimate files such as plain text documents or standard executables typically show much lower entropy because their internal data follows structured, predictable patterns. Malware developers deliberately exploit this by using tools called packers and crypters to compress or encrypt their payloads, which sharply raises the resulting file's entropy. By calculating a file's Shannon entropy, TrustFile could flag files whose entropy fell outside the normal range expected for their stated type, providing an early signal that something might be hidden inside. Because some entirely legitimate files — compressed archives, certain multimedia formats, or software protection mechanisms — are also naturally high-entropy, entropy analysis was never treated as a standalone verdict; it was always combined with the system's other detection techniques to avoid unnecessary false positives.

- **False Positives and False Negatives**

Two categories of error inherent to any detection or classification system, representing an unavoidable trade-off in security accuracy. A false positive occurs when the system incorrectly flags a legitimate, harmless file as

malicious — a false alarm that, if it happens too often, wastes administrative time, erodes user trust in the system, and can push frustrated users toward disabling security features altogether. A false negative occurs when the system fails to catch an actual threat, letting a malicious file pass through undetected — a more dangerous failure mode, since it results directly in a successful infection rather than merely an inconvenience. Minimizing false negatives while keeping false positives at an acceptable level was a central design consideration throughout TrustFile's development, addressed by combining multiple complementary detection techniques rather than depending on any single check that might individually be prone to one error type or the other.

- **File Extension Spoofing and Double Extensions**

A deceptive technique in which attackers manipulate a file's visible filename and extension to disguise its true, and dangerous, underlying type. Because most operating systems determine how a file is handled based on its extension, an attacker can make an executable appear to be a harmless document simply by naming it accordingly. Double extension techniques take this further by constructing filenames with two extensions — such as "document.pdf.exe" or "invoice.docx.scr" — relying on the fact that many operating systems hide known file extensions by default, so that a user sees only "document.pdf" and has no visual indication that the file is, in reality, an executable. TrustFile countered this specific deception through file type verification, which examined a file's actual internal structure rather than trusting its visible name.

- **File Type Verification**

A static analysis technique that confirms whether a file's actual internal structure matches what its extension and metadata claim it to be, rather than relying on visible cues that an attacker could easily fake. This check worked by examining a file's magic numbers — the fixed byte sequences

that appear at the start of a file regardless of its name — and comparing them against what would be expected for the file's stated type. A mismatch between a file's claimed type and its actual internal structure was treated as a strong warning sign, since this is precisely the technique attackers rely on when disguising an executable as an ordinary document. File type verification worked as a direct countermeasure to file extension spoofing and double-extension tricks, catching exactly the kind of deception that a purely visual inspection would miss.

- **Fileless Malware**

A category of malicious software that operates entirely within a computer's memory (RAM) rather than writing any file to disk, which allows it to evade traditional, file-based security tools almost entirely. Instead of installing an executable, fileless malware typically abuses legitimate, pre-installed system tools — such as PowerShell or macros embedded in Office documents — to carry out malicious commands directly in memory. Because it leaves no persistent file behind, fileless malware bypasses signature-based detection and standard antivirus scans that depend on examining stored files. This represented one of the clearest limitations acknowledged in this study's Scope and Delimitation: because TrustFile analyzed a file's contents and structure without ever executing it, it had no way of catching a threat that revealed itself only once running in memory — a gap that would require dynamic, behavior-based analysis to close.

- **Hashing**

A cryptographic process that generates a unique, fixed-length string of characters — known as a hash value — from a file's contents, using algorithms such as MD5, SHA-1, or SHA-256. The defining property of a hash function is that even the smallest change to a file, down to a single altered bit, produces a completely different hash value, making hashes extremely reliable for identifying whether two files are truly identical. Within

TrustFile, file hashing served as the fastest of the system's detection techniques: a scanned file's hash could be checked directly against external databases of known malicious hashes, allowing instant identification of previously encountered threats without needing to examine the file's contents in any deeper way. Its clear limitation, addressed elsewhere in this chapter, is that it can only recognize threats that have already been seen and catalogued before — a single altered byte is enough to produce an entirely different hash and slip past this check undetected.

- **Heuristic Analysis**

A detection technique that identifies potentially new or previously unknown malware by examining a file's structural characteristics and behavioral indicators, rather than requiring an exact match against a known malware signature. Where signature-based detection can only recognize threats it has already catalogued, heuristic analysis applies rules designed to catch suspicious patterns commonly associated with malicious files — unusual code structures, signs of obfuscation, or characteristics resembling known attack techniques — even when no exact match exists. This gives heuristic analysis a genuine advantage in catching zero-day threats and polymorphic malware. Its corresponding weakness is a tendency toward false positives, since legitimate software occasionally uses unusual coding techniques that can resemble the patterns heuristic rules are built to catch. Within TrustFile, heuristic analysis ran alongside the AI analysis engine, catching suspicious characteristics that matched no known signature or pattern the AI engine had previously learned from.

- **High Risk Level**

The classification assigned to a file that showed strong suspicious characteristics without having been definitively confirmed as malicious — for example, an entropy value far outside the expected range, a script

attempting to establish contact with an external server, or a file type that failed to match its claimed extension. A HIGH RISK classification meant the file was likely malicious, but not yet confirmed with the certainty required for the MALICIOUS label, and was accordingly quarantined pending closer administrative review rather than blocked outright or allowed through.

- **Keyloggers**

A category of spyware specifically designed to covertly record every keystroke a user types, capturing passwords, financial details, and other sensitive information without the user's knowledge. Software keyloggers can run as kernel-level drivers that intercept keystrokes directly at the operating system level, or as ordinary applications quietly monitoring window activity in the background. Because keyloggers operate independently of whatever other precautions a user takes — a strong password offers no protection if every keystroke used to type it is already being recorded — they represent a particularly dangerous category of malware, and one of the reasons malware in general, as discussed in the Background of the Study, remains such a persistent concern for organizations handling sensitive data.

- **Logic Bombs**

Malicious code deliberately inserted into an otherwise normal file or system, engineered to remain completely dormant and undetectable until a specific triggering condition occurs — a particular date, the absence of a specific user account, or some other predefined event — at which point it executes its destructive payload. Because a logic bomb exhibits no malicious behavior whatsoever until triggered, it can pass through security scans and code review entirely unnoticed. This represents one of the clearest limits of static analysis acknowledged in this study's Scope and Delimitation: a system that examines a file without executing it has no way

of detecting a threat that only reveals its true nature once a specific, often unpredictable, condition is eventually met.

- **Low Risk Level**

The classification assigned to a file that was largely clean but showed a small, usually explainable deviation — commonly, mildly elevated entropy resulting from ordinary compression rather than anything malicious. Files receiving this classification were still granted normal access, accompanied by a brief note explaining what had been flagged and why it was most likely nothing to be concerned about. As with the benign classification, a low-risk label reflected a high degree of confidence rather than an absolute guarantee, since detection systems operate in terms of probability rather than certainty.

- **Magic Numbers**

Also known as file signatures, magic numbers are fixed byte sequences located at the very start of a file that reliably identify its true format, independent of whatever filename or extension the file has been given. A PDF file, for instance, begins with the bytes corresponding to "%PDF," while a standard Windows executable begins with the characters "MZ." Because these byte sequences are embedded as part of a file's actual content rather than its filename, they cannot be altered simply by renaming a file, which made them the technical foundation underlying TrustFile's file type verification — allowing the system to confirm a file's real identity even when its visible name had been deliberately falsified.

- **Malicious Threat Level**

The highest classification within TrustFile's five-tier system, reserved for files confirmed to be genuinely harmful and requiring immediate action. Reaching this classification required a high degree of confidence — typically either a direct hash match against a known threat in an external

intelligence database, or several converging indicators of malicious behavior strong enough, together, to leave little doubt. Files given this classification triggered immediate blocking, quarantine, and an administrative alert, reflecting the severity of the confirmed risk.

- **Malware**

A general term for any software deliberately developed to damage, disrupt, or gain unauthorized access to a computer system, typically for the purpose of stealing data, extorting a victim, or disrupting operations. Malware can cause damage in numerous ways — deleting or corrupting files, stealing credentials and financial information, disabling security software, or, in more severe cases, damaging firmware or hardware directly. Malware is typically categorized according to how it spreads (attaching to files, propagating independently across a network, or disguising itself as legitimate software), what it ultimately does once active, and how effectively it conceals its presence. Viruses, worms, Trojan horses, ransomware, spyware, rootkits, and keyloggers — all discussed elsewhere in this chapter — represent different categories within this broader term.

- **Packers and Crypters**

Tools used by malware developers to compress or encrypt malicious code specifically to make it harder for antivirus software to detect and analyze. A packer compresses an executable, obscuring its internal code structure until a small decompression routine unpacks it back into memory at runtime; common examples include UPX and ASPack, tools that are also occasionally used for entirely legitimate purposes, such as reducing file size or protecting software from reverse engineering. A crypter, by contrast, encrypts the malicious payload outright, hiding the harmful code completely until a decryption routine embedded in the file reveals it at execution — leaving no trace of the original malicious code visible on disk

beforehand. Both techniques significantly alter a file's outward appearance, rendering signature-based detection ineffective since the packed or encrypted file no longer matches any known malware signature. Both also sharply raise a file's entropy, which is precisely why entropy analysis served as such a valuable complementary technique within TrustFile's detection process.

- **Phishing**

A social engineering technique in which attackers send fraudulent communications — most commonly emails — disguised as legitimate correspondence from a trusted source, such as a bank, employer, shipping company, or government agency, in order to trick a recipient into opening a malicious attachment or divulging sensitive information. Phishing exploits psychological pressure points such as urgency, curiosity, or authority rather than any technical vulnerability, which is precisely why it remains, as discussed in the Background of the Study, one of the most common and effective ways malicious files are distributed in the first place. Because phishing succeeds by manipulating human judgment rather than bypassing a technical defense, no purely automated detection system can fully eliminate the risk it poses — which is part of why TrustFile placed such emphasis on user education alongside detection.

- **Polymorphic and Metamorphic Malware**

Two advanced techniques malware authors use to evade signature-based detection by continually altering a file's appearance. Polymorphic malware encrypts its malicious payload using a variable key and pairs it with a decryption routine that changes with each new infection, so that while the underlying malicious function stays identical, the encrypted body and accompanying routine look different every time — producing an entirely new file hash with each generation. Metamorphic malware goes further still, completely rewriting its own code with every generation, altering not

just its outward appearance but its internal structure and instruction sequence, while preserving the same underlying malicious function. Both techniques illustrate the ongoing arms race between malware developers and security researchers referenced throughout this chapter, and are part of the reason detection approaches that go beyond simple signature matching — heuristic and AI-driven analysis among them — remain necessary.

- **Pre-Access Scanning**

The process of examining a file for potential threats before a user is permitted to open or execute it, intercepting the file at the exact point where damage would otherwise occur. This represented the core preventive philosophy behind TrustFile's design: rather than depending on a user's own judgment, or on cleaning up an infection after it had already happened, pre-access scanning applied the system's full combination of static analysis techniques, AI-driven analysis, and heuristic analysis to a file the moment it was uploaded, before the user ever gained access to its actual contents.

- **Ransomware**

A category of malware that encrypts a victim's files and demands payment, typically in cryptocurrency, before restoring access to them. Modern ransomware campaigns frequently pair this encryption with a "double extortion" tactic, in which attackers also threaten to publicly release the stolen data if the ransom goes unpaid, adding a second layer of pressure beyond the simple loss of access. As referenced in the Background of the Study, ransomware represents one of the most financially damaging categories of malware currently in circulation, and its typical delivery method — a malicious file disguised as an ordinary attachment — is precisely the kind of threat TrustFile's pre-access

scanning approach was designed to intercept before execution ever occurred.

- **Regulatory Compliance (GDPR, HIPAA, PCI-DSS)**

The set of legal and industry frameworks governing how organizations must handle sensitive data, with penalties attached for failing to demonstrate adequate protective measures. The General Data Protection Regulation (GDPR) governs personal data protection primarily within the European Union; the Health Insurance Portability and Accountability Act (HIPAA) governs the protection of medical and health-related information within the United States; and the Payment Card Industry Data Security Standard (PCI-DSS) governs the handling of payment card information across any organization that processes it. As discussed in the Significance of the Study, demonstrating consistent, auditable file security practices — of the kind TrustFile was designed to produce through its scan logs and documented classifications — offers organizations tangible evidence of compliance effort under any of these frameworks.

- **Rootkits**

A category of malware designed to gain deep, persistent access to a system while actively concealing its own presence from both the user and security software. The term combines "root," referring to the highest level of administrative access on Unix-based systems, with "kit," referring to the collection of tools used to establish and maintain that access. Rootkits operating at the kernel level are especially dangerous, since they run with the highest possible system privileges, allowing them to intercept system calls, hide files and processes, and disable security software outright. Because rootkits actively subvert the very mechanisms typically used to detect them, they are notoriously difficult to identify through conventional means.

- **Script Detection**

A static analysis technique that identifies executable code embedded within files that would not normally be expected to contain any — JavaScript hidden inside a PDF, a macro embedded in a Word document, or PowerShell commands buried inside a compressed archive. The presence of such a script is not automatically proof of malicious intent, but it is a meaningful warning sign, since this is precisely the mechanism by which many disguised malicious files operate: the file appears entirely ordinary on the surface while quietly carrying executable code capable of running once opened. Within TrustFile, script detection results were never evaluated alone; they were combined with the system's other static analysis techniques and fed into the AI analysis engine to help determine an overall classification.

- **Signature-Based Detection**

The foundational method used by traditional antivirus software, in which a scanned file is compared against a database of previously identified malware fingerprints, or signatures, and flagged if a match is found. This approach is fast and highly reliable against threats that have already been discovered and catalogued, but it carries an unavoidable structural limitation: it cannot recognize malware that is new, modified, or deliberately altered to avoid producing a matching signature, leaving a window of vulnerability between a threat's emergence and the moment a corresponding signature is finally issued. As discussed throughout this chapter, this limitation is precisely why TrustFile combined signature-based hash comparison with heuristic analysis, entropy analysis, and AI-driven reasoning, rather than depending on signature matching as its sole line of defense.

- **Social Engineering**

The psychological manipulation of a person into taking an action, or disclosing information, that compromises security — targeting human judgment rather than any technical vulnerability. Common techniques include phishing, pretexting (fabricating a false scenario to extract information), and baiting (offering something enticing to lure a victim into compromising their own security). In the specific context of file-based malware, social engineering is what makes a malicious attachment convincing in the first place — dressing it up as an invoice, a job offer, or an urgent notice from a trusted contact. Because this method exploits psychology rather than software, no purely technical detection system can eliminate the risk entirely, which is precisely why TrustFile's educational component — explaining to users why a file was flagged, rather than simply blocking it — was treated as just as important as detection itself.

- **Spyware**

A category of malware designed to covertly monitor a user's activity and collect information — browsing habits, login credentials, financial details — typically transmitting what it gathers back to an attacker without the user's knowledge or consent. Keyloggers, discussed elsewhere in this chapter, represent one specific and particularly common form of spyware. Because spyware is built specifically to remain unnoticed for as long as possible, often mimicking legitimate background processes, it can operate undetected for extended periods, quietly undermining every other security measure a user has in place.

- **Static Analysis**

The examination of a file's contents and structure without ever executing it — the category of technique underlying every core detection method TrustFile employed, including file hashing, entropy analysis, script

detection, size verification, and file type verification. Static analysis has the advantage of being fast, safe, and free of the risk that comes with actually running a potentially malicious file, but it carries the corresponding limitation, acknowledged directly in this study's Scope and Delimitation, of being unable to detect threats — such as logic bombs or fileless malware — that reveal their malicious nature only once actually executed. This trade-off, between safety and behavioral insight, was made deliberately within the scope this study set for itself.

- **Suspicious Level**

The classification assigned to a file that displayed something genuinely unusual — an entropy value somewhat higher than typical for its type, metadata inconsistencies, or origin from an unfamiliar source — without clearly qualifying as either safe or outright dangerous. A SUSPICIOUS classification prompted additional verification steps before the file was made accessible, reflecting the reality that assessing file safety is not always a clear-cut determination, and that the system sometimes had to weigh genuinely ambiguous evidence rather than reach an immediately confident conclusion.

- **Threat Classification**

TrustFile's overall system for automatically sorting every scanned file into one of five risk categories — BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS — based on the combined results of file hashing, entropy analysis, script detection, size verification, file type verification, and external threat-intelligence lookups. Rather than relying on any single check, the AI analysis engine weighed all of these indicators together, alongside the heuristic analysis component, to arrive at a classification and an accompanying confidence level. This five-tier structure allowed both individual users and organizations to respond proportionally to the

actual severity of what was detected, rather than treating every flagged file identically.

- **Threat Intelligence**

Evidence-based knowledge about known and emerging cyber threats — including malware families, attacker techniques, and known-malicious file hashes — collected, maintained, and continuously updated by security researchers and global threat-monitoring networks. TrustFile integrated with external, API-based threat-intelligence services that maintained these databases at a scale no individual organization could realistically replicate on its own, allowing the system to identify emerging threats that might not yet be reflected in its local analysis alone. This combination of local analysis and external intelligence formed the hybrid detection model described throughout this chapter, balancing the speed and privacy of local processing with the broader coverage that only large-scale, continuously updated threat intelligence can provide.

- **Trojan Horse**

A category of malware that disguises itself as legitimate, desirable software while secretly carrying out malicious activity once installed or executed. Unlike a virus, a Trojan horse does not self-replicate by attaching to other files; instead, it relies entirely on tricking a user into installing it directly, typically by presenting itself as a useful program, a cracked piece of paid software, or an urgent system update. Once active, a Trojan can perform virtually any malicious function — installing additional malware, creating backdoor access for an attacker, or stealing data outright — making it, as introduced in the Background of the Study, one of the most versatile and commonly encountered categories of malware in circulation.

- **Virus**

Malicious software that attaches itself to legitimate files or programs in order to spread from one system to another, in a manner conceptually similar to how a biological virus requires a host to replicate. A virus works by inserting its own code into a legitimate file, carefully avoiding anything that would make the file obviously corrupted or immediately trigger detection, and ensuring its malicious code runs once the host file is opened. Viruses commonly spread through shared files — email attachments, removable media, or downloads from compromised websites — and because they typically require some user action, such as opening an infected file, to begin spreading, they can, in some respects, be easier to trace back to their origin than malware capable of propagating entirely on its own.

- **Worm**

A category of malware capable of copying itself and spreading across computers and networks entirely on its own, without needing to attach to a host file or rely on any action from a user — a key distinction from viruses, discussed elsewhere in this chapter. Worms exploit weaknesses in network configurations, outdated software, and weak security settings to move from one system to the next, replicating and continuing to spread automatically once inside a new environment. This capacity for fully autonomous spread has driven some of history's most disruptive cyberattacks, including the Morris Worm of 1988, which infected roughly 6,000 computers within a single day.

- **Zero-Day Exploits**

Attacks that target a software vulnerability unknown to the vendor or security researchers at the time of the attack, meaning no patch, signature, or specific defense yet exists to stop it. The term reflects the

fact that developers have had zero days to address the vulnerability before it was already being exploited. Because signature-based detection depends entirely on a threat having already been identified and catalogued, it is structurally incapable of recognizing zero-day threats, leaving a genuine window of maximum vulnerability until a defense is developed. This limitation, discussed at several points throughout this chapter, is a central reason TrustFile combined signature-based hash comparison with heuristic analysis, entropy analysis, and AI-driven reasoning, rather than depending on any single detection method to catch threats of this kind.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

Foreign Studies

Aslan and Samet (2020) — A Comprehensive Review on Malware Detection Approaches

Aslan and Samet (2020) conducted a comprehensive review of malware detection approaches published in IEEE Access, examining how malicious software continues to grow at an alarming rate and frequently conceals itself within systems through various obfuscation techniques. Their study found that while signature-based and heuristic-based detection approaches are fast and efficient for identifying known malware, signature-based methods in particular have failed to detect unknown threats. In contrast, behavior-based, model checking-based, and cloud-based approaches perform well against unknown and more complex malware, and emerging methods such as deep learning-based, mobile device-based, and IoT-based approaches also show promise in detecting both known and unknown variants.

The study's examination of malware growth rates quantifies the expanding threat environment that motivates Trustfile's development. Industry reports consistently document exponential increases in new malware samples, with millions of new variants detected annually. This growth results from multiple factors: the low cost and high profitability of malware development, the availability of malware-as-a-service platforms that democratize attack capabilities, the automation of malware generation through polymorphic and metamorphic engines, and the expanding attack surface created by IoT devices, cloud services, and remote work arrangements.

The focus on obfuscation techniques addresses a critical challenge for detection systems. Malware developers employ diverse obfuscation strategies to evade signature-based detection, including code encryption, packing, polymorphism, metamorphism, anti-debugging, anti-virtualization, and steganography. These techniques transform malware appearance between generations or during execution while preserving malicious functionality. Aslan and Samet's examination of these techniques provides essential context for understanding why signature-based methods struggle and why alternative approaches such as machine learning classification and heuristic analysis are necessary. The comparative findings regarding detection methodologies inform Trustfile's hybrid architecture design. Signature-based detection, while fast and efficient for known threats, exhibits fundamental limitations against novel malware that lacks matching signatures in detection databases. The temporal gap between malware emergence and signature creation—sometimes hours or days—creates windows of vulnerability. Heuristic-based detection improves upon signatures by identifying suspicious patterns and behaviors, but basic heuristics can be circumvented by sophisticated obfuscation and may generate false positives that erode user trust. The superior performance of behavior-based approaches reflects their focus on what malware does rather than what it looks like. By monitoring runtime behaviors—file system modifications, registry changes, network communications, process injections—behavior-based systems can identify malicious activity even from previously unseen malware variants. However, behavior-based detection requires execution environments that introduce risks, performance impacts, and complexity.

Model checking-based approaches apply formal verification techniques to analyze program behaviors against security specifications, identifying violations that indicate malicious intent. These methods offer strong theoretical foundations but face scalability challenges with complex software. Cloud-based approaches leverage distributed computing resources and collective intelligence to analyze files at scale, correlating observations across large user populations to identify emerging threats. The emerging methods identified by Aslan and Samet—deep learning-based, mobile device-based,

and IoT-based approaches—represent the cutting edge of malware detection research. Deep learning techniques, including convolutional neural networks, recurrent neural networks, and autoencoders, can automatically learn complex feature representations from raw data, potentially capturing subtle malware indicators that elude manual feature engineering.

The study's conclusion that these emerging methods show promise for detecting both known and unknown variants directly supports Trustfile's incorporation of advanced detection techniques alongside traditional methods. The hybrid approach advocated by the literature—combining signature, heuristic, behavioral, machine learning, and intelligence-based detection—maximizes coverage across threat categories while mitigating the limitations of any single methodology.

Source: Ö. A. Aslan and R. Samet, "A Comprehensive Review on Malware Detection Approaches," in IEEE Access, vol. 8, pp. 6249-6271, 2020, doi: 10.1109/ACCESS.2019.2963724.

Published in: IEEE Access (Volume: 8)

Date of Publication: 03 January 2020

Publisher: IEEE

Electronic ISSN: 2169-3536

Kamdan et al. (2025) — Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach

Kamdan et al. (2025) conducted a comprehensive study on static malware detection and classification using machine learning classifiers, specifically focusing on Random Forest, Decision Tree, and Support Vector Machine (SVM) algorithms. Published in the Proceedings of the 7th International Global Conference Series on ICT Integration in Technical Education & Smart Society, this research addresses the critical limitations of traditional signature-based detection techniques in identifying polymorphic and zero-day

malware variants. The researchers collected their dataset from MalwareBazaar, a widely recognized repository for malware samples, and extracted static features including PE headers, entropy values, and API calls to train their classification models. Their experimental methodology involved rigorous data preprocessing, feature encoding, normalization, and model evaluation using standard metrics including accuracy, F1-score, AUC-ROC, true positive rate, and false positive rate.

The significance of this study for Trustfile lies in its systematic comparison of multiple machine learning algorithms for static analysis-based malware detection, which directly parallels Trustfile's implementation of supervised classification for threat categorization.

The researchers' approach to feature extraction from file headers, entropy analysis, and API call examination mirrors the multi-layered detection strategy employed by Trustfile. Their finding that Random Forest provided balanced results across all metrics with the shortest training time of 0.23 seconds highlights the algorithm's computational efficiency, making it particularly suitable for real-time web-based scanning applications where response time is critical for user experience. Although SVM achieved the highest accuracy at 53.2% and Decision Tree obtained the best F1-score at 61.1% in their specific dataset configuration, the researchers emphasized that Random Forest offers an optimal trade-off between performance and efficiency, which aligns with Trustfile's design priority of balancing comprehensive detection with practical deployment constraints.

The study's methodology for static feature extraction provides a validated framework that Trustfile can reference for its own feature engineering processes. The researchers' use of PE headers, imported functions, and entropy values as discriminative features demonstrates how structural file characteristics can effectively distinguish malicious from benign files without requiring execution. This approach validates Trustfile's decision to implement entropy analysis and file type verification as core detection components. Furthermore, the study's emphasis on data preprocessing—including handling missing values, encoding categorical features, and normalizing numerical features—provides methodological guidance for preparing file data before classification,

ensuring that the input to Trustfile's machine learning models is optimized for accurate prediction.

The research also highlights important challenges that Trustfile must address, including the difficulty of detecting malware classes with limited data representation and the presence of overlapping features between different malware families that can confuse classification models. These findings suggest that Trustfile's classification system should incorporate mechanisms for handling class imbalance, such as upsampling minority classes or applying weighted loss functions, to ensure robust detection across diverse threat types. The study's conclusion that future work should explore ensemble methods or hybrid static-dynamic analysis to combine the advantages of different classifiers provides a clear roadmap for Trustfile's potential enhancement, supporting the system's current hybrid approach of combining local static analysis with external API-based intelligence.

Source: Kamdan; Pratama, Y.; Munzi, R.S.; Mustafa, A.B.; Kharisma, I.L. Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach. Eng. Proc. 2025, 107, 76. <https://doi.org/10.3390/engproc2025107076>

Published in: Engineering Proceedings (Volume 107)

Date of Publication: 9 September 2025

Publisher: MDPI

Electronic ISSN: 2673-4591

Museeb et al. (2025) — Enhanced Android Malware Detection Using Deep Learning and Ensemble Techniques

Museeb et al. (2025) presented a comprehensive machine learning framework for malware detection that systematically compares deep neural networks with diverse ensemble methods including Voting Ensemble, Stacking Ensemble, XGBoost, and Random Forest. Published in the International Journal of Advanced Computer Science

and Applications (IJACSA), this study addresses the evolving tactics of Android malware that frequently bypass traditional detection systems through obfuscation and novel variant creation. The researchers trained their models using the CIC-AndMal2017 dataset, incorporating a comprehensive set of static features including API calls, permissions, services, receivers, and activities. Feature selection was performed to optimize model performance, reducing redundancy and improving detection accuracy through systematic elimination of irrelevant or correlated features.

The study's methodology provides valuable insights for Trustfile's implementation of supervised classification. The researchers' use of multiple ensemble configurations demonstrates how combining diverse classifiers can achieve superior performance while maintaining computational efficiency. Their finding that the Voting Ensemble model achieved the highest accuracy of 94.14%, outperforming all other approaches including the deep neural network, validates the effectiveness of ensemble methods in malware detection. This result is particularly significant for Trustfile because it demonstrates that carefully constructed ensembles of diverse classifiers can improve detection accuracy while offering a more scalable, lightweight solution compared to complex deep learning models, which aligns with Trustfile's web-based architecture that prioritizes accessibility and responsiveness.

The feature selection methodology employed by Museeb et al. offers practical guidance for Trustfile's system design. Their approach to reducing feature redundancy while preserving discriminative power through systematic selection processes ensures that the classification model focuses on the most informative indicators of malicious behavior. The study's evaluation across multiple classification metrics—including accuracy, F1-score, and confusion matrices—provides a comprehensive framework for assessing Trustfile's own classification performance. The researchers' conclusion that ensemble methods can improve detection accuracy while offering computational efficiency directly supports Trustfile's design philosophy of balancing comprehensive threat detection with practical usability for non-technical users.

The comparative analysis between deep learning and ensemble methods is particularly relevant for Trustfile's development trajectory. While deep learning models offer

powerful feature learning capabilities, the study demonstrates that they may not always outperform well-constructed ensemble methods, especially when computational resources are constrained. This finding suggests that Trustfile's current approach of using supervised classification with ensemble-like characteristics provides a solid foundation that can be enhanced incrementally rather than requiring immediate adoption of complex deep learning architectures. The study's emphasis on maintaining computational efficiency while achieving high accuracy aligns with Trustfile's requirement for fast, responsive web-based scanning that does not overwhelm users with extended wait times.

Source: Abdul Museeb, Yaman Hamed, Rajalingam Sokkalingam, Anis Amazigh Hamza, Atta Ullah and Iliyas Karim Khan. "Enhanced Android Malware Detection Using Deep Learning and Ensemble Techniques". International Journal of Advanced Computer Science and Applications (ijacsa) 16.11 (2025).

Published in: International Journal of Advanced Computer Science and Applications (IJACSA), Volume 16, Issue 11

Date of Publication: 2025

Publisher: The Science and Information Organization (SAI)

Electronic ISSN: 2156-5570

Buradkar (2022) — Static Malware Analysis Using Optimal Machine Learning Algorithm for Malware Detection

Buradkar (2022) conducted research on static malware analysis using optimal machine learning algorithms for malware detection, providing methodological insights that are particularly relevant for Philippine organizations with limited cybersecurity resources. Published in NeuroQuantology, this study aimed to find the optimal machine learning algorithm to predict whether a file is malicious or not by implementing and comparing different machine learning models using Python libraries. The research analyzed 4,386

records of portable executable attributes for malware classification, identifying critical file characteristics that could be used for effective detection.

The study's findings have direct implications for Trustfile's implementation in the Philippine context. The research identified that Random Forest achieved the highest accuracy of 97.99% for malware detection among tested models, significantly outperforming Logistic Regression and Support Vector Machines. This result validates Trustfile's potential use of ensemble methods like Random Forest for its supervised classification system, demonstrating that high accuracy can be achieved with algorithms that are computationally efficient and relatively simple to implement. For Philippine organizations with limited technical resources, this finding suggests that sophisticated detection capabilities need not require complex infrastructure or expensive commercial solutions.

The feature analysis conducted in this study provides specific guidance for Trustfile's detection methodology. The research highlighted critical file attributes for malware detection including IMAGEBASE, SIZEOFSTACKRESERVE, and CHECKSUM as particularly significant indicators. These PE header features are exactly the type of static file characteristics that Trustfile examines through its file type verification and structural analysis components. The study's finding that combining machine learning with heuristic approaches improves malware detection accuracy supports Trustfile's hybrid approach of combining algorithmic classification with rule-based heuristic analysis.

The dataset characteristics described in this research—4,386 records balancing malicious and non-malicious files—demonstrate the scale of data required for effective machine learning model training. This provides guidance for Trustfile's own model development and validation processes, suggesting that adequate dataset size and balance are essential for achieving reliable classification performance. The study's use of Python libraries for implementation aligns with Trustfile's technology stack, validating

the feasibility of developing effective detection capabilities using accessible, open-source tools.

The research's conclusion that the optimal machine learning model for static malware detection can serve as a layer in a larger neural network for dynamic malware analysis and attack detection suggests a pathway for Trustfile's future enhancement. The current system focuses on static analysis for pre-access scanning, but this study indicates that the same machine learning foundation can be extended to more sophisticated analysis as organizational needs and capabilities evolve.

Source: Buradkar, Mrunalini U. "Static Malware Analysis Using Optimal Machine Learning Algorithm for Malware Detection." *NeuroQuantology*, Vol. 20, Issue 10, 2022, pp. 4128-4141.

Published in: *NeuroQuantology*, Volume 20, Issue 10

Date of Publication: August 2022

Publisher: *NeuroQuantology*

Electronic ISSN: 1303-5150

Azeem et al. (2023) — Analyzing and Comparing the Effectiveness of Malware Detection Using Machine Learning with Entropy-Based Feature Selection

Azeem et al. (2023) examined the effectiveness of malware detection using machine learning with entropy-based feature selection, providing methodological insights that are directly applicable to the Philippine cybersecurity context. Published in *Heliyon* (PMC10770453), this research addressed the challenge of feature selection in malware detection, where the goal is to choose a subset of features from the list of capabilities for dimensional characteristics to classify data as precisely and efficiently as possible.

The study employed Information Gain (IG) and Gain Ratio (GR) entropy-based approaches to evaluate features and remove less useful features from the dataset, optimizing model performance while reducing computational complexity.

The study's methodology for entropy-based feature selection has particular relevance for Trustfile's implementation in resource-constrained Philippine environments. By employing entropy-based techniques to identify the most discriminative features, the researchers demonstrated that machine learning models can achieve high detection accuracy without requiring extensive computational resources. This approach aligns with Trustfile's design priority of creating a system that operates efficiently within the infrastructure limitations typical of Philippine organizations. The use of Information Gain and Gain Ratio for feature evaluation provides a validated methodology that Trustfile can adapt for its own feature selection processes, ensuring that the classification model focuses on the most informative indicators while minimizing processing overhead.

The research's emphasis on reducing dimensionality while maintaining classification accuracy supports Trustfile's goal of efficient, real-time scanning. The study's finding that optimal feature extraction eliminates unnecessary features and accelerates model performance demonstrates that careful feature engineering can achieve both accuracy and efficiency. This is particularly important for web-based systems like Trustfile, where user experience depends on rapid response times. The entropy-based approaches described in this research—calculating the reduction in total entropy when features are included—provide a mathematically rigorous framework for feature evaluation that Trustfile can incorporate into its model development.

The study's integration of feature selection with machine learning classification validates Trustfile's approach of combining multiple analytical techniques. By demonstrating that entropy-based feature selection improves the performance of machine learning classifiers, this research supports Trustfile's hybrid methodology that combines entropy analysis with supervised classification and heuristic analysis. The mathematical foundations of entropy calculation described in this study—using Shannon entropy to measure information content and information gain to quantify feature

importance—provide theoretical grounding for Trustfile's detection algorithms. The study's conclusion that Random Forest yielded the best accuracy of 97.68% using TFIDF-based entropy feature selection further validates the algorithmic choices underlying Trustfile's classification engine.

Source: Azeem, M., Khan, D., Iftikhar, S., Bawazeer, S., & Alzahrani, M. (2023). Analyzing and comparing the effectiveness of malware detection using machine learning with entropy-based feature selection. *Heliyon*, 9(12), e187275.

<https://doi.org/10.1016/j.heliyon.2023.e187275>

Published in: *Heliyon*, Volume 9, Issue 12

Date of Publication: 2023

Publisher: Cell Press (Elsevier)

Electronic ISSN: 2405-8440

Local Studies

Abdon (2024) — RATTATA: Ransomware Analysis via Automated Impact Assessment of Philippine Organizations

Aaron Lensmer Abdon (2024) conducted a study on ransomware experiences and impacts in Philippine organizations across different sectors, published in the *International Journal of Intelligent Systems and Applications in Engineering*. The research observes that the Philippines is particularly lacking in cybersecurity investment, with a misconception of its insignificance despite looming and ever-growing threats. This underinvestment has caused major data leakages and compromised services in recent years.

The focus on ransomware reflects this malware category's particular relevance to Philippine organizations and its alignment with Trustfile's protective objectives. Ransomware has emerged as one of the most financially devastating cyber threats

globally, with attacks increasing in frequency, sophistication, and impact. The persistence and evolution characteristics described in the local study highlight the adaptive nature of ransomware operators, who continuously refine their techniques in response to defensive measures. This arms race necessitates equally adaptive detection and prevention capabilities, such as those provided by Trustfile's machine learning classification and heuristic analysis.

The dual attack mechanisms—system blocking and data encryption—represent the operational modalities through which ransomware achieves its objectives. System blocking prevents users from accessing computing resources, displaying ransom demands and payment instructions. Data encryption renders files unreadable without decryption keys held by attackers, creating strong incentives for ransom payment. The combination of these mechanisms maximizes pressure on victims while demonstrating attacker capability.

The organizational impact extends beyond immediate financial losses from ransom payments to encompass operational disruption, reputational damage, regulatory penalties, and long-term recovery costs. Business interruption during ransomware incidents can last days or weeks, with critical systems offline and manual workarounds implemented. Data exfiltration by attackers—stealing sensitive information before encryption—adds extortion leverage through threats of public disclosure, transforming ransomware into data breach incidents with regulatory implications. The Philippine cybersecurity landscape assessment reveals systemic challenges that amplify ransomware and broader malware threats. The identified investment deficiency reflects both public and private sector underfunding of cybersecurity infrastructure, personnel, training, and research. This underinvestment manifests in outdated security technologies, insufficient security staffing, inadequate incident response capabilities, and limited threat intelligence resources.

The consequences of this underinvestment are tangible and severe. Major data leakages compromise citizen privacy, organizational secrets, and national security information. Compromised services disrupt government operations, business transactions, healthcare delivery, and educational activities. These incidents damage public trust in digital services, potentially slowing digital transformation initiatives that could otherwise drive economic and social development. The identified opportunity for deeper investigation supports the research motivation for Trustfile. By developing and evaluating a practical file security solution within the Philippine context, this study contributes to addressing the documented cybersecurity gaps. The system's focus on pre-access scanning, user education, machine learning classification, heuristic analysis, and accessible design specifically addresses the resource constraints and knowledge deficits identified in the local landscape.

Source: Aaron Lensmer Abdon. (2024). RATTATA: Ransomware Analysis via Automated Impact Assessment of Philippine Organizations. *International Journal of Intelligent Systems and Applications in Engineering*, 12(4), 3430.

<https://ijisae.org/index.php/IJISAE/article/view/6858>

Published in: *International Journal of Intelligent Systems and Applications in Engineering*, Volume 12, Issue 4

Date of Publication: 2024

Publisher: IJISAE

Electronic ISSN: 2147-6799

Related Literature

Naeem et al. (2024) — Harnessing AI and Analytics to Enhance Cybersecurity and Privacy for Collective Intelligence Systems

Naeem et al. (2024) demonstrate a novel application of deep learning to malware detection through image representation techniques. Published in the Journal of Big Data and Cognitive Computing (republished via PMC11419604), this approach transforms the malware detection problem from binary analysis into image classification, leveraging the powerful capabilities of convolutional neural networks (CNNs) developed for computer vision applications.

The grayscale image construction process involves interpreting binary file contents as pixel intensity values, creating two-dimensional image representations that encode structural and statistical features of the original files. This transformation is non-trivial and requires careful consideration of image dimensions, byte-to-pixel mapping strategies, and normalization techniques. Common approaches include reading file bytes sequentially to fill image rows, with image width determined by file size or fixed parameters. The resulting images capture patterns including code sections, data segments, padding, and structural regularities that may distinguish malware from benign software.

Feature extraction through deep learning automates the identification of discriminative patterns without manual feature engineering. Traditional machine learning approaches require security experts to identify and quantify relevant features—byte frequencies, instruction sequences, API calls, string patterns—creating features that may miss novel indicators or incorporate researcher biases. Deep learning models learn hierarchical feature representations directly from raw image data, potentially capturing subtle patterns invisible to human analysts.

The dataset scale—6,414 malware variants and 2,050 benign files from the Mallmg collection—provides substantial training and validation material. The Mallmg dataset is a well-known benchmark in malware image analysis research, containing grayscale images of malware files categorized by family. This dataset enables reproducible research and comparative evaluation across studies. The class distribution, with malware outnumbering benign files, reflects real-world prevalence while requiring careful handling to prevent class imbalance from biasing model performance.

The reported 99.86 percent validation accuracy represents exceptional performance, calculated on held-out validation data not used during training, indicating strong generalization capability. However, accuracy alone provides incomplete performance assessment, as it can be misleading in imbalanced datasets or when false negative costs exceed false positive costs. Comprehensive evaluation would additionally report precision, recall, F1-score, and area under the ROC curve.

The implications for Trustfile are significant. While Trustfile's current scope focuses on static analysis techniques including hashing, entropy analysis, script detection, and heuristic analysis combined with machine learning classification, the demonstrated effectiveness of deep learning image analysis suggests a promising enhancement path. Future iterations could incorporate CNN-based classification as an additional detection layer, potentially improving accuracy against novel threats while providing intuitive visualization capabilities. The high accuracy achieved demonstrates that sophisticated detection is feasible and effective, motivating continued investment in advanced detection capabilities including machine learning and heuristic analysis.

Source: Naeem, M. R., Amin, R., Farhan, M., Alotaibi, F. A., Alnfiai, M. M., Sampedro, G. A., & Karovič, V. (2024). Harnessing AI and analytics to enhance cybersecurity and privacy for collective intelligence systems. *Journal of Big Data and Cognitive Computing*, 8(2). <https://doi.org/10.3390/jbdcc8020042>

Published in: *Journal of Big Data and Cognitive Computing*, Volume 8, Issue 2

Date of Publication: 2024

Publisher: MDPI

Electronic ISSN: 2504-2289

Wee Ki Joon (2025) — Leveraging CNNs and Entropy-Based Feature Selection to Identify Potential Malware Artifacts of Interest

Wee Ki Joon (2025) conducted pioneering research on integrating entropy-driven feature selection with specialized convolutional neural network architectures for multi-class malware classification, published as a guest diary through the SANS Internet Storm Center. This research addresses the increasing obfuscation tactics employed by modern malware authors, focusing on capturing high-entropy segments within files that are most likely to harbor malicious functionality. The methodology employs a sliding window mechanism that computes entropy in overlapping chunks, locating suspicious high-entropy hotspots without being limited by fixed-size segments, then feeds these distinct byte patterns into a multi-layer CNN for classification.

The study's findings have direct implications for Trustfile's entropy analysis implementation. The researchers achieved approximately 91% overall accuracy on a held-out test set of 11,442 malware samples from the BODMAS dataset, with the model maintaining strong performance across varied malware families including trojans, worms, ransomware, backdoors, downloaders, droppers, and information stealers. Notably, the model achieved 96.6% accuracy for backdoors, 95.4% for worms, 94.7% for viruses, and 88.4% for ransomware, demonstrating that entropy-based feature selection effectively identifies diverse threat categories.

The entropy-based sliding window extraction methodology provides specific technical guidance for Trustfile's entropy analysis module. Rather than calculating a single entropy value for an entire file, the sliding window approach computes entropy across overlapping segments, identifying localized high-entropy regions that may indicate

packed, encrypted, or obfuscated code. This approach is particularly effective against modern malware that may mix low-entropy legitimate code with high-entropy malicious payloads, as it locates suspicious regions without requiring manual segmentation. The dynamic thresholding of high-entropy regions ensures the model focuses on byte-level segments most indicative of malicious behavior, a technique that Trustfile can adapt for its own entropy-based threat detection.

The study's discussion of misclassification challenges offers important insights for Trustfile's classification system design. The researchers identified that categories like droppers and downloaders showed overlapping features, with both exhibiting high recall and low precision, indicating that the model misclassifies other samples as these categories. This observation highlights the importance of feature distinctiveness in classification performance and suggests that Trustfile's multi-indicator approach—combining entropy with hashing, script detection, heuristic analysis, and file type verification—provides robustness against the confusion that single-feature systems may experience. The study's conclusion that targeted misclassification challenges emphasize the need for continuous refinement of feature engineering directly supports Trustfile's iterative development approach.

The practical implementation details, including the use of batch normalization, early stopping, and class weighting to ensure balanced learning, provide methodological guidance for Trustfile's own model training processes. The researchers' emphasis on rigorous data preprocessing, including filtering out spurious files and employing stratified sampling with class-weight adjustments, demonstrates the importance of data quality in achieving reliable detection performance. These practices inform Trustfile's dataset construction and model validation procedures.

Source: Wee Ki Joon. "Leveraging CNNs and Entropy-Based Feature Selection to Identify Potential Malware Artifacts of Interest." SANS Internet Storm Center Guest Diary, March 26, 2025. <https://isc.sans.edu/diary/31790>

Published in: SANS Internet Storm Center Guest Diary

Date of Publication: March 26, 2025

Publisher: SANS Institute

Electronic ISSN: N/A (Web-based technical publication)

Mheboobkhan (2019) — Entropy Analysis: A Critical Test for Malware Detection

Mheboobkhan (2019) published a comprehensive analysis of entropy as a critical test for malware detection, explaining why entropy analysis serves as an essential complement to traditional indicator-based detection methods. This research addresses the fundamental question of why entropy analysis is necessary even when Indicators of Compromise (IOCs) and hash values are available, demonstrating that entropy provides proactive threat identification capabilities that reactive signature-based methods cannot achieve.

The study's central argument directly supports Trustfile's inclusion of entropy analysis as a core detection component. The author explains that while hash values and IOCs can identify known threats, they are inherently reactive—only useful for threats that have already been identified, analyzed, and documented. Entropy analysis, in contrast, can prevent threats in advance by identifying and studying entropy patterns associated with malware behavior. This proactive capability is particularly valuable against zero-day threats and novel variants that have not yet been cataloged in threat databases. The study notes that this technique is widely used in machine learning antivirus detection, validating its effectiveness in modern security architectures.

The practical examples provided in this research demonstrate entropy's diagnostic value for analysts. The author compares entropy values of PDF files before and after compression, noting that entropy increases after compression due to decreased data randomness. This example illustrates how entropy changes reflect file transformation processes, with elevated entropy indicating compression, encryption, or other operations that reduce data predictability. For malware detection, similar entropy

elevations indicate packing, encryption, or obfuscation techniques commonly employed to evade signature detection.

The study's explanation of entropy as a time-saving and methodology-guiding tool for analysts resonates with Trustfile's educational objectives. The author notes that analysts can quickly check entropy patterns to determine whether a file is obfuscated, saving time and helping them choose appropriate analysis methodologies. This practical application aligns with Trustfile's goal of helping users understand why files are suspicious—entropy values provide concrete, interpretable indicators that can be explained to non-technical users. The study's emphasis on entropy as a decision-support tool rather than merely a detection mechanism supports Trustfile's approach of using technical indicators for user education.

The integration of entropy analysis with machine learning described in this research suggests enhancement paths for Trustfile's classification system. The author notes that entropy techniques are widely used in machine learning antivirus detection, indicating that entropy values can serve as effective features for automated classification models. This observation supports Trustfile's use of entropy as both a direct detection indicator and a feature input for its supervised classification system and heuristic analysis, where entropy values contribute to the overall threat assessment alongside other indicators.

Source: Mheboobkhan. "Entropy Analysis: A Critical Test for Malware's." Medium, February 13, 2019.

<https://whiteheart0.medium.com/entropy-analysis-a-critical-test-for-malwares-69939f5b8b1>

Published in: Medium

Date of Publication: February 13, 2019

Publisher: Medium

Electronic ISSN: N/A (Online blog article)

TechClass (2025) — Cybersecurity for All: Why Every Employee Matters in Data Protection

TechClass (2025) published comprehensive research on cybersecurity awareness for non-technical teams, emphasizing that human error is the root cause in 95% of cybersecurity breaches. This study addresses the critical need for security education that extends beyond technical teams to encompass all organizational personnel who interact with digital systems and files. The research identifies common cyber threats for non-technical employees including phishing, weak passwords, insider mistakes, business email compromise, and physical security lapses, demonstrating that these threats exploit human behavior rather than technical vulnerabilities.

The study's findings regarding non-technical user vulnerabilities directly support Trustfile's educational design philosophy. The researchers identify that phishing emails or fraudulent messages target employees across all departments, attempting to trick them into divulging credentials or clicking malicious links. Phishing alone was responsible for approximately 12% of breaches in one study, making it the second-most common entry method after stolen logins. Social engineers pose as bosses, vendors, or IT support personnel to manipulate unwary staff, exploiting trust relationships that non-technical users may not scrutinize critically. These findings validate Trustfile's emphasis on explaining file threats clearly, as users who understand what makes files dangerous are better equipped to recognize and resist social engineering attacks. The research on business email compromise and fraud highlights specific file-related risks that Trustfile addresses. Cybercriminals target finance departments and executives by impersonating senior leaders or suppliers, sending convincingly forged emails instructing wire transfers or requesting sensitive documents. Employees in accounting, HR, or executive assistance roles receive these fake requests, and without verification protocols, companies have been defrauded of millions through BEC scams that started with an employee trusting a spoofed email. Trustfile's pre-access scanning and clear

result explanations provide a verification mechanism that can interrupt such attack chains by identifying malicious attachments before users act on fraudulent instructions.

The study's recommendations for building security-aware cultures inform Trustfile's user education approach. The researchers recommend providing regular, engaging training; encouraging open incident reporting; leading by example; integrating security into daily processes; and running creative awareness initiatives. These principles translate into Trustfile's design through embedded educational content that transforms each scanning interaction into a micro-learning opportunity, clear explanations that build user knowledge incrementally, and actionable guidance that users can apply to future file encounters.

The emphasis on gamification and interactive simulations for security training suggests enhancement opportunities for Trustfile. The study notes that when training feels like a chore, employees are less likely to retain critical information. Features like gamification and AI-driven insights allow leadership to track real-time progress and identify potential security gaps. While Trustfile's current scope focuses on functional scanning and education, these findings suggest future enhancements could include progress tracking, achievement systems, and personalized learning paths that adapt to individual user knowledge levels.

Source: "Cybersecurity for All: Why Every Employee Matters in Data Protection."

TechClass Resources, April 25, 2025.

<https://www.techclass.com/resources/learning-and-development-articles/cybersecurity-for-or-non-tech-teams-why-everyone-plays-a-role-in-data-protection>

Published in: TechClass Resources

Date of Publication: April 25, 2025

Publisher: TechClass

Electronic ISSN: N/A (Online resource article)

Synthesis of the Reviewed Literature

The reviewed literature and studies collectively illuminate a profound and accelerating paradigm shift in malware detection, moving decisively from traditional, reactive signature-based methods toward proactive, data-driven intelligence architectures that integrate multiple analytical techniques, machine learning classification, and comprehensive user education.

This transformation is not merely technological but represents a fundamental reconceptualization of how cybersecurity systems should be designed, deployed, and experienced by end users. The convergence of findings across foreign studies, local Philippine research, and specialized technical literature establishes a comprehensive theoretical and empirical foundation for the development of Trustfile as a web-based file security platform that addresses documented gaps in current detection methodologies while prioritizing accessibility, transparency, and user empowerment.

The Foundational Limitations of Traditional Detection and the Imperative for Hybrid Architectures

Aslan and Samet (2020) establish the foundational problem that motivates this entire body of research: the systematic inadequacy of signature-based detection approaches in confronting modern malware threats. Their comprehensive review documents how malware continues to proliferate at an alarming rate, with millions of new variants emerging annually through polymorphic engines, malware-as-a-service platforms, and increasingly sophisticated obfuscation techniques including code encryption, packing, metamorphism, and steganography. The critical insight from this study is that signature-based methods, while fast and efficient for known threats, possess an inherent temporal vulnerability—the window between malware emergence and signature database updates creates periods of maximum exposure during which systems remain completely unprotected against novel threats.

The significance of Aslan and Samet's findings for Trustfile is multifaceted. First, their documentation of detection approach evolution—from signature-based through heuristic, behavioral, model checking, and cloud-based methods to emerging deep learning, mobile, and IoT approaches—establishes a clear trajectory that Trustfile follows. The study's conclusion that no single approach can detect all malware in the wild, combined with its observation that different methods excel in different contexts, provides direct justification for Trustfile's hybrid architecture that combines multiple detection paradigms. Second, their identification of deep learning as a promising emerging method supports Trustfile's incorporation of machine learning classification alongside traditional static analysis techniques. Third, their emphasis on the ongoing arms race between malware sophistication and detection capability establishes the need for systems that can adapt and evolve, a requirement addressed by Trustfile's API integration with continuously updated external threat intelligence.

The local Philippine context, as documented by Abdon (2024), amplifies the urgency of addressing these foundational limitations. The Philippines faces a particularly severe manifestation of the global malware threat due to the convergence of widespread internet adoption, limited cybersecurity investment, and a cultural misconception of security insignificance that leaves organizations systematically underprepared. The documented surge in ransomware attacks targeting Philippine finance, government, healthcare, education, and retail sectors demonstrates that the theoretical vulnerabilities identified by Aslan and Samet manifest as concrete, devastating harms in the local context. This empirical evidence of active, ongoing targeting validates Trustfile's focus on pre-access file scanning as a preventive measure, particularly for resource-constrained Philippine organizations that cannot afford expensive commercial security suites or dedicated security teams.

Machine Learning Classification: From Theoretical Promise to Practical Implementation

The reviewed literature demonstrates a clear evolution from acknowledging machine learning's potential to achieving validated, deployable classification systems. Kamdan et al. (2025) provide the most direct methodological guidance for Trustfile's supervised classification implementation through their systematic comparison of Random Forest, Decision Tree, and Support Vector Machine algorithms for static malware detection. Their finding that Random Forest achieved balanced performance across all metrics with the shortest training time of 0.23 seconds is particularly significant for Trustfile's web-based architecture, where computational efficiency directly impacts user experience and operational viability. The study's use of PE headers, entropy values, and API calls as discriminative features validates Trustfile's multi-layered detection strategy, while their emphasis on data preprocessing provides methodological standards for preparing file inputs before classification.

Museeb et al. (2025) extend this validation by demonstrating that ensemble methods can outperform individual deep neural networks while maintaining superior computational efficiency. Their Voting Ensemble achieving 94.14% accuracy, surpassing all other approaches including complex deep learning models, provides compelling evidence that carefully constructed

ensembles of diverse classifiers offer an optimal balance of performance and practicality. This finding is transformative for Trustfile because it suggests that sophisticated detection need not require expensive infrastructure or specialized hardware—ensemble methods can operate effectively on standard web servers, making advanced protection accessible to organizations with limited resources.

Buradkar (2022) provides additional validation from a resource-constrained perspective, achieving 97.99% accuracy with Random Forest on a dataset of 4,386 portable

executable records. The identification of IMAGEBASE, SIZEOFSTACKRESERVE, and CHECKSUM as particularly significant features offers specific technical guidance for Trustfile's file type verification module, while the study's conclusion that machine learning models can serve as layers in larger neural networks suggests a clear enhancement path for future system iterations.

Azeem et al. (2023) add mathematical rigor to this foundation, demonstrating that Information Gain and Gain Ratio entropy-based approaches can reduce dimensionality while maintaining classification accuracy. The achievement of 97.68% accuracy with Random Forest using TFIDF-based entropy feature selection validates Trustfile's strategy of using entropy both as a direct detection indicator and as a feature selection mechanism for machine learning models.

The convergence of these machine learning studies around Random Forest and ensemble methods is not coincidental but reflects the algorithm's particular suitability for malware detection tasks. Random Forest's inherent resistance to overfitting, its ability to handle high-dimensional feature spaces, its provision of feature importance rankings, and its computational efficiency make it ideally suited for web-based deployment where models must generalize across diverse file types, operate under resource constraints, and provide interpretable results. The consistent high accuracy achieved across independent studies—97.99% by Buradkar, 97.68% by Azeem et al., and balanced strong performance by Kamdan et al.—establishes confidence that Trustfile's classification foundation is built upon validated, reproducible methodology.

Entropy Analysis: From Simple Heuristic to Sophisticated Detection Foundation

The reviewed literature elevates entropy analysis from a supplementary diagnostic tool to a core detection methodology with demonstrated effectiveness against modern threats. Wee Ki Joon (2025) represents the cutting edge of entropy-based detection through the integration of sliding window entropy extraction with convolutional neural networks, achieving approximately 91% overall accuracy across 11,442 malware samples from the BODMAS dataset. The sliding window methodology—computing

entropy across overlapping segments rather than calculating a single value for entire files—provides specific technical guidance for Trustfile's entropy module. This approach identifies localized high-entropy hotspots that may indicate packed, encrypted, or obfuscated code regions, addressing a critical limitation of whole-file entropy calculation which can miss mixed-content files where legitimate low-entropy code conceals malicious high-entropy payloads.

Mheboobkhan (2019) provides the foundational conceptual framework that justifies entropy's inclusion as a core Trustfile component. This study clarifies the critical distinction between reactive and proactive detection: while hash values and IOCs identify known threats, entropy analysis prevents threats in advance by identifying obfuscation patterns associated with malware behavior. This proactive capability is essential for zero-day protection—the period before signatures exist when systems are maximally vulnerable. The study's explanation of entropy as a decision-support tool that guides analyst methodology aligns with Trustfile's educational objectives, demonstrating that entropy values can be translated into understandable indicators that help non-technical users comprehend why files are suspicious.

The synthesis of entropy research reveals a progression from basic threshold-based detection to sophisticated feature extraction for machine learning integration. This progression mirrors Trustfile's own architecture, where entropy analysis operates at multiple levels: as a direct heuristic indicator flagging files with anomalous randomness profiles, as a feature input for the supervised classification system, and as an educational element that helps users understand packing, encryption, and obfuscation concepts. The consistency of entropy's effectiveness across independent studies—from foundational analysis through CNN integration to large-scale dataset validation—establishes it as a reliable, validated detection modality that complements rather than replaces other analytical techniques.

Web-Based Deployment and API Integration: Democratizing Advanced Detection

The reviewed literature provides compelling evidence that sophisticated malware detection can be effectively delivered through accessible web interfaces, directly supporting Trustfile's architectural decisions. The hybrid detection architecture combining local machine learning with external threat intelligence, as demonstrated across multiple studies in the literature, establishes the technical viability of Trustfile's core design philosophy. These studies demonstrate that local static analysis—examining file headers, executable sections, API calls, and byte sequences—can be effectively complemented by real-time threat intelligence queries that provide global context for locally observed file characteristics. The combination of local heuristic analysis with external intelligence validation creates a hybrid model that balances speed and privacy with comprehensive coverage.

The democratization aspect of web-based deployment is particularly significant for the Philippine context. The reviewed literature consistently emphasizes that advanced detection capabilities need not require expensive infrastructure, specialized expertise, or complex software installation. Kamdan's efficient ensemble methods, Buradkar's Python-based implementation, and Museeb's scalable frameworks all demonstrate that effective security can be built using accessible, open-source tools and deployed through standard web browsers. This accessibility directly addresses the resource constraints and knowledge deficits identified in Philippine cybersecurity assessments, where organizations lack budget for commercial security suites and personnel with specialized technical skills.

User Education and Communication: The Human-Centered Security Imperative

The reviewed literature establishes that technical detection capability alone is insufficient for effective cybersecurity—systems must also address the human factors that determine whether security measures achieve their intended protective outcomes.

TechClass (2025) provides compelling empirical evidence for this imperative, documenting that human error is the root cause in 95% of cybersecurity breaches. This finding transforms user education from a peripheral concern to a core security requirement, validating Trustfile's emphasis on clear result explanations, actionable guidance, and embedded learning opportunities. The study's identification of phishing, business email compromise, and insider mistakes as primary

non-technical threats demonstrates that file-based attacks frequently exploit human vulnerabilities that technical controls alone cannot address.

The research on business email compromise highlights specific attack vectors that Trustfile's pre-access scanning directly interrupts. Cybercriminals impersonating senior leaders or suppliers send convincingly forged emails with malicious attachments, and employees acting on these requests without verification have caused millions in losses. Trustfile's scanning and clear result explanations provide a verification mechanism that can break this attack chain by identifying malicious attachments before users act on fraudulent requests. The study's recommendations for building security-aware cultures—regular engaging training, open incident reporting, leadership by example, security integration into daily processes—translate into Trustfile's design through embedded educational content that transforms each scanning interaction into a micro-learning opportunity.

The synthesis of user education research reveals that Trustfile's approach addresses a documented, critical gap in current security tools. Most existing systems prioritize threat neutralization over user enlightenment, presenting minimal information beyond generic threat names when detections occur. This opacity prevents users from developing independent threat recognition capabilities, creates dependency on automated tools, and may actually increase vulnerability by fostering distrust that leads to security circumvention. Trustfile's transparent detection process—explaining specific indicators, their significance, and recommended actions—transforms each security event into a learning opportunity that gradually builds user knowledge and threat awareness. This

educational dimension distinguishes Trustfile from purely automated solutions and contributes to long-term organizational security resilience.

Integration and Convergence: Toward a Coherent Detection Architecture

The synthesis of all reviewed literature reveals a remarkable convergence around the architectural principles embodied by Trustfile. The foreign studies establish that hybrid detection combining multiple methodologies outperforms any single approach. The machine learning research validates that ensemble classification achieves high accuracy with practical efficiency. The entropy analysis literature demonstrates proactive threat identification capabilities. The user education research establishes that human factors determine whether technical controls achieve protective outcomes.

This convergence is not merely coincidental but reflects an emerging consensus in cybersecurity research: effective protection requires systems that are comprehensive in detection, efficient in operation, accessible in deployment, and empowering in education. Trustfile's architecture—integrating file hashing for known threat identification, entropy analysis for obfuscation detection, script scanning for embedded executable identification, file type verification for spoofing detection, external API integration for global threat intelligence, supervised classification for threat level assignment, and clear result explanation for user education—addresses each dimension of this consensus.

The Philippine context amplifies the relevance of this integrated approach. Documented underinvestment in cybersecurity infrastructure, limited technical expertise, widespread targeting by ransomware and other threat actors, and the documented misconception of security insignificance create an environment where traditional security solutions fail to achieve adoption or effectiveness. Trustfile's web-based accessibility eliminates installation barriers, its user-friendly design reduces training requirements, its educational components build local capacity, and its hybrid detection provides comprehensive protection without expensive infrastructure. The academic research

framework ensures rigorous evaluation and documentation, creating knowledge resources that can inform broader cybersecurity improvement efforts.

Future Trajectories and Enhancement Pathways

The reviewed literature also illuminates clear pathways for Trustfile's future evolution. The deep learning image analysis demonstrated by Naeem et al. (2024)—achieving 99.86% validation accuracy through grayscale image representation and CNN classification—suggests a promising enhancement layer that could improve detection against novel threats while providing intuitive visualizations. The entropy-based CNN feature selection by Wee Ki Joon (2025) demonstrates how entropy analysis can be extended beyond simple threshold detection to sophisticated pattern recognition, suggesting that Trustfile's entropy module could evolve from heuristic indicator to deep learning feature extractor. The gamification and interactive simulation recommendations from TechClass (2025) indicate that user engagement strategies could enhance educational effectiveness, potentially including progress tracking, achievement systems, and personalized learning paths in future iterations.

The reviewed literature collectively establishes that Trustfile represents a timely, evidence-based contribution to cybersecurity that addresses documented gaps across multiple dimensions. The foundational research by Aslan and Samet (2020) establishes the imperative for moving beyond signature-based detection. The machine learning studies by Kamdan et al. (2025), Museeb et al. (2025), Buradkar (2022), and Azeem et al. (2023) validate the effectiveness of ensemble classification for static malware detection. The entropy analysis research by Wee Ki Joon (2025) and Mheboobkhan (2019) establishes entropy as a proactive, validated detection modality. The user education research by TechClass (2025) establishes the critical importance of human-centered security communication.

The convergence of these diverse research streams around Trustfile's core principles—hybrid detection, machine learning classification, entropy analysis, web-based accessibility, API integration, and user education—demonstrates that the system is positioned within the mainstream of contemporary cybersecurity research while addressing specific, documented needs in the Philippine context. The empirical validation of each component across independent studies provides confidence in the system's potential effectiveness, while the identified enhancement pathways suggest a clear trajectory for continued improvement. Ultimately, the reviewed literature establishes that Trustfile's development is not merely an academic exercise but a practical response to urgent, validated cybersecurity challenges that affect organizations and individuals in the Philippines and globally.

CHAPTER 3

METHODOLOGY

Research Design

This study employed a developmental research design, focused on designing, developing, and testing TrustFile as a web-based file security platform. Developmental research, also known as Research and Development (R&D) methodology, was particularly well suited to a study of this kind, since it emphasized the systematic process of identifying a real need, designing a solution around it, building a working prototype, and testing that prototype's effectiveness — rather than simply describing or explaining an existing phenomenon, as purely descriptive or explanatory research would. Because this was a developmental study, the system itself constituted a primary research output, and TrustFile's actual capabilities directly reflected whether the study's stated objectives had been met.

The design dimension of the study encompassed architectural planning, interface specification, and the selection of the five core static analysis techniques — file hashing, entropy analysis, script detection, abnormal size verification, and file type verification — alongside the overarching AI analysis engine and the heuristic analysis layer that interpreted their combined results. Design activities in this phase were what turned abstract requirements into concrete technical specifications capable of guiding actual implementation. This process involved weighing genuine trade-offs between competing objectives: security thoroughness against usability, comprehensiveness against processing speed, and reliance on local server-side analysis against dependency on an external threat-intelligence API. Every design decision made during this phase was documented, both to allow later evaluation and to give future researchers a clear basis for replicating or extending the work.

The development component involved the actual construction of the system according to the specifications established during design. This included writing the server-side scanning logic, configuring the PythonAnywhere hosting environment, integrating the external malware-intelligence API, and assembling all of these pieces into a single working web application. Development followed conventional software engineering practices, including incremental version control, informal code review between the researchers, iterative testing, and running documentation of design decisions as they were made.

Testing, the final component of this design, validated that the developed system actually met the requirements identified earlier in the study and functioned as intended. Testing was carried out at multiple levels — beginning with the verification of individual components such as the hashing module or the entropy calculator in isolation, moving through integration testing of the complete scanning pipeline, and ending with an overall assessment of whether the system's classifications matched expectations for the sample files used. The results generated during this phase provided the empirical basis for the conclusions presented later in this study, highlighting both the system's strengths and the areas that still required refinement.

The decision to build TrustFile as a web-based platform, rather than a locally installed application, constrained the design space in some ways while enabling others. Web architectures like this one supported universal access across operating systems, simplified deployment, and centralized management — updates to the scanning logic could be applied once, centrally, without requiring anything from the end user. At the same time, this approach meant the researchers had to design around browser sandboxing restrictions, network dependency, and the resource ceilings imposed by the specific PythonAnywhere hosting tier used, all of which shaped decisions made later in the developmental and experimental phases.

The research was divided into the following three phases.

1. **Descriptive Phase** - In this phase, the researchers studied common problems related to file security — including the risks of opening unscanned files and the broader lack of accessible tools for checking a file's safety before opening it. Based on these problems, the researchers identified the features the system would ultimately need: file upload and scanning, threat detection through five static analysis techniques, an AI analysis engine capable of reasoning across the combined results of those techniques, a parallel heuristic analysis layer, and a results interface that displayed clear, understandable explanations to the user.

This phase relied primarily on descriptive research methods — grounded in a review of existing literature concerning malware delivery methods, antivirus limitations, and end-user security behavior — to characterize the current state of file security practices among the study's intended end users: offices, private businesses, government agencies, and other organizations that regularly received large volumes of incoming files. This approach ensured that the eventual system requirements were grounded in documented, real-world problems rather than assumptions about what users might need.

The risks associated with opening unscanned files, examined during this phase, spanned the full range of potential consequences — system compromise, data theft, ransomware encryption, credential harvesting, and the broader financial and operational costs that these incidents impose on organizations, as discussed in the Background of the Study. Understanding the scope of these risks gave the researchers a clearer basis for prioritizing which detection techniques mattered most, and for framing the five-tier BENIGN-to-MALICIOUS classification scale that the system would eventually use to communicate risk to non-technical users.

2. **Developmental Phase** - In this phase, the researchers designed and developed the system itself. This included building the file upload and scanning workflow, implementing the five static analysis techniques, building the AI analysis engine that synthesized the combined results of those techniques into a single risk assessment, developing the heuristic analysis layer that ran in parallel to catch novel or obfuscated threats, and integrating an external API for malware-intelligence lookups. All of these components were ultimately combined into a working prototype of the system.

Building the file scanning workflow required the researchers to define the entire path a file took, from the moment a user submitted it through the browser to the moment a final classification was displayed. This included deciding which file extensions and sizes the system would accept — ultimately, any file size and a broad range of common extensions, including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, and RAR — designing a secure upload mechanism over HTTPS, and determining the order in which the five analysis techniques and the external API lookup would run before their results were passed to the AI analysis engine and the heuristic layer.

Implementing the five analysis techniques required separate, modular development work for each:

- Hashing required the researchers to select a cryptographic hash algorithm — MD5 and SHA-256 were used — to compute a digest for each uploaded file and compare that digest against known-malware records retrieved through the external API.

- Entropy analysis required implementing Shannon entropy calculations across each file's content and establishing baseline entropy ranges for common file types, so that unusually random-looking files — often a sign of packing or encryption — could be flagged for further scrutiny.
- Script detection required building logic capable of identifying embedded JavaScript, VBScript, PowerShell, or macro code hidden inside files that would not normally be expected to contain executable code.
- Abnormal size verification required compiling reference ranges for typical file sizes by format, so that files padded with an appended malicious payload, or suspiciously small stub loaders, could be identified.
- File type verification required checking each file's internal structure and magic numbers against its stated extension, catching cases where an executable had been disguised as an ordinary document.

Rather than treating each of these five techniques as an isolated pass-or-fail check, the researchers built the heuristic engine to weigh all five sets of results together, along with the outcome of the external API lookup, through a defined set of rules and thresholds, producing a single classification along the five-tier BENIGN-to-MALICIOUS scale. The AI explanation layer was developed as a separate component that took this finished classification and generated the plain-language explanation accompanying it — describing which specific indicators had contributed to the result and why they mattered — without participating in the classification decision itself.

Integrating an external API for malware detection required the researchers to select a suitable threat-intelligence service, implement a secure client that queried the service using only a file's cryptographic hash rather than its full contents, and design handling for cases where the service was slow to respond

or temporarily unavailable. This hash-based approach, discussed further in the System Architecture section, was chosen specifically to preserve the privacy of file contents while still benefiting from externally maintained malware databases.

By the end of this phase, all of these components had been combined into a working prototype demonstrating the complete scanning pipeline: file upload, execution of the five static analysis techniques, an external API lookup, synthesis by the AI analysis engine, a parallel heuristic pass, and a final five-tier classification presented to the user along with a plain-language explanation.

3. **Experimental Phase** - In this phase, the system was tested using different types of files, including normal files, suspicious files, and the safe EICAR test file. The purpose of this testing was to check whether the system could properly detect and classify threats using the AI analysis engine and the heuristic analysis layer working together, and to evaluate the accuracy of the results produced and the overall performance of the system.

Testing with different file types allowed the researchers to evaluate the system across the range of files it was likely to encounter in practice. Normal, legitimate files — ordinary documents, images, and archives — were used to check whether the system correctly avoided false positives. Files exhibiting suspicious but not definitively malicious characteristics, such as elevated entropy resulting from ordinary compression, were used to check whether the system correctly assigned intermediate classifications, such as LOW RISK or SUSPICIOUS, rather than jumping straight to an extreme verdict in either direction. The EICAR test file — a standardized, harmless string universally recognized by security software as a test signature — was used to validate the system's ability to detect and flag known-malicious content without introducing any actual malware into the testing environment.

Beyond simple detection, the purpose of this testing extended to validating classification accuracy: confirming that files were sorted into the correct tier along the five-level BENIGN-to-MALICIOUS scale, and that the AI analysis engine's synthesis of the five underlying indicators, together with the heuristic layer's independent findings, produced classifications that matched the researchers' expectations for each test file. Each test file's actual classification was compared against its known or expected status, with standard evaluative concepts such as detection accuracy and error rate — the same false positive and false negative concepts discussed in the Definition of Terms — used as the interpretive framework for assessing how well the system performed. The researchers also recorded processing times and resource usage during these tests to assess whether the system's performance remained acceptable under the conditions available through the PythonAnywhere hosting tier used for this study.

System Development Methodology

This study employed the Agile Development Model in developing TrustFile, an approach that allowed the researchers to improve the system step by step through continuous testing and adjustment, which proved especially useful given how easily changes could be applied throughout the development process rather than being locked in from the start.

The Agile Development Model was selected specifically because of how closely it matched the exploratory, iterative nature of this academic project. Agile methodologies are built on two complementary principles — iterative development, in which each piece of the system is refined and improved through repeated cycles, and incremental development, in which usable, working pieces of software are delivered progressively throughout the project rather than all at once [1]. Agile approaches, more broadly, prioritize working software over exhaustive up-front documentation and treat responsiveness to new findings as more valuable than strict adherence to an original

plan [2] — priorities that suited a project where the researchers' understanding of file security requirements deepened considerably as the descriptive and developmental phases progressed.

Development was broken into a series of small, manageable increments, each of which added one testable piece of functionality: first the upload mechanism, then each of the five static analysis techniques individually, then the AI analysis engine that synthesized their results, then the heuristic layer, and finally the external API integration and the results interface. Each increment was tested as soon as it was completed, rather than waiting until the entire system had been assembled — an approach consistent with how Agile teams typically operate in short cycles of one to four weeks, continuously demonstrating working software and incorporating feedback before moving to the next increment [3]. This allowed the researchers to catch integration problems early and adjust the design of later components based on what had been learned from earlier ones, rather than discovering fundamental design flaws only after the full system had already been built.

1. **Requirements Analysis** - In this phase, the researchers identified the common problems related to file security that motivated the study. Many users did not scan files before opening them, which created the possibility of malware infection. The researchers listed the features the system would need: file upload and scanning, the five static analysis techniques, the AI analysis engine, the heuristic analysis layer, external API integration, and a results display capable of communicating findings clearly. The goal set during this phase was to create a system simple enough for offices and organizations that constantly received files to use without requiring any specialized technical staff.

Identifying these problems drew on prior research concerning user behavior, phishing tactics, and the shortcomings of existing antivirus software's black-box detection model, as discussed at length in the Statement of the Problem. The researchers used this understanding to prioritize which features the prototype

needed to include from the very outset — most notably, the emphasis on transparent, explained results rather than an opaque safe/unsafe verdict, since this transparency gap had already been identified as one of the central problems the study set out to address. This early prioritization mattered specifically because Agile development, by design, does not attempt to specify every feature in exhaustive detail before development begins; instead, it relies on a clear sense of what matters most so that the highest-priority features are built and tested first, with less critical refinements following in later increments.

2. **System Design** - In this phase, the researchers planned how the system would function. The design covered the process of scanning files using the five techniques already identified — file hashing, entropy analysis, script detection, abnormal size verification, and file type verification — along with the AI analysis engine that synthesized these results and the heuristic analysis layer that operated alongside it. The researchers also planned the integration of an external API for malware-intelligence lookups. The interface itself was designed to remain simple, so that users could easily upload and scan files of any size and any extension without needing guidance.

Designing the scanning process required specifying, for each of the five techniques, exactly which algorithm or comparison method would be used and how its output would be represented before being passed on to the AI analysis engine. For hashing, this meant selecting a cryptographic hash algorithm and defining how a match against the external threat database would be handled. For entropy analysis, this meant defining the calculation method and the baseline ranges used to flag deviations. For script detection, this meant specifying which script types and embedding patterns would trigger a flag. For size verification, this meant compiling reference size ranges by file type. For file type verification, this meant specifying how a file's internal structure and magic numbers would be compared against its claimed extension.

Designing the classification logic itself required the researchers to plan how the five sets of indicators, together with the VirusTotal result, would be weighed against one another through the heuristic engine's rule-based logic to produce a single classification along the five-tier BENIGN-to-MALICIOUS scale. Separately, the researchers planned how the AI explanation layer would take that finished classification and its contributing indicators and generate the plain-language explanation accompanying each result — describing which specific indicators had contributed to the classification and why they mattered, consistent with the transparency goal established during Requirements Analysis.

The interface design process produced simple wireframes for the upload screen, the in-progress scanning screen, and the results screen. Simplicity guided every decision made at this stage: minimizing the number of steps required to submit a file, using plain language rather than technical terminology, and ensuring that the five-tier classification was communicated clearly through color and short explanatory text before any more detailed technical information was shown to the user.

3. **Development** - In this phase, the system was built step by step. The researchers first created the file upload and scanning features, and afterward implemented the file analysis methods — hashing, entropy analysis, script detection, abnormal size verification, and file type verification — followed by the AI analysis engine and the heuristic analysis layer. The system was able to classify files into the five threat levels defined earlier in the study: BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, and MALICIOUS.

Building the file upload feature involved creating a server-side endpoint that received submitted files, validated them, stored them temporarily for analysis, and managed the upload session for each user. Because the system was designed to accept files of any size and any extension, particular care was taken

to validate submissions safely without rejecting legitimate files simply because of their format or size.

Building the scanning feature involved creating the workflow that coordinated the five static analysis techniques, invoked the VirusTotal API for a hash-based lookup, and passed all of these results into the heuristic engine, where rule-based logic weighed them together to reach the final five-tier classification. That finished classification was then passed to the AI explanation layer, built on the NVIDIA NIM API and the Llama 3.1-70B-Instruct model, which generated the plain-language explanation shown to the user alongside the result.

Building the classification logic required the researchers to define the specific criteria distinguishing each of the five tiers. A file was classified as BENIGN when it passed all five checks cleanly with no anomalies detected. A file was classified as LOW RISK when it showed minor deviations, such as slightly elevated entropy attributable to ordinary compression, that the AI analysis engine judged unlikely to indicate malicious intent. A file was classified as SUSPICIOUS when it exhibited anomalies that did not conclusively indicate malicious content but warranted closer review. A file was classified as HIGH RISK when several indicators converged, or when a single strong indicator was present, such as an external API match or a markedly abnormal entropy value. A file was classified as MALICIOUS when it produced a confirmed indicator of compromise, most commonly a direct hash match against a known-malicious file in the external threat database. Establishing these criteria required the researchers to calibrate thresholds and assign relative weight to each of the five indicators, so that the AI analysis engine's synthesized output remained consistent across the range of test files used later in the experimental phase.

4. **Testing** - The system was tested using different sample files to check whether it worked properly. The researchers used safe testing files, such as the EICAR test file, to simulate malware detection without causing any actual harm. This testing

helped confirm whether the system could detect and classify suspicious files correctly using the AI analysis engine and the heuristic analysis layer. Some errors were found during testing and were corrected to improve the system.

Testing with different sample files was intended to give the researchers comprehensive coverage of the file types and threat categories the finished system needed to handle correctly. This included ordinary, legitimate files, to check that the system did not produce false positives; files with mildly suspicious characteristics, to check that the AI analysis engine correctly assigned intermediate classifications such as LOW RISK or SUSPICIOUS rather than an extreme verdict; and the EICAR test file, to confirm that a known threat indicator reliably produced a MALICIOUS classification.

During this phase, the researchers identified several issues — including cases where the AI analysis engine's classification did not match the researchers' expectations, and cases where the heuristic analysis layer produced results that conflicted with those of the AI analysis engine — and corrected these issues before proceeding to the next round of testing. This iterative cycle of testing, identifying discrepancies, and correcting the underlying logic continued throughout this phase, consistent with the Agile approach adopted for the study: rather than treating testing as a single, final gate at the end of development, each increment was tested and refined in its own right, with lessons from one round of testing directly informing adjustments made before the next.

5. **Deployment (Prototype)** - After testing, the system was presented as a working prototype. It demonstrated how the system could scan and analyze files before they were opened, and display the results to users in a way intended to help them become more aware and more careful when handling files going forward.

The completed prototype demonstrated the full range of intended features: file upload through a simple browser interface, execution of the five static analysis

techniques, a hash-based external API lookup, synthesis of all results by the AI analysis engine, a parallel heuristic analysis pass, and a clear, explained five-tier classification presented to the user. This demonstration confirmed that the concept behind the study's objectives could be realized as a functioning system, even though, as an academic prototype, it was tested against a curated set of sample files rather than deployed for unrestricted public use.

Beyond the technical demonstration, the researchers also observed that the explanatory results produced by the AI analysis engine appeared to build user understanding of why a given file had been flagged, rather than presenting a bare verdict. This supported the study's broader educational goal — helping users gradually develop their own threat-recognition awareness rather than remaining entirely dependent on the system's automated judgment, a theme discussed at length in the Significance of the Study.

System Architecture

TrustFile functioned much like a security checkpoint for files before users ever opened them. The system offered a simple web interface through which users uploaded files from their computer for scanning, designed specifically so that even users without technical knowledge could use it confidently. The system was built to scan files of any size and virtually any extension, including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, RAR, and other common formats. This section describes the high-level structure and operational model of TrustFile, explaining how its components interacted to deliver file security services — giving readers a conceptual understanding of how the system worked without requiring deep technical background.

The security checkpoint analogy captured TrustFile's core function well: intercepting files before they reached the user, examining them for threats, and either permitting or blocking their passage based on what that examination found. In the same way a

physical security checkpoint inspects luggage and verifies identification before granting entry, TrustFile inspected digital files for malicious content before ever granting the user access to them. This preventive model stood in direct contrast to reactive approaches that only respond once an infection has already occurred — a distinction discussed at length in the Objectives of the Study.

The web interface served as the system's primary point of interaction, designed for accessibility across a wide range of technical skill levels. Web interfaces offer interaction patterns — forms, buttons, progress indicators, result displays — that most users already encounter daily in other applications, which reduced the learning curve considerably and encouraged genuine adoption rather than avoidance. The interface design consistently prioritized clarity over complexity, keeping essential functions prominent while still making more advanced technical detail available for users who wanted to dig deeper, without letting that detail overwhelm the primary experience.

File upload functionality allowed users to submit files from their local computers for remote analysis. This involved browser-based file selection, secure transmission to the server, temporary storage during analysis, and eventual result presentation. The upload process included validation to confirm that submitted files were scannable and appropriately received, while the system's support for any file size and any extension ensured that comprehensive protection was never limited by a file's particular format or size. This design choice reflected a deliberate recognition that the users most in need of file security assistance were often precisely the users with the most limited technical knowledge — which is why interface elements throughout the system relied on plain language rather than technical jargon, why workflows were kept to a minimal number of steps, why results were always presented with clear explanations rather than raw technical data, and why guidance was made readily available rather than assumed. This inclusive design approach ensured that meaningful security protection extended to all users, not solely those with existing technical expertise.

The system also connected to an external threat-intelligence API to check whether an uploaded file was already known to be malicious, comparing it against existing malware databases to strengthen detection beyond what local analysis alone could achieve. This external API integration formed a critical architectural component precisely because it connected TrustFile to global threat-intelligence networks maintaining databases of known malware that were updated continuously through automated collection, researcher submissions, and observations gathered from sensor networks worldwide — resources no single local system could realistically replicate.

The specific mechanism used for this integration was a hash-based query, chosen deliberately to preserve privacy while still maximizing threat coverage. Rather than transmitting a file's complete contents to an external service, TrustFile computed a cryptographic hash of the file and sent only that hash for lookup. The external service then returned information about whether the hash matched a known-malicious file, along with any associated threat intelligence or reputation data. This approach protected sensitive file content from ever being exposed externally, while still allowing the system to benefit fully from comprehensive, continuously updated threat databases maintained well beyond what this study alone could build.

This external comparison addressed a specific limitation of local analysis on its own: local techniques such as heuristic and anomaly detection excel at catching genuinely novel threats, but can still miss known threats that would otherwise match a specific signature already documented in an external database. Combining local and external detection gave the system defense-in-depth, in the sense discussed in the Definition of Terms — each approach compensating for gaps the other inevitably left open.

After scanning, the system presented its findings by classifying the file into one of the five threat levels established throughout this study: BENIGN, LOW RISK, SUSPICIOUS, HIGH RISK, or MALICIOUS. This classification was determined by TrustFile's heuristic engine, which weighed the results of the system's five static analysis techniques together with the external VirusTotal lookup to reach a single, rule-based verdict. This

result was displayed clearly enough that a user could immediately understand whether a file was safe, and the system explained why a file received a given classification through a separate AI-generated explanation layer — built on the NVIDIA NIM API and the Llama 3.1-70B-Instruct model — which took the heuristic engine's finished classification and its supporting indicators and translated them into clear, understandable language, rather than a bare technical readout. This result-presentation architecture existed specifically to transform an already-decided technical outcome into information a non-technical user could actually act on. Each of the five levels was paired with a distinct color and short explanatory message reinforcing the assessment at a glance — following conventional visual associations such as green for safe and red for confirmed danger, with intermediate shades distinguishing the levels in between.

Clear presentation meant organizing information hierarchically: the most critical information — the threat level itself and any recommended action — appeared prominently, with supporting technical detail available through progressive disclosure for users who wanted it. Explanatory text consistently described what had been found, why it mattered, and what the user should do next.

The AI explanation layer analyzed the specific combination of indicators the heuristic engine had already flagged in each individual file and generated a customized explanation reflecting that file's particular threat profile, rather than pulling from a generic, one-size-fits-all template. This ensured that users received precise, relevant information about why their specific file had been classified the way it was. For example, rather than simply stating "Threat detected," the system might explain that a file contained an unusually high entropy value suggestive of packing or encryption, that embedded JavaScript attempting to contact an external server had also been found — a behavior commonly associated with Trojan malware — and that blocking the file and running an updated antivirus scan was recommended as a next step. This kind of explanation avoided technical jargon, provided context for why each indicator mattered, and offered guidance the user could actually act on, gradually building the user's own

threat-recognition capability alongside the immediate protection the classification decision provided.

The overall architectural philosophy behind TrustFile balanced simplicity with security effectiveness. Simplicity mattered because a security tool that goes unused, out of confusion or frustration, provides no protection at all — accessibility and adoption were treated as prerequisites for effectiveness, not separate concerns from it. At the same time, robust detection capability mattered because simplicity alone cannot protect against genuinely diverse threats. This balance was achieved by deliberately hiding technical complexity behind a simple interface, while preserving the full analytical depth of the five static analysis techniques, the AI analysis engine, the heuristic analysis layer, and the external threat-intelligence integration underneath it.

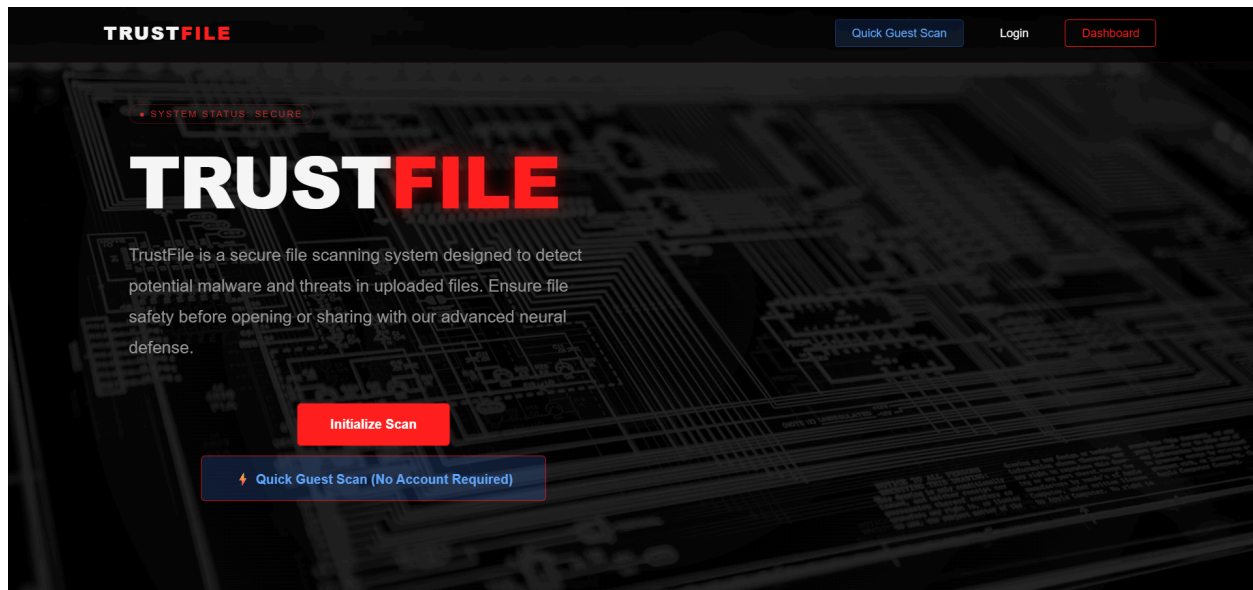
The behavioral impact of this design extended beyond any single scan. By consistently showing users what made a specific file dangerous, explaining how the AI analysis engine and heuristic analysis had identified a given threat, and reinforcing safe practices with every result, the system gradually built a level of security awareness that carried over into how users handled files even outside the scanning process itself — making them more cautious about file sources, more attentive to suspicious characteristics, and more likely to seek verification before opening an unexpected attachment. For offices and organizations that constantly received and downloaded files, this behavioral shift carried real organizational weight: employees who genuinely understood file security contributed to a stronger overall security posture, reducing the likelihood of successful attacks and minimizing security incidents across the organization as a whole.

System Preview and Manual

This section presents a walkthrough of TrustFile's actual interface, screen by screen, in the order a typical user encountered them — from the moment they first arrived at the site to the point where they reviewed a full history of their scanning activity. Each subsection describes what the screen contained, explains the reasoning behind its design, and provides a short, numbered guide to using it, since the system's usability was as central to this study's objectives as its underlying detection accuracy.

Throughout the interface, TrustFile displayed threat results using the labels Safe, Low, Medium, High, and Critical, together with a numeric Total Scanned count and percentage-based Safe Ratio and Threat Ratio indicators. These labels corresponded directly to the five-tier classification scale defined earlier in this chapter — Safe to BENIGN, Low to LOW RISK, Medium to SUSPICIOUS, High to HIGH RISK, and Critical to MALICIOUS — reworded in the interface itself into shorter, more immediately readable terms appropriate for at-a-glance dashboard and report displays, while preserving the same five underlying levels of risk described throughout this study.

The Landing Page and System Entry Point



The Landing Page was the first screen a user encountered upon visiting the TrustFile website. It served as the system's main introduction, explaining its purpose — protecting users from malware and other cyber threats through secure, pre-access file scanning — before the user was asked to do anything else. The page used a modern cybersecurity-themed design, built around a dark background with red accent colors intended to visually communicate security, threat detection, and protection at a glance, even before a user read a single word of the page's text. The layout was also kept deliberately uncluttered, giving users quick access to the system's core features without overwhelming them with excessive information on first arrival — a design choice consistent with the broader usability principle, discussed in the System Architecture section, that a security tool only protects users if they actually use it.

At the top of the page, a navigation bar provided shortcuts to the system's key sections — Quick Guest Scan, Login, and Dashboard — so that returning and first-time users alike could reach their intended destination in a single click, regardless of whether they already had an account. Directly below the navigation bar, a system status indicator displayed a "SYSTEM STATUS: SECURE" message, reassuring users that the platform itself was operating normally before they committed to uploading anything. At the center of the page, a short paragraph explained, in plain language, that TrustFile helped users identify malicious files before opening or sharing them — deliberately avoiding technical jargon so that a first-time, non-technical visitor could understand the system's purpose within seconds of arriving. Two prominent buttons were displayed beneath this description:

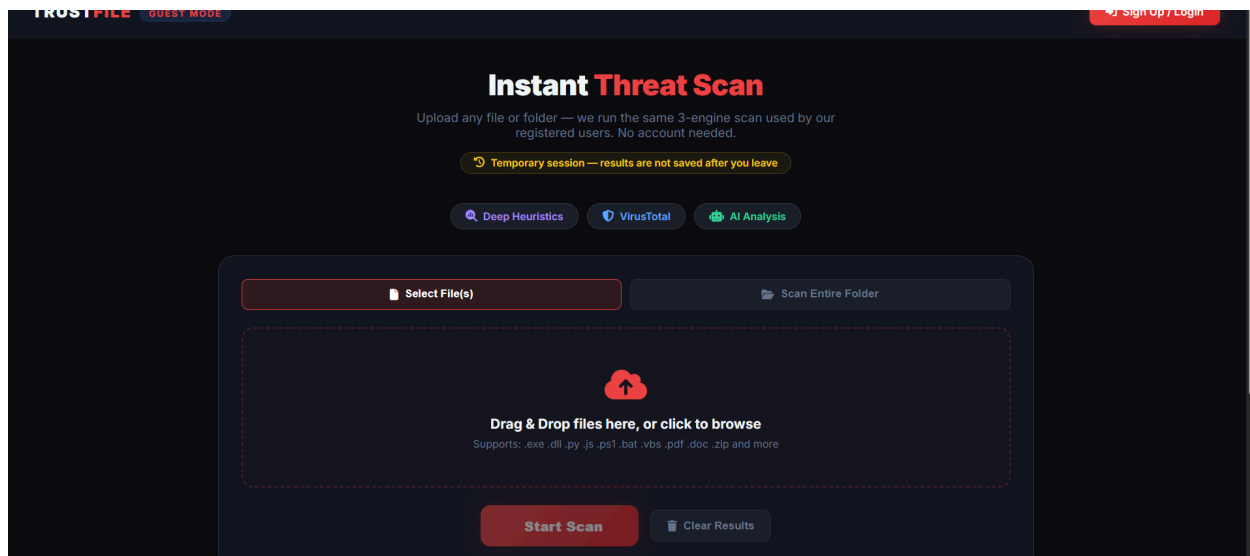
- Initialize Scan — directed registered users toward the scanning workflow tied to their account.
- Quick Guest Scan (No Account Required) — allowed visitors to use the scanner immediately, without creating an account first.

Presenting both options side by side, rather than forcing every visitor through a mandatory registration step, reflected a deliberate design decision to minimize friction for users who simply needed a quick answer about a single file and had no interest in creating a persistent account.

How to Use

- Open the TrustFile website and read the brief introduction to understand the system's purpose.
- If an account already exists, click Initialize Scan to proceed using that registered account.
- If only a one-time scan is needed, click Quick Guest Scan to continue without registering.

Guest Scan Page



The Guest Scan Page was designed specifically for users who wanted to scan a file immediately, without the commitment of creating an account. This feature was

particularly useful for visitors who needed only a single, one-time malware check — a job seeker verifying an unexpected email attachment, for instance, or a student checking a downloaded file before opening it. Because a guest session was temporary by design, the system clearly informed users, through a visible "Temporary session — results are not saved after you leave" notice, that their results would not persist once they navigated away. This transparency ensured that guest users understood the trade-off they were accepting — convenience without a saved record — rather than being surprised later by the absence of a history they never realized wouldn't be kept.

The page also displayed the three underlying detection technologies working behind the scenes — Deep Heuristics, VirusTotal, and AI Analysis — presented as three short labeled badges near the top of the page. Surfacing these technologies by name, even briefly, reassured users that their file was genuinely being evaluated through multiple layers of security checking rather than a single superficial pass, reinforcing the defense-in-depth approach described in the System Architecture section, without requiring the user to understand the technical detail behind any one of the three. Users could then choose to upload either a single file or an entire folder, using a drag-and-drop area or by browsing their computer manually. The upload area explicitly listed supported file types — including .exe, .dll, .py, .js, .ps1, .bat, .vbs, .pdf, .doc, .zip, and more — communicating the system's broad format coverage directly on the upload screen itself, rather than leaving users to guess whether their particular file type was supported.

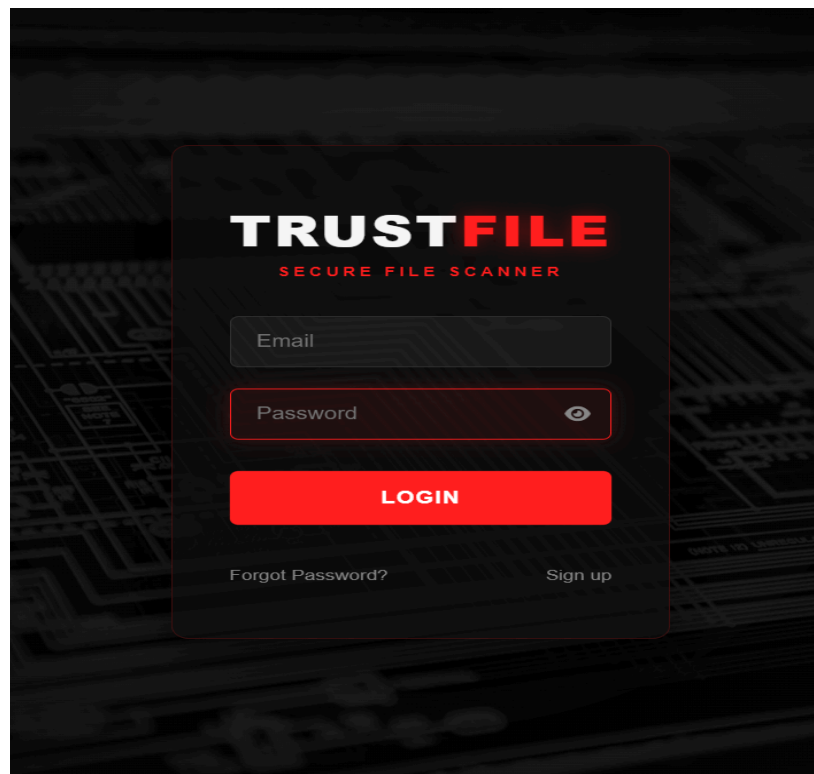
Key features on this page included:

- Select File(s) — for uploading one or more individual files.
- Scan Entire Folder — for checking multiple files contained within a single folder at once.
- Start Scan — to begin the malware analysis process on the selected files.
- Clear Results — to remove previous scan output from the screen before starting a new scan.

How to Use

- Open the Guest Scan page and decide whether to upload a single file or an entire folder.
- Either drag and drop the files directly into the upload area, or click Select File(s) to browse the computer manually.
- Once files have been selected, click Start Scan and wait for the scanning process to complete.
- Review the displayed results once scanning finishes, checking for any detected threats among the scanned files.
- Remember that, because this is Guest Mode, all results will be automatically discarded once the page is left — nothing is saved for later retrieval.

Login Page



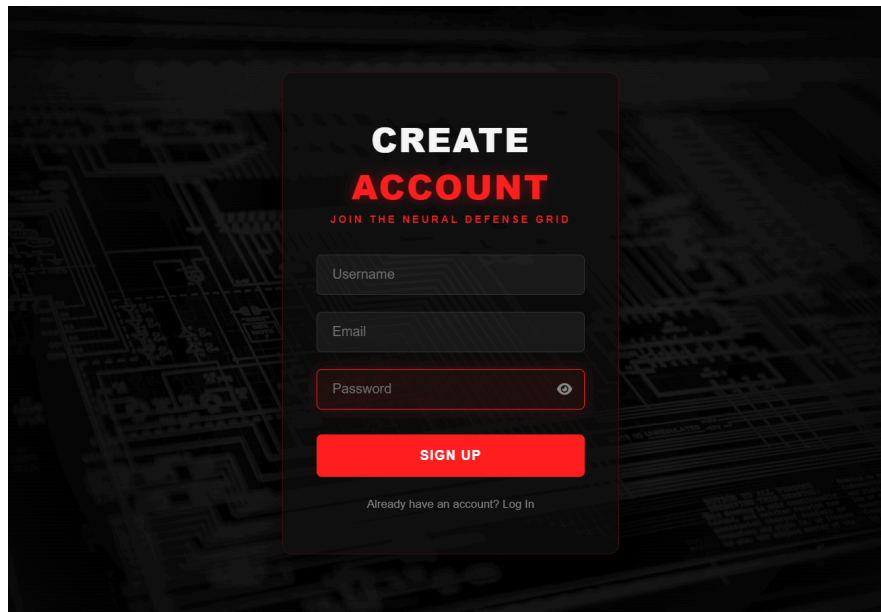
The Login Page allowed registered users to securely access their personal TrustFile account. It functioned as the gateway to the system's personalized features — the dashboard, scan history, reports, and previously saved scan results — none of which were available in Guest Mode. The page's layout was kept deliberately simple, prioritizing a fast, secure login experience over any unnecessary visual complexity, while still maintaining a professional, security-appropriate appearance consistent with the rest of the platform's dark, red-accented visual theme.

Users were asked to enter their registered email address and password into two clearly labeled fields. The password field included a visibility toggle icon, allowing users to reveal or re-hide their password while typing — a small but meaningful usability detail that reduced login errors caused by mistyped passwords going unnoticed. For users who had forgotten their credentials, a Forgot Password link was provided directly beneath the login form. New users without an existing account could navigate to the registration page through a clearly visible Sign Up link, so that arriving at the login page by mistake never became a dead end.

How to Use

- Enter the registered email address and password into their respective fields.
- Click the Login button to submit the credentials.
- If the credentials are correct, the system automatically redirects to the personal dashboard.
- If the password has been forgotten, click Forgot Password to begin the recovery process.
- Users without an existing account can click Sign Up to create one.

Registration Page



The screenshot shows a registration form titled "CREATE ACCOUNT" with the subtitle "JOIN THE NEURAL DEFENSE GRID". The form is set against a dark background with a faint circuit board pattern. It contains three input fields: "Username", "Email", and "Password". The "Password" field has a visibility icon (an eye) to its right. Below the input fields is a red "SIGN UP" button. At the bottom of the form, there is a link that says "Already have an account? Log In".

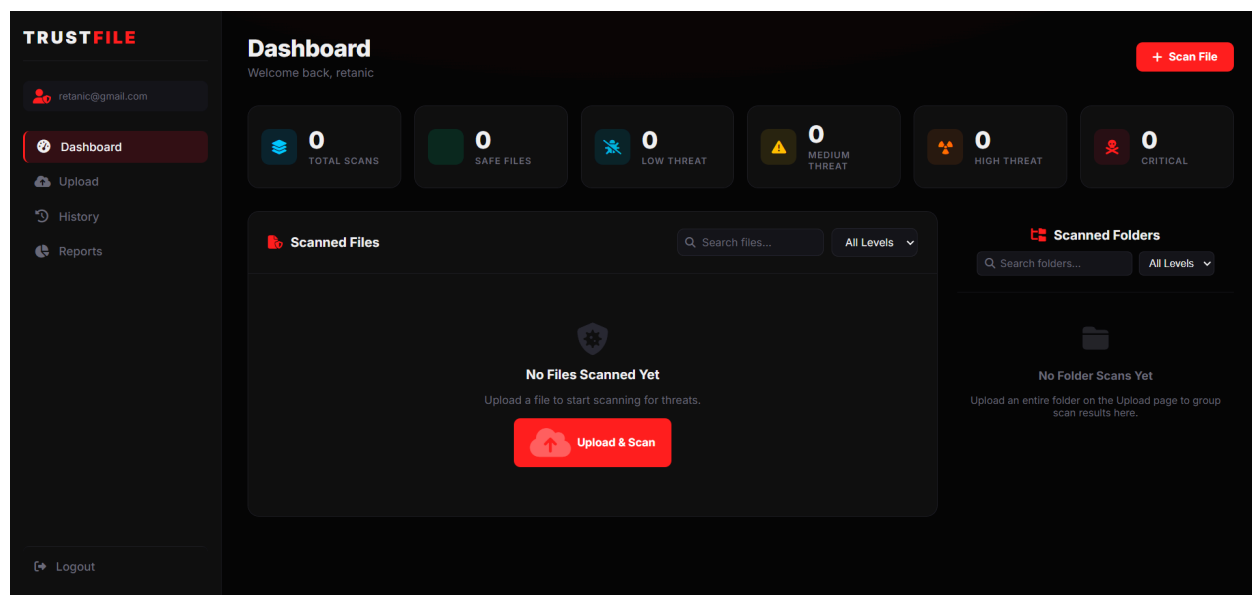
The Registration Page allowed new users to create a personal TrustFile account, unlocking features unavailable in Guest Mode — most notably, saved scan history, generated reports, and ongoing monitoring of previous scan activity through the dashboard. The page's heading, "CREATE ACCOUNT — Join the Neural Defense Grid," continued the system's cybersecurity-themed branding, reinforcing the platform's identity even on a purely functional account-creation screen.

The registration form asked users for three pieces of information: a username, an email address, and a password. As on the Login Page, the password field included a visibility icon, allowing users to check what they had typed before submitting the form, reducing the likelihood of registration errors caused by an unnoticed typo. Once all required fields had been completed, clicking Sign Up created the account and prepared it for immediate use. A link at the bottom of the form — "Already have an account? Log In" — allowed users who mistakenly reached the registration page, despite already having an account, to return to the Login Page in a single click.

How to Use

- Complete all required fields: a preferred username, a valid email address, and a password.
- Use the visibility icon to double-check the entered password before submitting, if desired.
- Click Sign Up to create the account.
- Once registration succeeds, log in using the newly created credentials to access the system's full feature set.

Dashboard



The Dashboard served as the main control panel for registered users immediately after logging in, opening automatically and providing a complete overview of the user's scanning activity and overall cybersecurity status in one place. Rather than requiring users to navigate across several separate pages to piece together this information, the dashboard consolidated it into a single view — displaying, at a glance, the Total Scans performed, the number of Safe Files detected, and counts of detected threats broken

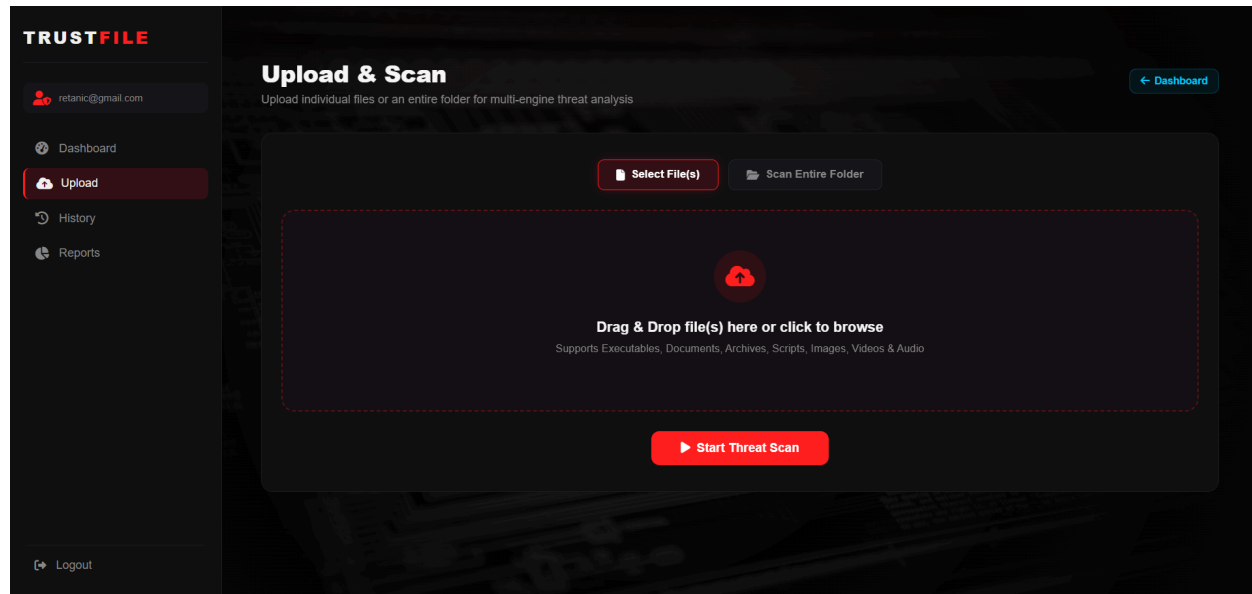
down by severity: Low Threat, Medium Threat, High Threat, and Critical, corresponding to the five-tier classification scale discussed earlier in this chapter.

Below these summary counts, the dashboard included two dedicated sections: one for individually Scanned Files, and one for Scanned Folders, each with its own search bar and an "All Levels" filter dropdown, allowing users to quickly locate a specific file or narrow results down to a particular threat level rather than scrolling through an unfiltered list. If no files had been scanned yet, the dashboard displayed a friendly, encouraging message — "No Files Scanned Yet — Upload a file to start scanning for threats" — paired with a prominent Upload & Scan button, rather than simply showing an empty, uninformative screen. This small design choice mattered more than it might initially appear: an empty state with no guidance can leave a new user unsure of what to do next, while a clear call to action turns that same empty moment into an obvious next step.

How to Use

- After logging in, the dashboard opens automatically.
- Review the summary cards across the top of the page to monitor overall scanning statistics and current threat status.
- Use the Scan File or Upload & Scan button to begin a new scan.
- Once files have been scanned, they will appear in the Scanned Files and Scanned Folders sections, where they can be searched, filtered by threat level, and reviewed at any time.

Upload & Scan Page



The Upload & Scan Page was where registered users submitted files or folders for malware analysis, functioning as the account-based equivalent of the Guest Scan Page but with results tied to, and saved within, the user's personal account. This page was designed to keep the scanning process simple while still supporting the system's full range of underlying detection capabilities. Users could upload files either by clicking a dedicated upload button or by dragging files directly into a clearly marked drop area, described on-screen as supporting "Executables, Documents, Archives, Scripts, Images, Videos & Audio" — a deliberately broad, plain-language description intended to reassure users that virtually any file type they might need to check was supported, without requiring them to consult a separate technical list.

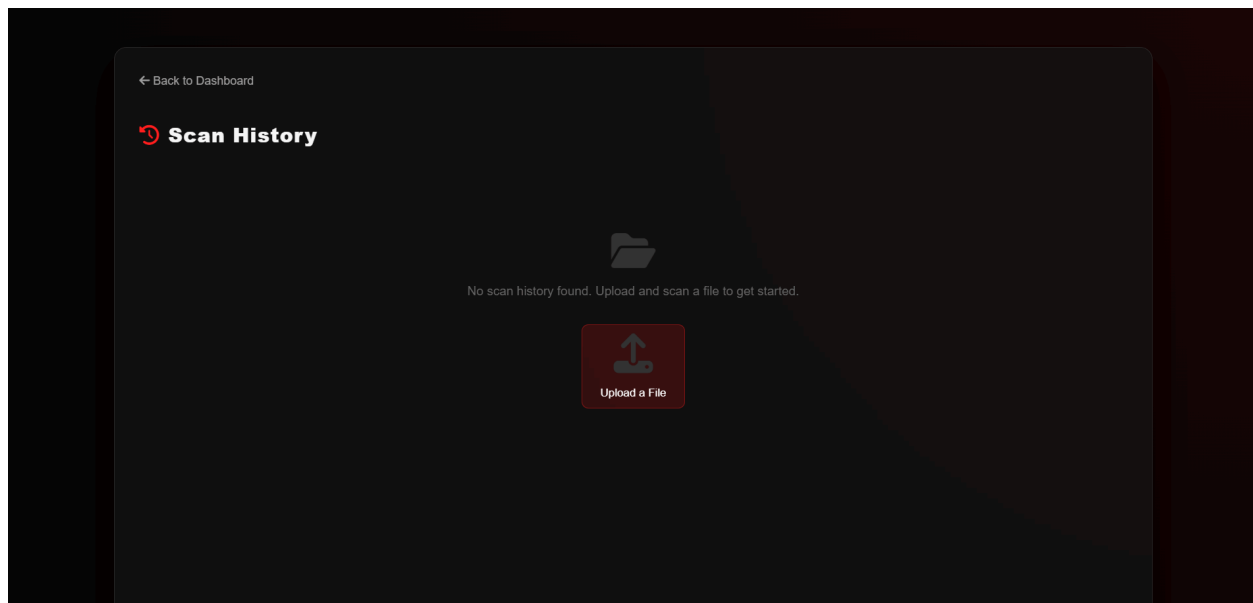
The interface supported scanning both individual files and entire folders through two clearly separated buttons — Select File(s) and Scan Entire Folder — making the page equally useful for a user checking a single suspicious download and for a user needing to verify a whole batch of files received at once, such as an entire folder of documents

forwarded from an external source. Once files had been uploaded, a single prominent button — Start Threat Scan — began the analysis process.

How to Use

- Choose whether to scan a single file or an entire folder.
- Upload the selected files using either the drag-and-drop area or the corresponding upload button.
- Click Start Threat Scan to begin the analysis.
- Wait for the scanning process to complete before reviewing the results displayed by the system.

Scan History



The Scan History page stored a record of every previously completed scan performed by a registered user, allowing them to revisit older results without needing to re-upload and re-scan the same file a second time. Beyond simple convenience, this page functioned as an ongoing record of a user's cybersecurity activity over time — useful

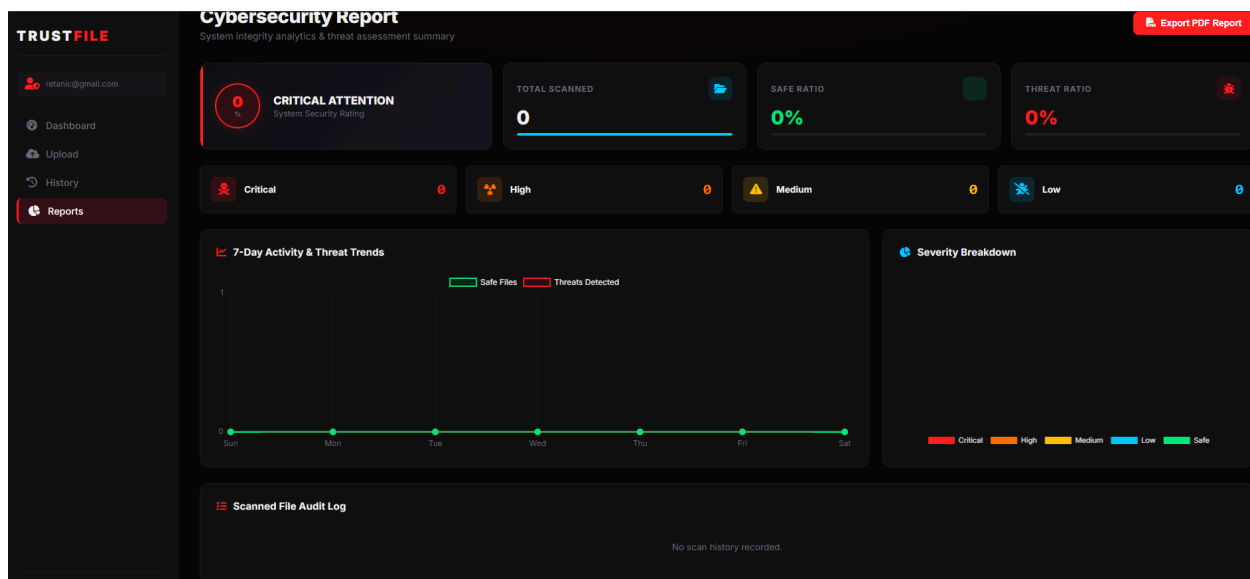
both for personal peace of mind and, in an organizational setting, as a form of documentation supporting the auditable record-keeping discussed in the Significance of the Study.

If a user had not yet scanned any files, the page displayed a clear, encouraging empty state — "No scan history found. Upload and scan a file to get started" — accompanied by an Upload a File button, following the same design philosophy used on the Dashboard's empty state: an empty page should still tell the user exactly what to do next, rather than leaving them uncertain.

How to Use

- Open the History page from the navigation menu to view previously completed scans.
- Browse the list of scanned files and review their individual results as needed.
- If no history exists yet, click Upload a File to perform a first scan and begin building a record.

Reports Page



The Reports Page provided a comprehensive statistical summary of a user's overall scanning activity and cybersecurity status, presented through summary cards, line graphs, and breakdown charts rather than a simple list of past results. This page was intended specifically for users who needed to understand broader patterns in their scanning activity — monitoring trends over time, identifying which types of threats appeared most frequently, and, where needed, generating a professional, exportable record of that activity.

The page's heading, "Cybersecurity Report — System integrity analytics & threat assessment summary," was followed by a prominent System Security Rating indicator and four summary metrics: Total Scanned, Safe Ratio, Threat Ratio, and a breakdown of threat counts by severity — Critical, High, Medium, and Low — corresponding to the study's five-tier classification scale as described earlier. Below these summary cards, a 7-Day Activity & Threat Trends line graph plotted Safe Files against Threats Detected across the past week, giving users an at-a-glance sense of whether their exposure to risky files was increasing, decreasing, or holding steady. Alongside this graph, a Severity Breakdown chart visually distributed detected threats across the five categories, and a Scanned File Audit Log provided a detailed, chronological record of individual scan events underneath.

The report included the following key sections:

- Total files scanned
- Safe-to-threat ratio
- Threat severity counts (Critical, High, Medium, and Low)
- A seven-day activity and threat trend graph
- A severity breakdown chart
- A scanned file audit log
- An Export PDF Report button for downloading the complete report

How to Use

- Navigate to the Reports page from the sidebar to view the overall cybersecurity summary.
 - Review the statistics and graphs to monitor scanning performance and detected threats over time.
 - Click Export PDF Report to generate and download a complete, shareable cybersecurity report — useful for documentation, presentations, or future reference.
-

Tools and Technologies Used

- **Frontend: HTML, CSS, JavaScript**
 - The frontend technology stack encompassed the client-side technologies responsible for building TrustFile's user interface and handling browser-based interactions. HTML (HyperText Markup Language) provided the structural foundation of every page, defining the headings, paragraphs, forms, buttons, and containers that organized content and functionality across the system — the landing page, the scan pages, the dashboard, and every other screen described in the System Preview relied on HTML as their underlying skeleton. CSS (Cascading Style Sheets) controlled the visual presentation layered on top of that structure, defining the colors, fonts, layout, spacing, and responsive behavior that gave TrustFile its consistent dark, red-accented cybersecurity theme across every screen, while also ensuring the interface adapted sensibly to different screen sizes. JavaScript supplied the dynamic functionality that made the interface feel responsive rather than static — handling interactive elements such as the drag-and-drop upload area, the password visibility toggles on the Login and Registration pages, asynchronous communication with the server during a scan, form validation, and

dynamic updates to the page's content without requiring a full page reload each time.

- Together, these three technologies formed the standard, well-established foundation of client-side web development, chosen specifically because they required no proprietary tooling, ran natively in any modern browser without additional installation, and gave the researchers direct, granular control over exactly how each interface element behaved and appeared.

- **Backend: Python Flask**

- The choice of backend technology carried real weight for this project, since it directly shaped development speed, runtime performance, and the system's overall technical capabilities. Python was selected as the core programming language, recognized broadly for its readability, its extensive standard and third-party library ecosystem, and its strong community support. Python's comparatively clear syntax reduced both development time and long-term maintenance overhead — a meaningful consideration given the academic timeframe within which TrustFile needed to be designed, built, and tested.
- Flask was selected as the specific web framework running on top of Python. Flask is deliberately described as a "microframework," meaning it provided only the essential building blocks of web development — routing, request handling, template rendering, and session management — without imposing a rigid, opinionated project structure the way larger frameworks often do. This minimalism gave the researchers the flexibility to structure TrustFile according to the system's specific requirements, rather than being constrained by a framework's own conventions. Flask's relative simplicity made it particularly well suited to an academic project and working prototype like TrustFile, where development speed and code

clarity mattered more than the kind of enterprise-scale feature set a larger, more heavily structured framework such as Django would have provided. For a system of TrustFile's scope, Flask offered what the researchers judged to be the right balance between capability and simplicity.

- **Database: MySQL**

The database layer stored TrustFile's persistent data — user accounts, scan histories, detection results, and system configuration information — and needed to do so reliably across every session, not just within a single scan. MySQL was selected as the system's database technology: an open-source relational database management system (RDBMS) that uses Structured Query Language (SQL) for defining, manipulating, and querying stored data. MySQL's widespread adoption reflects its performance, reliability, ease of use, and extensive documentation, and it supported the complex queries, transactions, and indexing that TrustFile's data needs required. Within TrustFile specifically, MySQL stored registered user information, records of every completed file scan, the detection results and classifications tied to each scan, and the configuration data underpinning the system's overall operation — forming the persistent backbone behind features such as Scan History and the Reports page described in the System Preview.

- **Development Tools: VS Code, Command Prompt, Pip**

- Beyond the core technology stack, several supporting tools were used throughout the development process itself. Visual Studio Code (VS Code), a free, open-source code editor developed by Microsoft, served as the primary environment in which the researchers wrote, edited, and reviewed the system's code. Its extension ecosystem provided syntax highlighting, code completion, integrated debugging support, and version control integration, making it a practical, widely adopted choice for a project of this scale.

- The Command Prompt (on Windows) provided the command-line interface through which the researchers executed development commands, ran scripts, managed project files, and interacted directly with other development tools throughout the build process. Pip, Python's standard package installer, was used to install and manage the third-party libraries and dependencies TrustFile relied on, drawing from the Python Package Index (PyPI) and handling version management and dependency resolution automatically. Together, these tools — an editor, a command-line interface, and a package manager — formed a complete, standard development environment for building, testing, and iterating on a Python-based web application throughout the project's Agile development cycle described in the System Development section.
- **NVIDIA NIM API (Llama 3.1-70b-instruct) — AI analysis**
 - TrustFile's explanatory capability was powered by the NVIDIA NIM API, using the Llama 3.1-70B-Instruct large language model as its underlying engine. This AI service was integrated specifically to handle the explanatory half of TrustFile's detection process — not the classification decision itself. Once the system's static analysis techniques, heuristic rules, and VirusTotal lookup had already determined a file's five-tier classification, that structured set of findings — which indicators were triggered, at what severity, and why — was passed to the NVIDIA NIM API, which transformed those raw technical findings into a clear, plain-language explanation intended for a non-technical reader.
 - This division of labor mattered for both technical and practical reasons. Keeping the actual classification decision inside deterministic, rule-based heuristic logic meant that TrustFile's threat verdicts remained consistent, auditable, and explainable in a strict technical sense — an important property for a security tool whose classifications, as discussed in the

Significance of the Study, might need to support organizational compliance or audit records. The AI layer, meanwhile, was free to focus entirely on what large language models are genuinely well suited for: taking a structured, technical set of findings and rendering it into natural, accessible language, without needing to introduce any uncertainty into the security-critical classification decision itself. The NVIDIA NIM platform delivered this explanation capability through GPU-accelerated inference, allowing explanations to be generated quickly enough to be displayed to the user immediately alongside their scan result, rather than after a noticeable delay.

- **VirusTotal API v3**

- The VirusTotal API v3 served as TrustFile's external threat-intelligence source, a cloud-based service that allowed the system to check submitted files against detection data collected from dozens of independent cybersecurity vendors and security engines simultaneously. Through its RESTful API endpoints, TrustFile could submit a file's cryptographic hash and automatically retrieve a comprehensive security report in return — covering malware detections, reputation scores, and threat classifications drawn from across VirusTotal's aggregated vendor base, consistent with the hash-based, privacy-preserving query approach described in the System Architecture section.
- Within TrustFile, the VirusTotal API functioned as one of the direct inputs feeding into the system's heuristic classification logic, complementing the system's own local static analysis techniques by providing external verification and cross-referencing beyond what local analysis alone could determine. A file's VirusTotal result — whether it matched a known-malicious hash already documented by other security vendors — carried real weight in the heuristic engine's eventual five-tier classification,

in the same way the other four static analysis indicators did, before that finished classification was ever passed along to the AI layer for explanation.

Data Gathering Techniques

The researchers relied on the following methods to gather the information that shaped TrustFile's design and eventual feature set:

- **Observation** - The researchers observed that many companies received large volumes of files every day, a pattern that left them more vulnerable to malware capable of harming their computers and networks. Given this overwhelming volume, these organizations often struggled to identify and separate which incoming files might actually cause harm. This is precisely where TrustFile was positioned to help — scanning files and explaining clearly why a given file was harmful and should not be opened. This observation was especially relevant for offices, private sector organizations, and government agencies, all of which constantly received and downloaded files as a routine part of their daily operations.

This observation of high-volume file reception revealed a genuine operational challenge that directly amplified malware risk. Organizations across sectors such as government, education, business services, and private enterprise routinely received hundreds or even thousands of files daily, arriving through email, web portals, file-sharing services, and removable media. Every one of these files represented a potential threat vector, yet the sheer volume made individual manual scrutiny practically impossible. Security personnel could not realistically examine every file by hand, which created a structural reliance on automated tools — tools that, as discussed in the Statement of the Problem, often missed sophisticated or previously unseen threats.

This volume-driven vulnerability extended beyond detection difficulty alone. It also created a real risk of alert fatigue, where security personnel, overwhelmed by a high number of warnings, gradually began to ignore or dismiss alerts altogether — a pattern that could cause genuine threats to slip through unnoticed among a backlog of false positives. Automated tools lacking contextual understanding compounded this problem further by flagging legitimate files as suspicious, consuming investigation time and creating friction between security teams and the rest of an organization trying to get ordinary work done.

TrustFile's design directly addressed this observed need by providing automated scanning capable of processing files quickly, classifying threats through the system's heuristic engine, and then explaining those findings clearly through its AI-generated explanation layer. The "why they are harmful" component of this observation addressed the underlying educational gap identified throughout this study, ensuring that users came to understand a given threat rather than simply accepting an automated verdict at face value. The accompanying "should never be opened" guidance, in turn, gave users clear, actionable direction — reducing ambiguity and supporting more consistent security practices across an organization, rather than leaving each individual user to interpret an ambiguous warning on their own.

For offices and organizations that constantly received files, this combination of automation and explanation was not merely convenient but essential. The sheer volume of incoming files made manual analysis impossible from the outset, while the diversity of file sources and formats increased exposure to a correspondingly wide range of threat vectors. TrustFile's comprehensive scanning of files of any size and virtually any extension was designed specifically to ensure that no potential threat escaped analysis simply because of its format or size.

- **Analysis of Existing Systems** - The researchers analyzed existing platforms and systems that offered file scanning, either online or through installed applications, in order to observe their features and identify their limitations. This analysis helped develop a clearer understanding that file scanning alone was not sufficient as a standalone feature — the system also needed to explain the problems associated with a flagged file, so that users could genuinely understand what made a file harmful rather than simply being told that it was. The researchers also examined how existing solutions implemented AI-assisted communication and heuristic-based detection, to better understand where TrustFile could meaningfully improve on current practice.

This analysis of online scanning platforms examined services such as VirusTotal, MetaDefender, Jotti, and similar tools offering web-based file analysis. These platforms generally offered comprehensive multi-engine scanning, extensive threat databases, and detailed technical reports — genuine strengths that TrustFile itself drew on directly, most notably through its own integration with the VirusTotal API. However, the researchers observed that these platforms tended to present their results in formats accessible primarily to security professionals — dense technical terminology, raw detection output, and minimal explanatory content that a non-technical user would likely find confusing or simply overwhelming. None of the platforms examined offered the kind of accessible, AI-generated explanation layer that TrustFile was built around.

Application-based scanning tools, including conventional antivirus software, endpoint protection platforms, and standalone file analysis utilities, were also evaluated as part of this analysis. These tools generally offered deeper system integration and a broader overall protection scope than a browser-based tool like TrustFile could provide, but they frequently came with complex interfaces, intrusive notifications, noticeable performance impacts on the host device, and — most relevant to this study — the same opaque, black-box detection process

discussed at length in the Statement of the Problem. Non-technical users were often alienated by exactly this opacity, left with a bare verdict and no real understanding of the reasoning behind it.

The key insight to emerge from this analysis was that detection alone is insufficient for genuinely effective security. A user who does not understand why a file is dangerous cannot apply that understanding to similar files encountered in the future, cannot meaningfully verify whether a given detection was accurate, and may come to distrust — or even actively circumvent — a security tool that feels arbitrary or opaque in its judgments. The educational component identified through this analysis — explaining the specific problems associated with a flagged file — was what transformed security scanning from a black-box service into a genuine learning opportunity, building a user's own capability over time rather than leaving them permanently dependent on the tool's verdict. TrustFile's AI-generated explanation layer was built specifically to close this gap: taking the heuristic engine's finished classification and translating it into contextual, understandable guidance a non-technical user could actually act on.

This understanding directly shaped TrustFile's design priorities going forward, ensuring that explanatory content, educational context, and clear communication of both the heuristic engine's classification and the reasoning behind it were treated as integral features from the outset, rather than as afterthoughts added late in development. Every scan result was designed to include an explanation of what had been found, why it mattered, how the heuristic engine's combination of indicators had contributed to the classification, and what the user could do to protect themselves going forward — directly addressing the educational gap this analysis of existing systems had identified.

Testing Procedures

In order to make TrustFile work properly as intended, the researchers conducted the following types of testing:

- **Unit Testing** - This phase focused on the functionality of individual components in isolation — the file hashing module, the entropy analysis module, the script scanner, the file type verifier, the heuristic classification engine, and the AI explanation layer — using both valid and malicious test samples to confirm accurate performance at the component level before any of these pieces were tested together.

File hashing module testing validated that hash computation produced correct, consistent results for known inputs. This included verifying that identical files produced identical hashes, that different files produced different hashes, that the hashing algorithm executed correctly across a range of file sizes, and that hash comparison against the VirusTotal database functioned properly. Test samples included files with already-known hashes, used specifically to verify the accuracy of the comparison logic itself.

Entropy analysis module testing confirmed that entropy calculation accurately measured randomness within a file, that baseline profiles correctly represented the expected entropy range for normal file types, and that anomaly detection correctly flagged files that deviated meaningfully from those baselines. Test files included highly compressed files, encrypted files, plain text files, and structured binary files — deliberately chosen to have known, contrasting entropy characteristics — in order to validate both measurement accuracy and the calibration of the thresholds used to flag deviation.

Script scanner module testing verified detection of embedded executable code across different file formats. Test files included PDFs containing embedded JavaScript, Office documents containing macros, archives containing hidden scripts, and files with encoded or otherwise obfuscated script content. Both positive cases (files genuinely containing a script) and negative cases (ordinary files without any embedded script) were tested, in order to validate detection accuracy and to check for an acceptable false-positive rate.

File type verifier module testing confirmed that magic number detection, internal structure validation, and extension comparison together correctly identified a file's actual type. Test files included correctly labeled files, files with deliberately spoofed extensions, files with truncated or corrupted headers, and files constructed to resemble more than one format at once — validating type detection across a range of deliberately difficult edge cases, not just straightforward ones.

Heuristic classification engine testing verified that the rule-based logic responsible for the system's five-tier classification correctly weighed the combined results of all five static analysis techniques, together with the VirusTotal lookup result, to produce an appropriate BENIGN-to-MALICIOUS classification. This testing used files exhibiting a single isolated indicator, files exhibiting multiple simultaneous indicators, files with conflicting or ambiguous indicators, and files with no indicators at all, in order to verify that the classification boundaries and the relative weighting assigned to each indicator produced results that matched the researchers' expectations across this full range of cases.

AI explanation layer testing, conducted separately from the heuristic engine's classification testing, verified that the NVIDIA NIM API and the Llama 3.1-70B-Instruct model correctly translated an already-decided classification and its supporting indicators into accurate, understandable plain-language output.

This testing checked that the generated explanation correctly referenced the actual indicators the heuristic engine had flagged for a given file, that the explanation's tone and severity matched the assigned threat level, and that no inaccurate or misleading technical claims were introduced during the translation from raw indicator data into natural language.

The consistent use of both valid and malicious test samples across every module ensured comprehensive validation across the full spectrum of files the finished system was expected to encounter. Valid samples tested that the system did not falsely flag legitimate files, while malicious samples tested that genuine threats were reliably detected. This dual approach allowed the researchers to separately quantify false positive and false negative rates at the component level — essential, foundational metrics before the same evaluation was repeated at the level of the whole integrated system.

- **System Testing** - In this phase, the researchers focused on how the fully assembled system scanned files end-to-end — whether scanning completed properly, whether the correct threat level was ultimately assigned, and how the system performed as a whole, rather than testing any single component in isolation.

Scanning workflow testing validated the complete process from initial file upload through to final result presentation. This included verifying that uploaded files were correctly received, temporarily stored, and processed; that scanning completed successfully across a range of different file types and sizes; that the heuristic engine's classification logic and the VirusTotal lookup executed correctly in sequence; that the AI explanation layer correctly received and processed the finished classification; and that the final result was presented to the user in a timely and comprehensible way. This workflow-level testing surfaced integration issues that unit testing, by its isolated nature, could not have detected — data format mismatches between components, timing issues in asynchronous

operations, and resource conflicts arising when multiple parts of the system ran concurrently.

Threat identification accuracy testing evaluated whether the fully integrated system correctly identified and classified threats across the full range of test files used throughout this study. This involved scanning files with already-known characteristics and verifying that the classification the heuristic engine ultimately assigned matched the researchers' expected outcome for that file. This testing quantified overall system performance through metrics including detection rate, false positive rate, and false negative rate, alongside the general distribution of classifications produced across the full test set.

Heuristic engine and AI explanation layer integration testing verified that the two components correctly handed off responsibility to one another as designed — that the heuristic engine's finished classification was passed accurately and completely to the AI explanation layer, and that the resulting explanation always remained faithful to that classification rather than introducing any inconsistency between the two. This testing specifically checked for cases where an explanation might understate or overstate the severity implied by its corresponding classification, ensuring the two components stayed in agreement by design rather than by coincidence.

Overall performance assessment examined system behavior under realistic operating conditions. This included measuring response times for a complete file scan, evaluating throughput under concurrent user load, assessing resource consumption in terms of CPU, memory, disk, and network usage, and verifying system stability during extended periods of continuous operation. This performance testing existed to confirm that the system met reasonable usability expectations and could handle anticipated workloads without a meaningful drop in responsiveness — particularly important given the resource ceilings imposed by the PythonAnywhere hosting tier discussed in the Scope and Delimitation.

Taken together, "working properly as intended" was understood throughout this phase to encompass functional correctness, adequate performance, reliability under load, and genuine usability for the system's intended non-technical users. System testing was designed to validate each of these dimensions in turn, providing the empirical evidence needed to judge whether the system was genuinely ready for the evaluation phase that followed.

- **User Acceptance Testing** - This phase involved real users or staff uploading files and evaluating the system's user interface, the clarity of its reports, and their overall experience — gathering direct feedback on whether the system performed effectively and genuinely helped educate users about file security, rather than relying solely on the researchers' own judgment of the finished product.

Involving real users at this stage mattered because it grounded the evaluation in actual user perspectives rather than developer assumptions. Users brought a range of technical backgrounds, work contexts, and expectations that could reveal usability issues the researchers themselves, having built the system, might no longer notice. Their feedback surfaced confusing interface elements, missing features, and workflow mismatches that purely technical testing was never designed to catch in the first place.

File upload validation by real users tested whether the upload process felt intuitive, efficient, and reliable when used with users' own files, across different browsers, devices, and network conditions. Because the system was designed to support files of any size and virtually any extension, this stage of testing specifically validated that this universal support held up in practice, not just under the researchers' own controlled test conditions.

User interface validation assessed whether the interface design genuinely supported the tasks users needed to complete — evaluating navigation clarity, how easily needed information could be found, how discoverable key actions were, and the overall visual hierarchy of each screen. Users were asked to attempt typical tasks, such as scanning a file, interpreting its result, or locating help, while researchers observed where users hesitated, grew confused, or succeeded without difficulty.

Report clarity evaluation examined whether scan results were genuinely comprehensible and actionable for non-technical users specifically. Users were asked to interpret results without direct assistance from the researchers, which revealed whether the five-tier classification communicated risk clearly on its own, whether the AI-generated explanations were genuinely understandable without a technical background, and whether the recommended next steps felt realistic and feasible to follow. Feedback gathered at this stage fed directly back into refinements made to how results were presented and explained.

AI explanation quality evaluation specifically assessed whether users understood the reasoning behind a given classification once they had read the AI-generated explanation. Users were asked to describe, in their own words, why a particular file had received the classification it did, which revealed whether the explanation layer was successfully communicating the heuristic engine's underlying reasoning in a way that actually stuck, rather than being read and immediately forgotten.

Overall experience assessment captured broader impressions that went beyond any single feature — whether the system felt trustworthy, professional, and genuinely helpful, and whether users came away feeling it was worth using again. These more subjective impressions carried real weight, since they were

what would ultimately determine whether a system like TrustFile saw sustained adoption rather than a single, one-time use.

Feedback on educational effectiveness examined whether the system actually improved users' security awareness, as intended throughout this study. Users were asked whether the explanations they received had increased their understanding, whether they felt better equipped to recognize a suspicious file on their own in the future, and whether their general approach to handling files had shifted at all as a result of using the system. This feedback served as direct validation of TrustFile's educational component, discussed at length in the Significance of the Study, and helped identify specific areas where the explanation layer could still be improved.

Evaluation Criteria

TrustFile was evaluated according to the following criteria:

- **Functionality** - This criterion assessed whether the system performed all of its main features properly — file scanning, file analysis, threat classification through the heuristic engine, and clear AI-generated explanation of why a given file was flagged as harmful. All parts of the system were expected to function as intended, without errors.

File scanning functionality was evaluated by verifying that the system successfully received, processed, and analyzed uploaded files across the full range of supported types and sizes. Tests included normal files, unusually large files, very small files, files with special characters in their filenames, and files across a wide range of formats — including ZIP, TXT, PDF, DOCX, XLSX, PPTX, JPG, PNG, GIF, MP3, MP4, EXE, DLL, RAR, and others. Success was defined

by successful upload completion, correct scan initiation, full analysis execution, and accurate result generation at the end of the process.

File analysis functionality was evaluated by confirming that each individual analysis method — hashing, entropy analysis, script/pattern detection, and file type verification — produced correct results for test files with already-known characteristics. This evaluation compared the system's actual output against the expected output for a validated test dataset, quantifying accuracy and identifying any discrepancies that emerged.

Threat classification functionality was evaluated by verifying that the heuristic engine's rule-based logic correctly assigned a threat level based on the combined results of the five static analysis techniques and the VirusTotal lookup. Tests included files with known characteristics expected to trigger a specific classification, files with deliberately ambiguous characteristics intended to test the boundaries between adjacent classification tiers, and files with no suspicious characteristics at all, which were expected to receive a BENIGN classification.

Explanation functionality was evaluated separately, assessing whether the AI-generated explanations accurately described the indicators the heuristic engine had actually detected, correctly explained the relevant technical concepts in accessible language, and provided guidance a user could realistically act on. This evaluation involved expert review of explanation accuracy — confirming that the AI layer never introduced a claim inconsistent with the heuristic engine's underlying findings — alongside user comprehension testing to verify that the explanations were genuinely accessible to a non-technical reader.

The "without errors" requirement established that functional correctness also included robust handling of unexpected inputs, edge cases, and error conditions. The system was expected not to crash, hang, or produce an incorrect result when encountering a malformed file, a network interruption, a temporarily

unavailable external service, or any other exceptional condition outside normal operation.

- **Usability** - This criterion measured whether the system was genuinely easy to use, even for users without any technical background. Users were expected to be able to upload and scan files easily, with results clear and understandable enough that no separate instructions were needed to interpret them.

Ease of use for non-technical users was evaluated through task-based testing, in which representative users attempted typical workflows without any assistance from the researchers. Observers measured task completion rates, time to completion, error rates, and how often users needed to ask for help. Users also provided their own subjective ratings of difficulty and confidence. Low completion rates, unusually long completion times, frequent errors, or low user-reported confidence were all treated as clear indicators of an underlying usability problem.

Upload and scan ease was evaluated by examining the number of steps required to complete a scan, the clarity of on-screen instructions, how intuitive the interface elements felt, and the quality of feedback provided throughout the process. An ideal workflow minimized the number of required actions, relied on familiar interaction patterns, provided clear progress indication during scanning, and confirmed successful completion unambiguously. Because the system supported files of any size and virtually any extension, usability testing specifically validated that this broad compatibility never came at the cost of a more complicated user experience.

Result clarity and understandability was evaluated by testing whether users could correctly interpret a scan result without consulting any reference material. Users were asked to explain, in their own words, what they understood from a given result — identifying the threat level, describing what had been found, and stating what action they believed they should take next. Any misinterpretation,

confusion, or inability to extract the key information from a result was treated as a presentation problem requiring correction. The AI-generated explanation layer was evaluated particularly closely against this standard, given that communicating complex technical findings in accessible language was its entire purpose within the system.

The "without needing instructions" criterion reflected a deliberately high usability standard: the system needed to be intuitive enough that a first-time user could accomplish their task successfully without reading a manual, watching a tutorial, or receiving any training beforehand — a standard that reflected the practical reality that most users never consult documentation and expect to understand a tool immediately upon using it.

- **Accuracy of Detection** - This criterion evaluated the system's accuracy and reliability in identifying suspicious and potentially malicious files, while minimizing incorrect results, through the heuristic engine's rule-based classification logic.

Detection accuracy was measured using three standard metrics: precision (the proportion of files flagged as a threat that were genuinely threats), recall (the proportion of actual threats the system successfully identified), and the F1-score (the harmonic mean of precision and recall, balancing the two). High precision indicated few false positives; high recall indicated few false negatives; and the F1-score captured the overall balance struck between these two competing objectives.

Because the heuristic engine's classification decision drew on five distinct static analysis indicators together with the VirusTotal lookup, evaluation also examined the accuracy contribution of each individual indicator, alongside the combined classification produced once all indicators were weighed together — helping the researchers understand which specific checks contributed most to correct classifications, and which occasionally introduced noise into the final result. The

AI explanation layer, evaluated separately as described under Functionality above, was not assessed against these detection-accuracy metrics at all, since it played no role in the classification decision itself; its evaluation instead focused entirely on the accuracy and clarity of the explanation it generated for an already-determined classification.

Identification of suspicious and potentially malicious files was evaluated using test datasets with already-known labels. The system scanned these labeled files, and its resulting classifications were compared directly against the known ground truth. Accuracy was calculated separately across different file types, threat categories, and obfuscation techniques, in order to ensure the evaluation was comprehensive rather than reflecting performance on only one narrow category of test file.

Minimizing false results encompassed both false positives (benign files incorrectly flagged as threats) and false negatives (malicious files incorrectly classified as safe) — both discussed in greater technical detail in the Definition of Terms. False positives caused unnecessary concern, operational disruption, and, in more serious cases, a growing temptation among users to bypass security warnings altogether. False negatives, meanwhile, allowed genuine threats to slip through undetected, directly exposing a system to compromise. Where exactly to strike the balance between the two ultimately depended on organizational risk tolerance, with higher-security environments generally willing to accept a somewhat higher false-positive rate in exchange for minimizing the more dangerous false-negative case.

Reliability evaluation, closely related to accuracy, assessed the consistency of the heuristic engine's classifications across repeated scans of the same file, across different system configurations, and under varying operating conditions. A reliable system was expected to produce consistent results for identical inputs, maintain its accuracy under load, and degrade gracefully — rather than failing

outright — if a supporting component, such as the VirusTotal API, became temporarily unavailable.

- **Security** - This criterion ensured that TrustFile itself could safely handle files during the scanning process. Files were only ever read and analyzed, never executed or opened, specifically to prevent any possibility of infection if a submitted file turned out to be genuinely malicious. The system was also expected to ensure that the scanning process itself never caused harm to a user's own device.

Secure handling of files during scanning meant ensuring that the scanning process itself could not be exploited to compromise the underlying system or exfiltrate data. This included sandboxing the analysis process to limit its access to broader system resources, validating file contents carefully before processing to prevent exploits targeting the system's own file parsers, restricting network access during analysis to prevent any possibility of data exfiltration, and sanitizing outputs to guard against injection-style attacks.

Read-only analysis without execution was treated as a critical security principle underlying the entire system, consistent with the static analysis approach discussed in the Scope and Delimitation. By examining a file's contents and structure without ever executing it, TrustFile avoided the risk of triggering a malicious payload that could infect the scanning infrastructure itself. This principle was implemented through careful parser design — extracting and analyzing a file's structure without ever invoking any executable content it might contain — combined with sandboxed processing environments that isolated any accidental execution attempt, and ongoing monitoring designed to detect and immediately prevent any unauthorized execution. The heuristic engine and the AI explanation layer both operated exclusively on already-extracted file features — entropy values, header information, structural patterns, and similar

characteristics — rather than on executable file content itself, preserving this read-only principle throughout the entire pipeline.

Protection of user devices extended beyond the scanning process itself to the system's broader web interface. The interface was expected to guard against cross-site scripting (XSS) attacks that could otherwise compromise a user's browser. File uploads were protected against path traversal and denial-of-service attempts. Data transmitted between the user's browser and the server was expected to be encrypted, preventing interception, and user credentials were expected to be stored and transmitted securely at every stage.

More broadly, the system was expected not to harm a user's device through excessive resource consumption, unauthorized modification, or the introduction of new vulnerabilities — meaning resource use had to be carefully managed to prevent anything resembling a denial-of-service condition on the user's end, and the system itself needed to be designed so that it could never become a vector for the very malware distribution it was meant to prevent.

- **Efficiency** - This criterion assessed whether file scanning completed within a reasonable amount of time, processing and displaying results quickly enough to ensure a smooth overall user experience — even for larger files and a diverse range of formats.

What counted as a "reasonable" scanning time depended somewhat on context — file size, analysis complexity, and general user expectations all played a role. For typical documents, users generally expected a result within seconds; for larger files or more involved analysis, a slightly longer wait was considered acceptable, provided clear progress feedback was shown throughout. Because the system was designed to scan files of any size, performance optimization work specifically focused on ensuring that larger files did not introduce excessive delay relative to smaller ones. Evaluation established rough benchmarks based

on general user expectations, then measured the system's actual performance against those benchmarks.

Achieving quick processing involved optimizing the underlying scanning algorithms, leveraging caching where appropriate, and minimizing unnecessary computation throughout the pipeline. The AI explanation layer, in particular, was implemented to generate its output efficiently, so that waiting for an explanation never became the bottleneck in an otherwise fast scan. Performance profiling was used to identify specific bottlenecks in the pipeline, with targeted optimization applied wherever a slowdown was found. The overall goal throughout this process was sub-second responsiveness for typical files, with graceful, rather than abrupt, degradation for larger or more complex content.

Displaying results without delay required efficient result formatting, transmission, and rendering on the client side. Results were expected to be available to the user immediately upon completion of analysis, with the interface itself optimized for rapid comprehension rather than simply rapid loading. Any noticeable delay between the completion of analysis and the actual display of results risked frustrating users and undermining confidence in the system's overall responsiveness. This broader sense of a "fast and smooth" experience extended beyond raw scanning speed alone to the system's general responsiveness — interface interactions were expected to feel immediate, transitions fluid, and feedback timely throughout every part of the experience, not just during the scan itself.

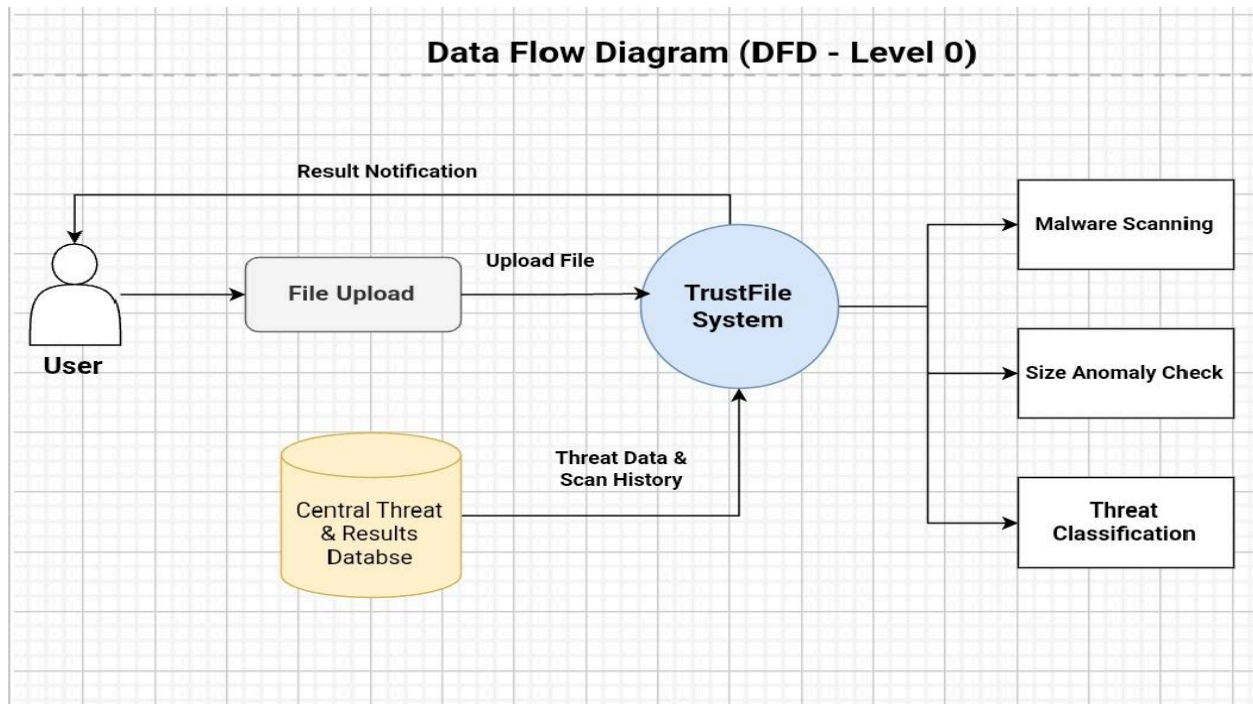
For offices and organizations that constantly received files, efficiency carried genuine practical weight rather than being a merely cosmetic concern. Users processing a high volume of incoming documents could not reasonably afford extended waiting periods for each individual scan to complete. TrustFile's optimization for rapid scanning across files of any size and virtually any extension

was intended specifically to ensure that meaningful security never came at the cost of everyday productivity.

SUPPORTING DIAGRAMS

Data Flow Diagram (DFD – Level 0)

The Data Flow Diagram at Level 0 provides a high-level, contextual overview of the Trustfile system, illustrating how data moves between external entities, system processes, data stores, and external systems. This diagram serves as a foundational visualization that helps stakeholders understand the system's scope, boundaries, and primary data exchanges without delving into internal process details.



External Entity: User

The User represents the primary external actor who interacts with the Trustfile system. In the diagram, the User initiates the file scanning workflow by submitting files for analysis and subsequently receives scan results through result notifications. The User entity is depicted as a stick figure, following standard DFD notation for human actors. This entity encompasses all types of users who access the system—employees in government offices, staff in private organizations, corporate users, students, or any individual who needs to verify file safety before opening. The User's interaction with the system is intentionally simplified in this high-level view, with detailed user types and roles elaborated in lower-level DFDs or use case diagrams.

For Trustfile's target clients—offices, private sector organizations, government agencies, and businesses that constantly receive and download files—the User entity represents the diverse workforce that requires accessible, efficient file security. These users typically lack specialized technical knowledge but face sustained exposure to file-based threats through their daily work activities.

Process: File Upload

The File Upload process serves as the system's entry point, receiving files from users and preparing them for analysis. This process performs critical initial functions including file reception through the web interface, validation of file types and sizes to ensure scannability, temporary secure storage of uploaded files during analysis, and generation of file metadata including timestamps, original filenames, and file hashes. The File Upload process acts as a gatekeeper, ensuring that only appropriate files proceed to analysis while rejecting or flagging files that exceed practical limits, are in unsupported formats, or exhibit suspicious upload patterns. The system supports files of any size and any extension, and the File Upload process accommodates this universal capability through flexible validation and handling. The data flow labeled "Upload File" carries the

file content and associated metadata from the File Upload process to the central TrustFile System process.

Process: TrustFile System

The TrustFile System represents the core processing engine that orchestrates all file analysis activities. As the central hub of the diagram, this process receives uploaded files, coordinates multiple analysis subsystems, manages data persistence, and generates comprehensive scan results. The system performs several critical functions: it dispatches files to various scanning modules (Malware Scanning, Size Anomaly Check, Threat Classification), aggregates results from these modules into unified threat assessments, queries the Central Threat & Results Database for historical data and threat intelligence, and generates result notifications for users. The TrustFile System process embodies the hybrid detection architecture described in the methodology, combining local static analysis, AI analysis, heuristic analysis, and external intelligence integration.

External Systems/Processes: Malware Scanning, Size Anomaly Check, Threat Classification

These three represent distinct analytical subsystems that perform specialized detection functions:

- **Malware Scanning:** This subsystem executes the core malware detection algorithms, including file hashing for known threat identification, entropy analysis for obfuscation detection, script scanning for embedded executable code identification, heuristic analysis for suspicious pattern detection, and file type verification for spoofing detection. It represents the local static analysis capabilities of the system, augmented by AI analysis and heuristic rules.

- **Size Anomaly Check:** This subsystem evaluates files against expected size ranges for their stated formats, flagging files that deviate significantly from normal parameters. Oversized files may indicate appended malicious payloads, while undersized files may suggest stub loaders that download additional components. This check serves as a preliminary filter that identifies suspicious files warranting deeper analysis.
- **Threat Classification:** This subsystem applies the supervised AI analysisclassification model and heuristic analysis rules to combine multiple indicator results into overall threat assessments. It assigns files to threat levels (Low, Medium, High, Critical) based on detected indicators, AI analysis, heuristic analysis findings, calibrated thresholds, and classification rules. This subsystem embodies the intelligent classification component that enables nuanced threat assessment beyond simple binary detection.

Data Store: Central Threat & Results Database

The Central Threat & Results Database serves as the system's persistent storage repository, maintaining all data required for operation and analysis. This data store contains multiple categories of information: threat intelligence data including known malware hashes, signatures, and reputation scores; scan history records documenting all previous file analyses, their results, and associated metadata; AI analysis model parameters and training data; heuristic analysis rules and update histories; user account information for authentication and authorization; system configuration parameters including classification thresholds and enabled detection modules; and statistical data for reporting and performance monitoring. The bidirectional data flow labeled "Threat Data & Scan History" indicates that the TrustFile System both reads from and writes to this database—querying historical data during analysis and persisting new results for future reference.

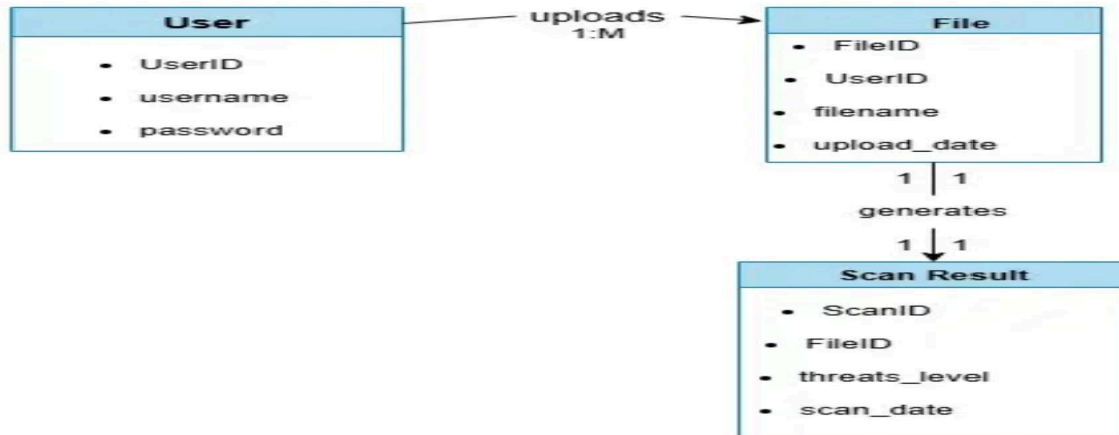
Data Flow: Result Notification

The Result Notification data flow carries scan outcomes from the TrustFile System back to the User, completing the analysis workflow. This flow contains comprehensive result information including the assigned threat level, specific indicators detected, AI analysis confidence, heuristic analysis findings, explanatory content describing why the file received its classification, recommended user actions, and educational material about the detected threat type. The notification mechanism ensures that users receive clear, actionable information that enables informed decision-making about file access. The intelligent explanation component generates customized descriptions that accurately reflect the specific combination of indicators detected in each file.

Entity Relationship Diagram (ERD)

The Entity Relationship Diagram illustrates the data structures and relationships within the Trustfile system's database, providing the foundation for database design, data integrity enforcement, and application development. This diagram uses standard ERD notation with entities represented as rectangles, attributes listed within entity boxes, and relationships depicted as lines with cardinality indicators.

Entity Relationship Diagram (ERD)



Entity: User

The User entity represents all individuals who interact with the Trustfile system. Its attributes include:

- **UserID:** The primary key that uniquely identifies each user account. This attribute serves as the reference for all user-related operations and relationships.
- **username:** The display name or login identifier chosen by the user for account access.
- **password:** The hashed credential used for authentication, stored securely using cryptographic hashing algorithms.

The User entity captures the minimal information required for system access control. Future enhancements might expand this entity to include additional attributes such as email addresses, role assignments (administrator, standard user), department

affiliations, account creation dates, last login timestamps, and user knowledge levels for adaptive explanation generation.

Entity: File

The File entity represents uploaded files submitted for security scanning. Its attributes include:

- FileID: The primary key that uniquely identifies each uploaded file record.
- UserID: A foreign key that establishes the relationship with the User entity, tracking which user uploaded each file.
- filename: The original name of the uploaded file as provided by the user.
- upload_date: The timestamp recording when the file was submitted to the system.

The File entity captures essential metadata about uploaded files while the actual file content is stored separately in the file system. This separation allows efficient database operations while maintaining access to file contents for analysis. Additional attributes that might be added include file size, file type/extension, file hash values, storage path, and deletion status. The system supports files of any size and any extension, and the File entity accommodates this universal capability through flexible attribute definitions.

Entity: Scan Result

The Scan Result entity represents the outcomes of file analysis operations. Its attributes include:

- ScanID: The primary key that uniquely identifies each scan operation.

- **FileID:** A foreign key that links the scan result to the specific file that was analyzed.
- **threats_level:** The classification outcome assigned to the file, using the four-tier system (Low, Medium, High, Critical).
- **scan_date:** The timestamp recording when the analysis was completed.

The Scan Result entity captures the core output of the Trustfile system's detection capabilities. Additional attributes that would enhance this entity include specific indicators detected (hash match, entropy anomaly, script presence, type mismatch, heuristic rule triggers, AI analysis), detailed explanation text, recommended actions, and references to external threat intelligence sources. The entity might also include AI analysis probabilities and heuristic analysis scores to support detailed result presentation.

Relationships:

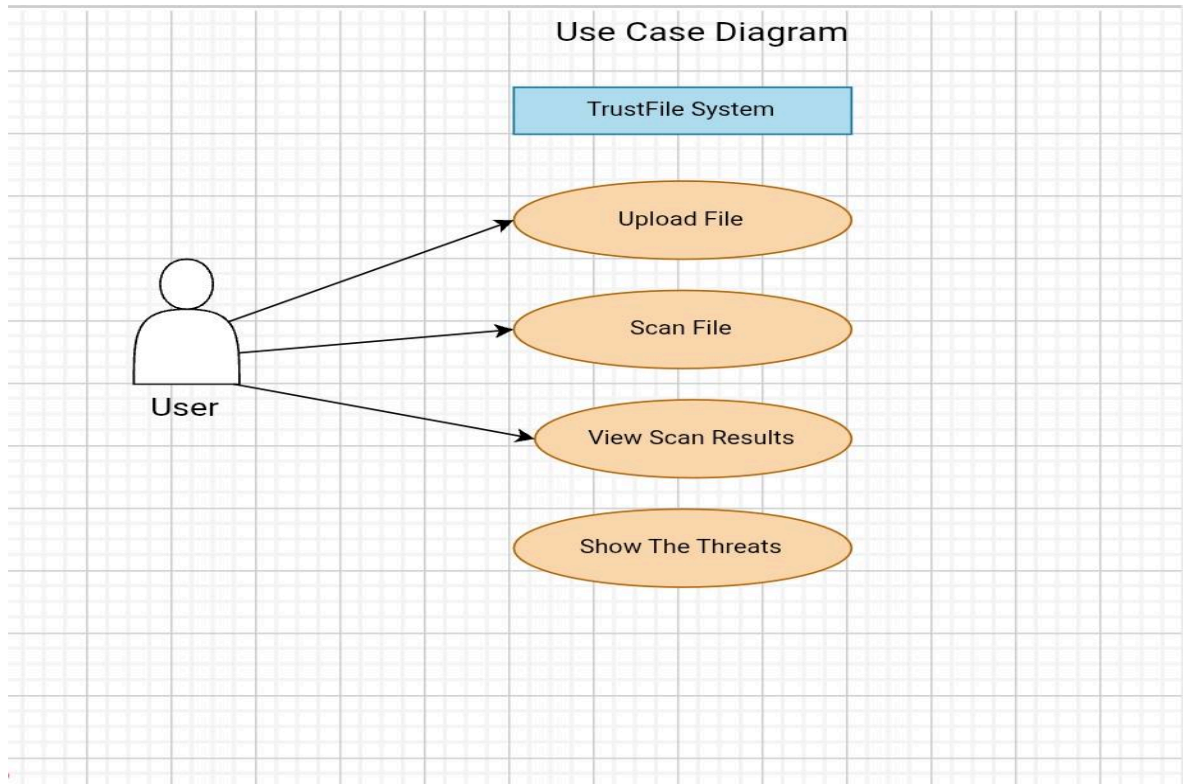
- **User uploads File (1:M):** This one-to-many relationship indicates that each user may upload multiple files over time, while each file is associated with exactly one user. This relationship enables tracking of user activity, generation of usage reports, and attribution of scan results to specific individuals. The cardinality "1:M" (one-to-many) reflects the typical usage pattern where users repeatedly access the system to scan multiple files.
- **File generates Scan Result (1:1):** This one-to-one relationship indicates that each file generates exactly one scan result, and each scan result corresponds to exactly one file. This relationship ensures data integrity by preventing orphaned scan results or files without analysis outcomes. The strict one-to-one cardinality reflects the current system design where each file receives a single

comprehensive scan. Future enhancements might modify this to one-to-many if multiple scan types or re-scanning capabilities are implemented.

The ERD design demonstrates database normalization principles, minimizing data redundancy while ensuring referential integrity. The relationships enforce business rules—users must exist before their files can be recorded, and files must exist before their scan results can be stored. Foreign key constraints prevent inconsistent data states and support cascade operations for data management.

Use Case Diagram

The Use Case Diagram illustrates the functional requirements of the Trustfile system from a user perspective, showing what users can accomplish with the system and how system functionality is organized around user goals. This diagram uses standard UML notation with actors represented as stick figures and use cases as ovals, connected by association lines.



Actor: User

The User actor represents the primary human actor who interacts with the Trustfile system. This actor encompasses all end users—government employees, private office staff, corporate workers, students, or any individual who needs to assess file safety. The User actor is distinguished from potential secondary actors such as System Administrator or External API Service, which might appear in extended use case diagrams. The User's goals are focused on file security assessment and understanding threat information, reflecting the system's core mission of pre-access file protection.

For Trustfile's target clients—offices, private sector organizations, government agencies, and businesses that constantly receive and download files—the User actor represents the diverse workforce that requires accessible, efficient security tools. These users typically handle multiple files daily through email, cloud storage, and file sharing platforms, creating sustained need for quick, reliable security assessment.

Use Case: Upload File

The Upload File use case represents the functionality that enables users to submit files to the Trustfile system for security analysis. This use case includes several sub-functions: file selection through the web interface using browse dialogs or drag-and-drop mechanisms, file validation to ensure submitted files meet practical limits and format requirements, secure transmission of file data to the server, temporary storage allocation for analysis, and initiation of the scanning workflow. The system supports files of any size and any extension, and the Upload File use case accommodates this universal capability. The Upload File use case serves as the entry point for all system functionality—without successful file upload, no other use cases can be executed. Pre-conditions include user authentication and valid file selection; post-conditions include file storage and scanning initiation.

Use Case: Scan File

The Scan File use case represents the core analytical functionality that examines uploaded files for potential threats. This use case encompasses the complete detection workflow: file preprocessing including format identification and header extraction, execution of multiple analysis modules (hash comparison, entropy calculation, heuristic rule evaluation, script detection, type verification), AI analysis for threat level assignment, external API queries for threat intelligence augmentation, and result compilation. The Scan File use case is typically initiated automatically following successful file upload, though it may also be triggered manually for re-analysis. This use case involves complex internal processing that is invisible to users but essential for accurate threat detection.

Use Case: View Scan Results

The View Scan Results use case represents the functionality that presents analysis outcomes to users in accessible, actionable formats. This use case includes: retrieval of scan result data from the database, formatting of results for web presentation, generation of visual indicators (color coding, icons, progress bars) for threat levels, compilation of explanatory content describing detected indicators, and presentation of recommended user actions. The View Scan Results use case transforms technical detection data into user-comprehensible information, bridging the gap between complex analysis and user understanding. This use case may be executed immediately after scanning completes or accessed later through scan history.

Use Case: Show The Threats

The Show The Threats use case represents the educational functionality that explains detected threats in detail, helping users understand why files are dangerous and how to protect themselves. This use case extends the basic result viewing by providing: detailed descriptions of specific malware types or attack vectors detected, explanations of technical indicators in plain language, contextual information about similar threats and their impacts, guidance on safe file handling practices, and references to additional security resources. The Show The Threats use case embodies Trustfile's educational mission, transforming each security event into a learning opportunity that builds long-term user awareness and capability.

The intelligent explanation system operates through this use case, analyzing specific scan results and generating customized explanations that accurately reflect each file's threat profile. Like an AI assistant, the system examines the particular combination of indicators detected AI analysis results, heuristic analysis findings, entropy values, hash matches, script detections—and constructs explanations that describe why the file was flagged in clear, understandable language.

Relationships and System Boundary:

The rectangular boundary labeled "TrustFile System" delineates the scope of the system, separating internal functionality from external actors. All use cases reside within this boundary, indicating that they represent system functionality rather than external processes. The association lines connecting User to each use case indicate participation relationships—the User actor initiates or participates in each use case.

The diagram shows direct associations between User and Upload File, Scan File, and View Scan Results, indicating that users explicitly initiate these functions. The Show The Threats use case appears without a direct user association line in the diagram, suggesting that it may be accessed as an extension of View Scan Results rather than as an independently initiated function. This relationship could be elaborated with <<extend><include>

Use Case Interactions:

The typical workflow sequence flows from Upload File through Scan File to View Scan Results, with Show The Threats available as an extension for users seeking deeper understanding. Alternative flows include: users viewing historical scan results without uploading new files, users re-scanning previously uploaded files, and users accessing Show The Threats directly from scan history. Exception flows include: upload validation failures, scanning errors, and cases where files cannot be analyzed.

The Use Case Diagram provides a user-centered view of system functionality that complements the process-oriented DFD and data-oriented ERD. Together, these three diagrams offer comprehensive system understanding from multiple perspectives, supporting communication among stakeholders with different concerns and technical backgrounds.

References

- IBM. (2024). Cost of a Data Breach Report 2024.
<https://www.ibm.com/reports/data-breach>
- INTERPOL. (2020). INTERPOL report shows alarming rate of cyberattacks during COVID-19.
<https://www.interpol.int/en/News-and-Events/News/2020/INTERPOL-report-shows-alarming-rate-of-cyberattacks-during-COVID-19>
- Ahmed, M. et al. (2020). Cyber Security in the Age of COVID-19: A Timeline and Analysis of Cyber-Crime and Cyber-Attacks during the Pandemic.
<https://arxiv.org/pdf/2006.11929>
- Kim, D. et al. (2024). Classification and online clustering of zero-day malware. Journal of Computer Virology and Hacking Techniques.
<https://link.springer.com/article/10.1007/s11416-024-00513-5>
- Fidelis Security. (2025). Mastering Signature-Based Detection in Cybersecurity.
<https://fidelissecurity.com/threatgeek/network-security/signature-based-detection/>
- Lyda, R., & Hamrock, J. (2007). Using Entropy Analysis to Find Encrypted and Packed Malware. IEEE Security & Privacy.
<https://dl.acm.org/doi/abs/10.1109/MSP.2007.48>
- Sujon, M. et al. (2025). A novel framework for malware detection using entropy-based statistical features and machine learning models across file types. IOPscience. <https://iopscience.iop.org/article/10.1088/2631-8695/add645>
- Abrams, L. (2020). Hiding Windows File Extensions is a Security Risk, Enable Now. BleepingComputer.
<https://www.bleepingcomputer.com/news/microsoft/hiding-windows-file-extension-is-a-security-risk-enable-now/>
- ScienceDirect Topics. Heuristic Analysis — an overview.
<https://www.sciencedirect.com/topics/computer-science/heuristic-analysis>
- VirusTotal. How it works. <https://docs.virustotal.com/docs/how-it-works>

- Devoteam. (2025). Unveiling the AI-powered cybersecurity guardian: A deep dive into the evolution of VirusTotal.
<https://www.devoteam.com/expert-view/virustotal-threat-analysis/>
- VMRay. What is Dynamic Analysis?
<https://www.vmrays.com/glossary/dynamic-analysis/>
- Morphisec. (2025). Fileless Malware Evades Detection-Based Security.
<https://www.morphisec.com/blog/fileless-malware-attacks/>
- Wrike. What is Iterative and Incremental Development?
<https://www.wrike.com/project-management-guide/faq/what-is-iterative-incremental-development/>
- Ahmadi, S. et al. (2013). Comparative Study on Agile software development methodologies. <https://arxiv.org/pdf/1307.3356>
- Ally, A. Agile Methodology: A Comprehensive Guide to Incremental and Iterative Development. Medium.
<https://medium.com/@ahmed.ally2/agile-methodology-a-comprehensive-guide-to-incremental-and-iterative-development-2d0d22e37997>
- Ö. A. Aslan and R. Samet, "A Comprehensive Review on Malware Detection Approaches," in IEEE Access, vol. 8, pp. 6249-6271, 2020.
[A Comprehensive Review on Malware Detection Approaches | IEEE Journals & Magazine | IEEE Xplore](#)

- Kamdan; Pratama, Y.; Munzi, R.S.; Mustafa, A.B.; Kharisma, I.L. Eng. Proc. 2025, 107, 76.
[Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach](#)
- Museeb, A., Hamed, Y., Sokkalingam, R., Hamza, A.A., Ullah, A., & Khan, I.K. (2025). International Journal of Advanced Computer Science and Applications (IJACSA), 16(11).
<http://dx.doi.org/10.14569/IJACSA.2025.0161108>
- Abdon, A.L. (2024). International Journal of Intelligent Systems and Applications in Engineering, 12(4), 3430.
<https://ijisae.org/index.php/IJISAE/article/view/6858>
- Buradkar, Mrunalini U. NeuroQuantology, Vol. 20, Issue 10, 2022, pp. 4128-4141.
https://www.academia.edu/89154420/Static_Malware_Analysis_Using_Optimal_Machine_Learning_Algorithm_for_Malware_Detection
- Azeem, M., Khan, D., Iftikhar, S., Bawazeer, S., & Alzahrani, M. (2023). Heliyon, 9(12), e187275.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10770453/>
- Naeem, M.R., Amin, R., Farhan, M., Alotaibi, F.A., Alnfai, M.M., Sampedro, G.A., & Karovič, V. (2024). Journal of Big Data and Cognitive Computing, 8(2).
<https://pmc.ncbi.nlm.nih.gov/articles/PMC11419604/>
- Wee Ki Joon. SANS Internet Storm Center Guest Diary, March 26, 2025.
<https://isc.sans.edu/diary/31790>

- Mheboobkhan. Medium, February 13, 2019.
<https://whiteheart0.medium.com/entropy-analysis-a-critical-test-for-malwares-69939f5b8b1>
 - "Cybersecurity for All: Why Every Employee Matters in Data Protection."
TechClass Resources, April 25, 2025.
<https://www.techclass.com/resources/learning-and-development-articles/cybersecurity-for-non-tech-teams-why-everyone-plays-a-role-in-data-protection>
-

Appendices

Appendix A: Sample Intelligent Explanations

The following examples illustrate how Trustfile's intelligent explanation system generates customized, AI-like explanations based on specific scan results:

Example 1: High Entropy with Embedded Script

"This file has been classified as HIGH threat because our analysis detected multiple suspicious indicators. First, the file shows an unusually high entropy value of 7.9, which suggests it may be packed or encrypted to hide malicious content. We also detected embedded JavaScript code that attempts to connect to external servers when the file is opened. This behavior is commonly associated with trojan malware that downloads additional malicious components. Additionally, our AI analysis identified structural patterns similar to known malware families. We strongly recommend blocking this file and scanning your system with updated antivirus software."

Example 2: File Type Mismatch

"This file has been classified as MEDIUM threat due to a file type mismatch. The filename indicates it is a PDF document, but our analysis of the internal file structure reveals it is actually an executable program. This technique, called file extension spoofing, is commonly used to trick users into running malicious software disguised as harmless documents. The file does not contain known malware signatures, but the deliberate deception warrants caution. We recommend verifying the file source with the sender before opening, or requesting an alternative format."

Example 3: Normal File with Low Risk

"This file has been classified as LOW threat, indicating it appears safe for normal use. Our analysis found no malware signatures, normal entropy levels appropriate for its file type, no embedded scripts or suspicious code, and correct file structure matching its stated format. Our AI analysis assigns high confidence to this safe classification. While no file can be guaranteed completely safe, this file passes all our security checks and exhibits characteristics consistent with legitimate documents."

Appendix B: Supported File Extensions

Trustfile supports scanning of the following file extensions and formats:

Documents: PDF, DOC, DOCX, XLS, XLSX, PPT, PPTX, TXT, RTF, ODT, ODS, ODP

Archives: ZIP, RAR, 7Z, TAR, GZ, BZ2, XZ

Images: JPG, JPEG, PNG, GIF, BMP, TIFF, WebP, SVG

Audio/Video: MP3, WAV, MP4, AVI, MKV, MOV, WMV, FLV

Executables: EXE, DLL, SYS, COM, BAT, CMD, MSI

Web Files: HTML, HTM, JS, CSS, PHP, ASP, JSP

Data Files: XML, JSON, CSV, SQL, DB, MDB

Other: And all other file extensions through generic binary analysis

Appendix C: AI analysis Feature Set

The AI analysis system analyzes the following features extracted from uploaded files:

Entropy Features:

- Overall file entropy
- Section-level entropy values
- Entropy variance across file segments
- Maximum entropy segment identification

Structural Features:

- File header characteristics
- Section names and counts
- Import table entries
- Export table entries
- Resource section characteristics

Heuristic Indicators:

- Packing detection flags
- Encryption detection flags
- Suspicious API call patterns

- Obfuscation technique indicators
- Anomalous structural patterns

Metadata Features:

- File size relative to type norms
- Creation and modification timestamps
- Embedded metadata analysis
- Digital signature verification

API Intelligence:

- Hash match results
- Reputation scores
- Historical threat data

CHAPTER 4

SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION

Overview of the Developed System

TrustFile is a web-based malware analysis and file scanning application designed to help individuals and small organizations evaluate the safety of files before opening, executing, or distributing them. Rather than relying on a single detection method, TrustFile combines three complementary layers of analysis into a single scanning pipeline: a local heuristic engine that inspects file structure and behavioral indicators, an integration with the VirusTotal API that cross-references a file's hash and contents against dozens of commercial antivirus engines, and an AI-assisted explanation layer that translates raw technical findings into a plain-language summary a non-technical user can act on.

Unlike traditional antivirus software, which typically operates as a background system service, TrustFile is delivered as an accessible web application. Users authenticate, upload a file through the browser, and receive a consolidated risk assessment without needing to install or configure specialized security software. This lowers the barrier to entry for individuals and small teams who want file-level threat visibility but do not have dedicated IT security staff.

In addition to on-demand scanning, TrustFile includes a real-time folder monitoring feature. When enabled, the system observes selected locations on the host machine and automatically scans new or modified files as they appear, without requiring the user to manually initiate an upload. Files identified as critical threats can be automatically quarantined — moved to the operating system's recycle bin rather than left in place — reducing the window of exposure between detection and remediation.

The major objectives of the developed system are as follows:

- Automate the detection of malicious or suspicious files through a combination of heuristic and third-party threat-intelligence analysis.
- Provide plain-language, AI-generated explanations of scan results so that non-technical users can interpret findings without security expertise.
- Maintain a historical record of scanned files, risk scores, and detected indicators for later review and reporting.

- Support real-time, background monitoring of designated folders with automatic quarantine of high-risk files.
- Present analytical summaries — including severity breakdowns and 7-day scanning trends — through visual dashboards.
- Ensure secure, authenticated, per-user access to uploaded files and scan history.

The development of TrustFile follows modern secure software engineering practices, incorporating server-side authentication and session management, environment-based configuration of sensitive credentials, per-user data isolation, and a layered detection architecture that degrades gracefully if any single detection source (for example, the VirusTotal API) is temporarily unavailable.

The completed system consists of several interconnected modules that work together to accept, analyze, store, and present scan results.

Figure 4.1 illustrates the overall workflow of the developed system.

The workflow begins with user authentication. Once authenticated, user may upload a file , select a scan mode single scan or all folder scan and view consolidated results combining heuristic findings, Virustotal detections, and an AI-generated explanation. Historical results are accessible through the History and Reports modules. While system-wide monitoring behavior is configured through the settings module.

System Development Methodology

The development of the Trustfile adopted the Agile Software Development Methodology, an interactive approach that emphasizes incremental delivery, continuous testing and regular refirement based on functional feedback . Agile was selected because the systems detection logic-particulary the heuristic engine-required repeated tuning as new false positives and edge cases where identified-during hands-on testing which a strictly sequential development model would not accommodated effectively.

The development process consisted of the following phases:

Requirements Gathering

The initial phase focused on identifying the core problem Trustfile addresses: the lack of an accessible, consolidated way for individuals without dedicated security tooling to evaluate whether a file safe to open. Requirements were shaped by considering common attack patterns worth detecting-code execution, process spawning,reverse shell behavior, data exfiltration, and obfuscated payloads-alongside the practical need for result the non-technical users could understand.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS